

Dojo dei Sistemi Operativi v6

La Grande Pergamena dei Sistemi Operativi (Edizione Completa e Potenziata)

Forza e Onore. Il tuo viaggio verso la maestria continua, e ogni passo ti rende più forte.

Davanti a te hai una mappa interattiva e completa del regno dei Sistemi Operativi. Ogni concetto è una città, ogni link un sentiero. Non avere fretta. Esplora un territorio alla volta.

Mappa della Pergamena (Indice)

- **Sezione I:** [I. I Fondamenti del Sistema](#)
- **Sezione II:** [II. Gestione dei Processi e dei Thread](#)
- **Sezione III:** [III. Scheduling della CPU](#)
- **Sezione IV:** [IV. Sincronizzazione e Concorrenza](#)
- **Sezione V:** [V. Deadlock: L'Abbraccio Mortale](#)
- **Sezione VI:** [VI. Gestione della Memoria](#)
- **Sezione VII:** [VII. Gestione dell'Archiviazione e I/O](#)
- **Sezione VIII:** [VIII. File System](#)
- **Sezione IX:** [IX. Esercizi e Esempi Pratici](#)

I. I Fondamenti del Sistema

Cos'è un Sistema Operativo?

Analogia: Il Manager del Ristorante

- **Hardware** (CPU, RAM, Disco): La cucina, i fornelli, il frigorifero.
- **Applicazioni** (Chrome, Word, un gioco): I clienti che fanno le ordinazioni.
- Il **Sistema Operativo**: È il **manager del ristorante**. Non cucina, ma prende le ordinazioni (comandi), gestisce i cuochi (la CPU), assegna i tavoli (la memoria) e si assicura che tutto funzioni in modo efficiente e senza conflitti.

Una definizione "pratica" di Sistema Operativo (Silberschatz et al.)

Il prof. Finzi spesso cita il Silberschatz, e una delle definizioni più dirette e "terra terra" di cosa sia un sistema operativo è:

"Everything a vendor ships when you order an operating system."

Questa frase, apparentemente semplice, cattura perfettamente la vastità di un sistema operativo moderno. Non è solo il kernel, ma include anche tutti i programmi di sistema, le utilità, le librerie, le interfacce grafiche e tutto ciò che rende il computer utilizzabile. È l'intero pacchetto che ricevi quando compri Windows, macOS o installi una distribuzione Linux!

Le sue funzioni principali sono:

- **Gestore di Risorse**: Arbitra e alloca le risorse del computer (CPU, memoria, dischi).
- **Programma di Controllo**: Supervisiona l'esecuzione dei programmi utente per prevenire errori e usi impropri.
- **Fornitore di Interfaccia**: Offre un'interfaccia (CLI, GUI) con cui l'utente può interagire.
- **Fornitore di Servizi**: Mette a disposizione servizi comuni tramite [System Call](#).

- **Contabilità:** Tiene traccia dell'utilizzo delle risorse del computer (cicli CPU, memoria, I/O, file) da parte di utenti e programmi, utile per la fatturazione o l'analisi delle prestazioni.

🔗 Concetto di "Sistema Operativo come Macchina Estesa"

Hai notato un punto chiave! Oltre a essere un gestore di risorse, il sistema operativo funge anche da **"macchina estesa" (extended machine)**. Questo significa che astrae la complessità dell'hardware sottostante, fornendo agli utenti e ai programmi un ambiente più "bello", semplice e facile da usare. Invece di dover interagire direttamente con registri hardware, interrupt e indirizzi fisici, il SO offre un set di astrazioni di alto livello (come file, processi, memoria virtuale) che rendono la programmazione e l'interazione molto più intuitive.

Esempio/Caso d'uso: Il SO trasforma l'interazione con registri e istruzioni macchina in operazioni intuitive come **apri un file**, **stampa un documento** o **avvia un programma**, nascondendo tutti i dettagli complessi di basso livello.

👑 Modalità Kernel e Modalità Utente (Dual Mode)

Questa è una delle idee più importanti! Per proteggere il sistema, l'SO divide il mondo in due "livelli di potere".

🔗 Definition

- **Modalità Utente (User Mode):** È la modalità normale in cui girano i programmi. Ha poteri limitati.
- **Modalità Kernel (Kernel Mode):** È la modalità privilegiata in cui gira il cuore dell'SO (il **kernel**). Ha accesso completo a tutto l'hardware.

Caratteristica	Modalità Utente (Il Cliente al Tavolo)	Modalità Kernel (Il Manager in Cucina)	
Chi ci gira?	Applicazioni (Chrome, giochi, etc.)	Codice del Sistema Operativo	
Accesso Hardware?	NO . Non può accedere direttamente.	SI . Controllo totale.	
Sicurezza	Alta. Un'app che crasha non blocca il PC.	Bassa. Un errore qui può bloccare tutto.	

❓ Perché questa divisione è così cruciale per la stabilità di un sistema moderno? Cosa succederebbe se il tuo browser potesse disabilitare gli interrupt o modificare la page table di un altro processo?

Protezione della Memoria (Registri Base e Limite)

Ottimo, hai notato che mancava un dettaglio fondamentale su come l'hardware supporta la protezione in modalità utente!

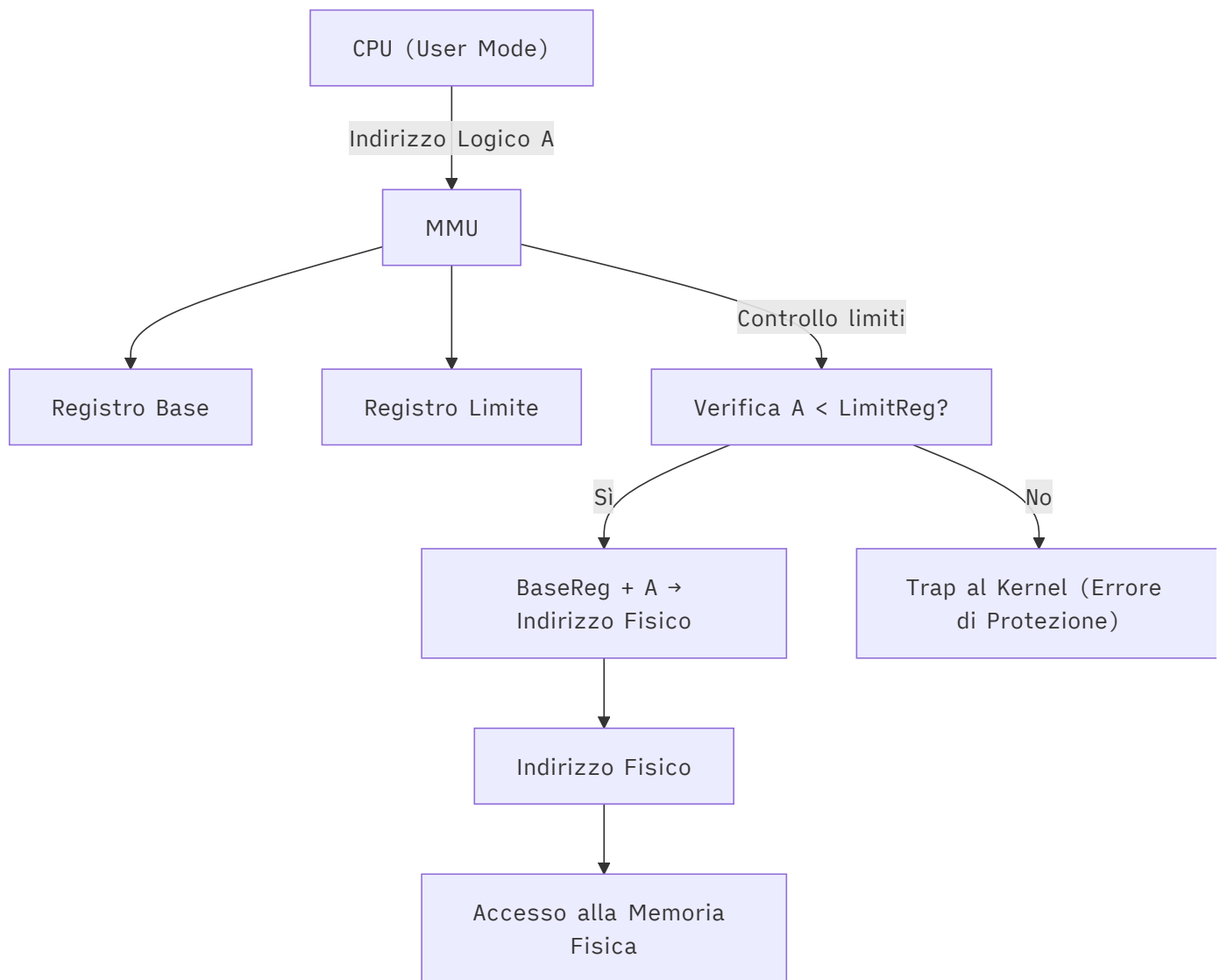
🔗 I **registri base e limite** sono una coppia di registri della CPU che definiscono lo spazio degli indirizzi logici accessibili a un processo in modalità utente.

- Il **registro base** contiene l'indirizzo fisico iniziale più basso a cui un processo può accedere.
- Il **registro limite** specifica la dimensione (o il range) degli indirizzi consentiti a partire dall'indirizzo base.

Come funziona:

Ogni volta che la CPU genera un indirizzo in modalità utente, l'hardware (la **MMU**) controlla che questo indirizzo ricada all'interno del range definito da questi registri.

- L'indirizzo logico **A** viene sommato al valore del registro **base** per ottenere l'indirizzo fisico.
- Contemporaneamente, l'hardware verifica che **A** sia minore del valore del registro **limite**.



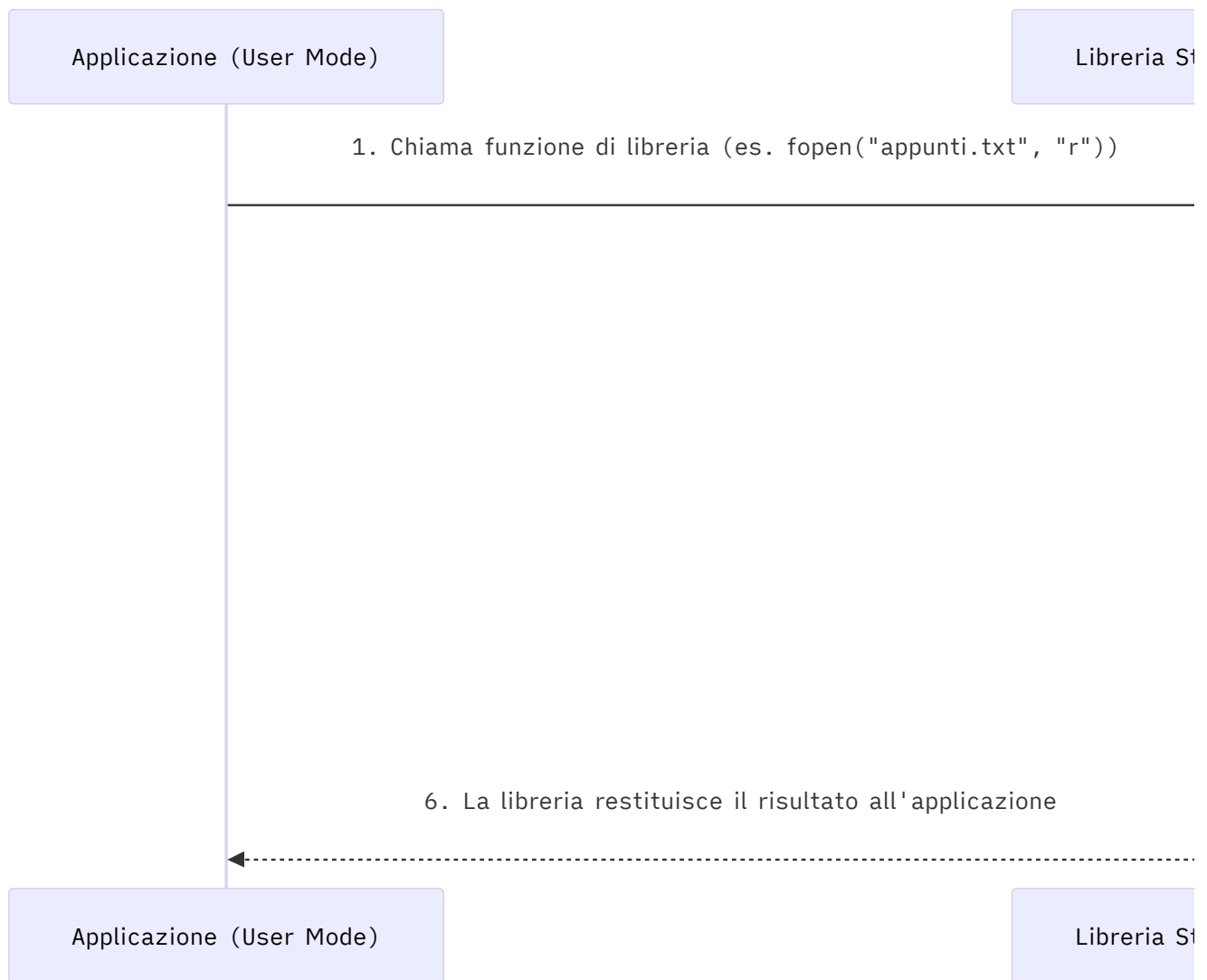
Se un processo tenta di accedere a un indirizzo al di fuori di questo range (cioè $A \geq \text{limite}$), viene generato un **trap hardware** al kernel. Il kernel, a sua volta, tipicamente termina il processo per prevenire accessi non autorizzati o corruzione della memoria. Questo meccanismo impedisce ai processi utente di interferire tra loro o di accedere a aree di memoria privilegiate del kernel, garantendo la stabilità e la sicurezza del sistema.

Il dispatcher, durante un context switch, è responsabile di caricare i valori appropriati di questi registri per il nuovo processo che sta per essere eseguito.

System Call (Chiamate di Sistema)

 Una **System Call** è il meccanismo con cui un programma in Modalità Utente chiede un servizio al kernel. È l'unica porta d'ingresso sicura al "mondo privilegiato" della Modalità Kernel.

È come il menù di un ristorante: una lista predefinita di cose che puoi chiedere. Non puoi entrare in cucina, ma puoi ordinare "leggi un file" o "scrivi sullo schermo".



API (Application Programming Interface)

Le System Call sono l'interfaccia a basso livello fornita dal sistema operativo. Tuttavia, i programmatori raramente le usano direttamente. Invece, usano **API (Application Programming Interface)** che forniscono un'interfaccia più astratta e user-friendly. Una singola funzione API può invocare una o più System Call.

- **WinAPI (Windows API)**: L'API nativa per lo sviluppo di applicazioni su sistemi Microsoft Windows. Fornisce funzioni per la gestione di finestre, processi, file, rete, ecc. (es. `CreateFile`, `ReadFile`, `WriteFile`).
- **POSIX API (Portable Operating System Interface)**: Uno standard per le API che definiscono l'interfaccia tra programmi e sistemi operativi Unix-like. Permette di scrivere codice portabile tra diversi sistemi Unix (es. `open`, `read`, `write`, `fork`, `pthread_create`). La **GNU C Library (glibc)** è l'implementazione più comune delle API POSIX in Linux.
- **Java API**: L'API standard per lo sviluppo di applicazioni Java. Le sue classi e metodi (es. `java.io.File`, `java.lang.Process`) a loro volta invocano le System Call del sistema operativo sottostante tramite la Java Virtual Machine (JVM).

Passaggio dei Parametri alle System Call

Quando un'applicazione invoca una System Call, deve passare dei parametri al kernel. Questo può avvenire in tre modi principali:

1. **Tramite Registri**: Il metodo più semplice. I parametri vengono passati direttamente nei registri della CPU. Funziona bene per un numero limitato di parametri.
 - **Architettura x64 (Linux System V AMD64 ABI)**: Per le system call in Linux su architetture x64, i parametri vengono passati in registri specifici in un ordine predefinito:

- `rax` : Contiene il numero della system call. Al ritorno, contiene il valore di ritorno della system call.
- `rdi` : Primo argomento (arg0)
- `rsi` : Secondo argomento (arg1)
- `rdx` : Terzo argomento (arg2)
- `r10` : Quarto argomento (arg3)
- `r8` : Quinto argomento (arg4)
- `r9` : Sesto argomento (arg5)

Se ci sono più di sei argomenti, i rimanenti vengono passati tramite lo stack.

2. **Tramite un Blocco in Memoria:** Se i parametri sono molti o sono strutture dati complesse, l'applicazione può metterli in un blocco di memoria e passare l'indirizzo di questo blocco in un registro. Il kernel accede poi a questa area di memoria.
3. **Tramite lo Stack:** I parametri vengono "pushati" sullo stack del processo. Il kernel li preleva dallo stack. Questo metodo è flessibile e non limita il numero di parametri.

Tipi di Chiamate di Sistema

Le System Call possono essere raggruppate in categorie in base al servizio che forniscono:

- **Controllo Processi:**
 - Creazione/Terminazione processi (`fork()` , `exec()` , `exit()` , `wait()`).
 - Caricamento/Esecuzione programmi.
 - Controllo degli attributi dei processi (`getpid()` , `setpriority()`).
 - Comunicazione interprocesso (`pipe()` , `shm_open()`).
- **Gestione File:**
 - Creazione/Cancellazione file (`create()` , `delete()`).
 - Apertura/Chiusura file (`open()` , `close()`).
 - Lettura/Scrittura file (`read()` , `write()`).
 - Posizionamento nel file (`seek()`).
 - Controllo attributi file (`stat()` , `chmod()`).
- **Gestione Dispositivi:**
 - Richiesta/Rilascio dispositivi.
 - Lettura/Scrittura su dispositivi (`read()` , `write()` generiche).
 - Controllo attributi dispositivi (`ioctl()`).
- **Gestione Informazione:**
 - Ottenere/Impostare data e ora.
 - Ottenere informazioni di sistema (versione SO, memoria libera).
 - Ottenere/Impostare attributi di processo/file.
- **Comunicazione:**
 - Creazione/Cancellazione connessioni.
 - Invio/Ricezione messaggi (`send()` , `recv()`).
 - Attacco/Distacco di regioni di memoria condivisa.
- **Protezione e Sicurezza:**
 - Controllo degli accessi ai file e alle risorse.
 - Autenticazione utente.



Programmi di Sistema

Programmi di Sistema come Categoria Distinta

Fantastico, hai colto un altro aspetto fondamentale! Oltre alle [System Call](#) che sono l'interfaccia a basso livello per il kernel, esistono i **programmi di sistema**. Questi sono utilità fornite con il sistema operativo che facilitano lo sviluppo, la gestione e l'interazione dell'utente con il sistema stesso. La maggior parte degli utenti interagisce più con questi programmi che direttamente con le system call.

Esempi:

- **Manipolazione File:** `cp` (copia), `mv` (sposta/rinomina), `rm` (cancella), `mkdir` (crea directory), `rmdir` (cancella directory), `ls` (elenca file), `cat` (visualizza contenuto file).
- **Informazioni sullo Stato:** `ps` (processi in esecuzione), `top` (monitoraggio risorse), `df` (spazio disco), `date` (data/ora).
- **Modifica File:** Editor di testo come `vi`, `nano`, `gedit`.
- **Programmazione:** Compilatori (`gcc`), debugger (`gdb`), make utility (`make`).
- **Comunicazione:** `ssh`, `ftp`, `ping`.

Comandi come `ls`, `cat`, `ps` (già presenti nei tuoi esercizi) sono esempi perfetti di programmi di sistema che interfacciano le system call in modo user-friendly. Questi programmi costruiscono un ponte tra le funzionalità grezze del kernel e le esigenze pratiche dell'utente.

⚡ Interrupts e Polling

✍ Un **interrupt** è un segnale inviato da un dispositivo hardware o software alla CPU per notificare che un evento richiede attenzione immediata.

🔗 Analogia: Il Cuoco e il Timer del Forno

Immagina un cuoco (la **CPU**) che sta tagliando le verdure (**processo**).

1. Mette un arrosto nel forno (**operazione di I/O**) e imposta un timer.
2. While l'arrosto cuoce, il cuoco non aspetta, ma continua a tagliare le verdure (la CPU è efficiente!).
3. Quando il timer suona (**l'interrupt!**), il cuoco smette di tagliare.
4. Mette da parte il coltello in modo ordinato (**salva lo stato** del processo).
5. Toglie l'arrosto dal forno (**esegue la routine di servizio dell'interrupt**).
6. Torna al suo tagliere, riprende il coltello (**ripristina lo stato**) e continua da dove aveva interrotto.

Meccanismo	Descrizione	Pro	Contro
Interrupt	Il dispositivo notifica la CPU quando è pronto.	Efficiente. La CPU è libera di fare altro.	Leggero overhead per il context switch.
Polling	La CPU controlla continuamente lo stato del dispositivo.	Semplice.	Inefficiente. Spreca cicli di CPU in attesa.

Vettore delle Interruzioni

Quando arriva un interrupt, come fa la CPU a sapere cosa fare? Usa il **vettore delle interruzioni**.

✍ Il **Vettore delle Interruzioni** è una tabella in memoria che associa un numero di interrupt all'indirizzo della routine di codice (ISR - Interrupt Service Routine) che deve gestirlo.

Flusso:

1. Un dispositivo (es. tastiera) invia un segnale di interrupt con un numero (es. `IRQ 1`).
2. La CPU ferma il lavoro corrente e salva il suo stato.
3. Usa il numero `1` come indice per cercare nel Vettore delle Interruzioni.
4. Trova l'indirizzo della ISR per la tastiera.
5. Salta a quell'indirizzo ed esegue la ISR (es. legge il carattere dal buffer della tastiera).
6. Al termine della ISR, ripristina lo stato del lavoro interrotto e continua.

Tipi di Interruzione

Le interruzioni possono essere classificate in base alla loro mascherabilità:

- **Interruzioni Non Mascherabili (Non-Maskable Interrupts - NMI):**
 - **Descrizione:** Sono interruzioni hardware di massima priorità che non possono essere ignorate o disabilitate dalla CPU.

- **Scopo:** Utilizzate per eventi critici del sistema, come errori hardware gravi (es. errore di parità della memoria, malfunzionamento della CPU) che richiedono un'azione immediata per prevenire danni o perdita di dati.
- **Gestione:** Il sistema operativo deve rispondere immediatamente a queste interruzioni.
- **Range Intel:** Nei processori Intel, gli interrupt non mascherabili sono tipicamente assegnati ai vettori da **0 a 31**.
- **Interruzioni Mascherabili (Maskable Interrupts):**
 - **Descrizione:** Sono interruzioni hardware o software che possono essere abilitate o disabilitate dalla CPU (mascherate) tramite un registro specifico (es. Interrupt Mask Register - IMR).
 - **Scopo:** Utilizzate per la maggior parte delle operazioni I/O (es. tastiera, disco, rete) e per gli interrupt di clock.
 - **Gestione:** Il kernel può mascherare queste interruzioni per brevi periodi critici (es. quando sta aggiornando strutture dati condivise) per garantire l'atomicità delle operazioni.
 - **Range Intel:** Nei processori Intel, gli interrupt mascherabili sono tipicamente assegnati ai vettori da **32 a 255**.

Interrupt Chaining

✍ **L'Interrupt Chaining** (o "catena di interrupt") è una tecnica usata quando più dispositivi condividono la stessa linea di interrupt (IRQ). Invece di avere una ISR dedicata per ogni IRQ, il sistema operativo associa una lista (una "catena") di ISR a un singolo IRQ.

Come funziona:

Quando un interrupt arriva su una linea condivisa, la CPU invoca la prima ISR nella catena. Questa ISR controlla se è stata lei a generare l'interrupt. Se sì, lo gestisce. Se no, passa il controllo alla successiva ISR nella catena, e così via, finché non viene trovata la ISR corretta o la catena termina.

Controllori Hardware di Interruzione Avanzati

I sistemi moderni utilizzano controller di interruzione più sofisticati per gestire un gran numero di dispositivi e ottimizzare la consegna degli interrupt ai core della CPU:

- **APIC (Advanced Programmable Interrupt Controller):**
 - **Descrizione:** Utilizzato nelle architetture Intel x86, l'APIC è un'architettura di controller di interrupt che permette di distribuire gli interrupt in modo efficiente tra i vari core di una CPU multi-core o tra più CPU in un sistema multiprocessore.
 - **Componenti:** Include un **Local APIC (LAPIC)** per ogni core (che gestisce gli interrupt locali e riceve quelli da altri LAPIC) e un **I/O APIC (IOAPIC)** che riceve gli interrupt dai dispositivi I/O e li instrada ai LAPIC.
 - **Vantaggi:** Migliora le prestazioni e la scalabilità dei sistemi multi-core, permettendo un bilanciamento del carico degli interrupt.
- **GIC (Generic Interrupt Controller):**
 - **Descrizione:** L'equivalente dell'APIC nelle architetture ARM. Il GIC è uno standard per la gestione degli interrupt in sistemi basati su ARM, anch'esso progettato per ambienti multi-core.
 - **Funzionalità:** Simile all'APIC, instrada gli interrupt dai dispositivi I/O ai core appropriati, supporta la priorità degli interrupt e la loro mascherabilità.

Processo di Boot del Sistema e Fasi di Avvio

Come si avvia un computer?

1. **Accensione e BIOS/UEFI:** Viene eseguito il codice del **BIOS/UEFI** (Basic Input/Output System / Unified Extensible Firmware Interface) da una memoria non volatile sulla scheda madre.
2. **POST (Power-On Self-Test):** Il BIOS/UEFI controlla che l'hardware di base funzioni (RAM, CPU, schede).
3. **Ricerca del Bootloader:** Il BIOS/UEFI cerca un dispositivo di avvio (es. SSD, HDD) e carica il primo settore, il **Master Boot Record (MBR)** o usa la **GUID Partition Table (GPT)** per trovare il **bootloader**.

(es. GRUB in Linux).

- Il **Boot Block** è il primo blocco di una partizione, contenente il codice necessario per avviare il sistema operativo da quella partizione.
4. **Caricamento del Kernel**: Il bootloader carica in memoria il file del **kernel** del sistema operativo e gli passa il controllo.
 5. **Avvio del Kernel**: Il kernel inizializza i driver, le strutture dati (es. scheduler, gestione memoria) e avvia il primo processo in user-space, chiamato **init** (o **systemd**), che a sua volta avvia tutti gli altri servizi e il login dell'utente.
 - Il **Superblock** (di cui parleremo meglio nel File System) è cruciale in questa fase, perché contiene le informazioni essenziali per montare il file system root.

Dettagli su NTLDR e GRUB

- **NTLDR (NT Loader)**: Il bootloader primario per le versioni più vecchie di Windows (fino a Windows XP e Windows Server 2003). Gestiva il caricamento del kernel di Windows e permetteva la selezione tra più sistemi operativi.
- **GRUB (Grand Unified Bootloader)**: Il bootloader più comune nei sistemi Linux. È molto flessibile, supporta il multiboot (permette di scegliere tra diversi sistemi operativi installati) e può caricare kernel Linux da diverse posizioni.

Tipi di Disco (Breve Panoramica)

Quando parliamo di "dischi", ci riferiamo principalmente a due tecnologie:

- **HDD (Hard Disk Drive)**: Dischi meccanici, con piatti rotanti e testine magnetiche. Sono più lenti, ma offrono grande capacità a basso costo. Il loro accesso è dominato dal **seek time** (tempo per muovere la testina).
- **SSD (Solid State Drive)**: Dischi basati su memoria flash. Sono molto più veloci, non hanno parti mobili, ma sono più costosi per GB.

Progettazione e Implementazione di un SO

Dettagli sulla "Progettazione e Implementazione di un SO"

Hai fatto benissimo a chiedere di aggiungere questo! Capire gli obiettivi e le scelte dietro la progettazione di un sistema operativo ti dà una visione più completa.

La progettazione di un sistema operativo è un compito complesso che bilancia diverse esigenze e compromessi. Gli obiettivi principali si dividono in due categorie:

1. Obiettivi Utente:

- **Comodità e Facilità d'uso**: Il SO deve essere intuitivo e semplice da usare per l'utente finale.
- **Affidabilità**: Deve essere stabile e funzionare correttamente senza crash frequenti.
- **Sicurezza**: Deve proteggere i dati e le risorse da accessi non autorizzati.
- **Velocità/Reattività**: Deve rispondere rapidamente alle richieste dell'utente e delle applicazioni.

2. Obiettivi di Sistema:

- **Facilità di Progettazione e Implementazione**: Il codice deve essere modulare e ben strutturato per facilitare lo sviluppo.
- **Manutenzione**: Deve essere facile da aggiornare, correggere bug e aggiungere nuove funzionalità.
- **Flessibilità**: Deve potersi adattare a diverse configurazioni hardware e a diversi carichi di lavoro.
- **Efficienza**: Deve utilizzare le risorse hardware in modo ottimale.

Linguaggi di Implementazione:

Storicamente, i primi sistemi operativi erano scritti quasi interamente in **assembly** per massimizzare le prestazioni e il controllo sull'hardware. Tuttavia, questo rendeva lo sviluppo e la manutenzione estremamente difficili.

Oggi, la maggior parte dei kernel dei sistemi operativi moderni (come Linux, Windows) è scritta principalmente in **C** e **C++**. Questi linguaggi offrono un buon equilibrio tra controllo a basso livello (necessario per interagire con l'hardware) e produttività del programmatore. Alcune parti critiche per le prestazioni o per l'interazione diretta con l'hardware possono ancora essere scritte in assembly. Le utilità e i programmi di sistema (come la shell o gli script di avvio) sono spesso scritti in linguaggi di scripting (es. Python, Bash) o altri linguaggi di alto livello.

Trade-off: Performance vs. Portabilità:

La scelta del linguaggio e dell'architettura del kernel spesso implica un compromesso. Il codice in assembly è estremamente performante ma non è portabile (funziona solo su una specifica architettura CPU). Il codice in C/C++ è meno performante dell'assembly puro ma molto più portabile, permettendo al sistema operativo di girare su diverse architetture (x86, ARM, ecc.) con modifiche minime. I sistemi operativi bilanciano questi fattori per raggiungere i loro obiettivi.

Architettura Monolitica del Kernel

✍ Un **kernel monolitico** è un'architettura di sistema operativo in cui l'intero sistema operativo (gestione processi, memoria, I/O, file system, driver di dispositivo) è un singolo, grande programma che gira interamente in modalità kernel.

✍ Pensa a un kernel monolitico come a un **enorme e unico edificio** che contiene tutti gli uffici e i servizi della città. È tutto sotto lo stesso tetto, gestito da un'unica amministrazione centrale.

Caratteristica	Descrizione	Pro	Contro
Struttura	Tutti i servizi del SO sono parte di un unico blocco di codice eseguibile.	Velocità ed Efficienza: La comunicazione tra i moduli è molto rapida (semplici chiamate a funzione).	Difficile da sviluppare/debuggare: Un bug in un modulo può compromettere l'intero sistema.
Driver	I driver di dispositivo sono parte integrante del kernel.		Meno modulare: Aggiungere nuove funzionalità o driver richiede spesso la ricompilazione dell'intero kernel.
Stabilità			Meno robusto: Un errore in un driver può far crashare l'intero sistema.

Esempi di SO con kernel monolitico: Linux, Unix tradizionale, MS-DOS.

Architettura a Microkernel

Ottimo, questa è una distinzione fondamentale per capire le diverse filosofie di design dei SO!

✍ Un **microkernel** è un'architettura di sistema operativo che mantiene nel kernel solo le funzioni più essenziali e fondamentali (gestione dei processi e dei thread, comunicazione inter-processo - IPC, gestione della memoria di base). Tutti gli altri servizi del sistema operativo (come driver di dispositivo, file system, network stack) vengono eseguiti come processi separati in **modalità utente**.

✍ Se il kernel monolitico era un unico, grande edificio, il microkernel è come un **piccolo centro di controllo** che gestisce solo le comunicazioni e le fondamentali, mentre tutti gli altri uffici e servizi (driver, file system) sono in **edifici separati** (processi utente). Se un ufficio ha un problema, non crolla l'intero centro.

Caratteristica	Descrizione	Pro	Contro
Struttura	Solo i servizi essenziali nel kernel. Il resto in user-space.	Maggiore Modularità: Facile aggiungere/rimuovere servizi.	Meno Performante: L'overhead delle comunicazioni (IPC) tra i moduli è elevato.
Driver	I driver di dispositivo sono processi in user-space.	Maggiore Stabilità/Sicurezza: Un driver che crasha non blocca l'intero sistema.	Le chiamate a funzione sono sostituite da messaggi IPC, che sono più lenti.
Dimensioni	Kernel molto più piccolo.	Più facile da sviluppare/debuggare: Codice più ridotto e isolato.	

Esempi di SO con microkernel: Mach (base per macOS), QNX, Minix.

Architettura a Strati (Layered Systems)

Fantastico, hai notato che mancava un'altra architettura importante! L'architettura a strati è stata una delle prime idee per organizzare i sistemi operativi complessi.

 Un sistema operativo con **architettura a strati (layered system)** è organizzato in una gerarchia di livelli. Ogni strato è costruito sopra a quello inferiore e utilizza solo le funzioni e i servizi forniti dallo strato immediatamente sottostante. Il livello più basso (strato 0) interagisce direttamente con l'hardware, mentre il livello più alto (strato N) fornisce l'interfaccia utente.

 Pensa a una torta a più piani. Ogni piano (strato) è fatto con ingredienti e tecniche che si basano solo sul piano sottostante. Non puoi mettere la glassa prima di aver cotto la base!

Come funziona:

- **Strato 0 (Hardware)**: Gestisce l'hardware fisico.
- **Strato 1 (CPU Scheduling)**: Gestisce la CPU.
- **Strato 2 (Gestione Memoria)**: Gestisce la memoria.
- ...
- **Strato N (Interfaccia Utente)**: L'interfaccia con cui l'utente interagisce (shell, GUI).

Caratteristica	Descrizione	Pro	Contro
Struttura	Gerarchia di livelli, ogni strato usa solo i servizi del livello inferiore.	Semplifica la progettazione e il debug : Ogni strato può essere sviluppato e testato isolatamente.	Meno efficiente : Ogni richiesta può dover attraversare molti strati, con un overhead significativo per ogni passaggio.
Dipendenze	Dipendenze chiare e unidirezionali.	Facilita la manutenzione : Le modifiche a uno strato non impattano gli strati superiori, purché l'interfaccia non cambi.	Difficile definire gli strati : Non sempre è chiaro come suddividere le funzionalità in strati ben definiti.

Esempi storici: THE multiprogramming system, OS/2.

Architettura Ibrida (Hybrid Systems)

Ottimo, hai notato che mancava l'architettura più comune nei sistemi operativi moderni!

 Un' **architettura ibrida** (o "kernel ibrido") combina elementi delle architetture monolitiche e microkernel (e talvolta anche a strati) per ottenere un equilibrio tra prestazioni, modularità e stabilità. L'obiettivo è sfruttare i vantaggi di entrambi gli approcci, minimizzando gli svantaggi.

 Immagina un ristorante moderno. Non è una singola, enorme cucina dove tutti fanno tutto (monolitico), né una serie di piccoli chioschi indipendenti che comunicano solo con un centro di controllo (microkernel). Invece, hai una cucina centrale efficiente per le operazioni più critiche (kernel), ma con sezioni modulari e ben definite per compiti specifici (driver, file system) che possono essere aggiunte o rimosse facilmente, e che comunicano in modo efficiente con il nucleo.

Come funziona:

- Il **kernel** (che gira in modalità privilegiata) contiene le funzionalità più critiche e sensibili alle prestazioni, come la gestione dei processi, la schedulazione, la gestione della memoria di base e una parte del sottosistema I/O.
- Molti **driver di dispositivo** e **moduli del file system** possono essere caricati e scaricati dinamicamente (come moduli), senza dover ricompilare l'intero kernel. Questo offre modularità e

stabilità: un bug in un modulo caricabile dinamicamente è meno probabile che faccia crashare l'intero sistema rispetto a un bug in un kernel monolitico puro.

- La comunicazione tra i componenti del kernel è efficiente (chiamate a funzione), ma le interfacce tra i moduli sono ben definite per mantenere un certo grado di isolamento.

Vantaggi:

- **Prestazioni elevate:** Le funzioni critiche sono nel kernel, minimizzando l'overhead delle comunicazioni inter-processo (IPC).
- **Modularità e Flessibilità:** Permette di aggiungere o rimuovere funzionalità (es. driver) senza dover ricompilare l'intero kernel.
- **Stabilità migliorata:** Un errore in un modulo caricabile dinamicamente è meno probabile che comprometta l'intero sistema rispetto a un kernel monolitico puro.
- **Adattabilità:** Possono essere ottimizzati per diversi carichi di lavoro.

Esempi di SO con Architettura Ibrida:

- **Linux:** Sebbene spesso classificato come monolitico, il kernel Linux è altamente modulare e permette il caricamento dinamico di driver e file system, rendendolo di fatto un ibrido.
- **Windows NT (e successivi Windows):** Il kernel di Windows è un esempio classico di architettura ibrida, con un "Executive" che gira in kernel mode e servizi come il gestore I/O, il gestore della memoria virtuale e il gestore dei processi.



Meccanismo vs Politica



La separazione tra **meccanismo** e **politica** è un principio fondamentale nella progettazione dei sistemi operativi.

- Il **meccanismo** definisce **come** fare qualcosa (l'implementazione tecnica).
- La **politica** definisce **cosa** fare (la decisione su quale meccanismo usare o come configurarlo).



Pensa a un'auto.

- Il **meccanismo** è il motore, il cambio, il volante: ti permettono di accelerare, frenare, girare.
- La **politica** è il modo in cui tu, il guidatore, decidi di usare quei meccanismi: se accelerare forte, frenare dolcemente, o svoltare a destra o sinistra.

Perché è importante?

- **Flessibilità:** Permette di cambiare le politiche senza dover riscrivere i meccanismi di base. Ad esempio, un SO può avere un meccanismo per lo scheduling (es. la gestione delle code di processi), ma diverse politiche (es. FCFS, SJF, RR) che decidono quale processo scegliere.
- **Adattabilità:** Il SO può adattarsi a diverse esigenze (es. un server ha politiche diverse da un desktop) semplicemente cambiando le politiche.



Comunicazione tra Componenti Segregati

La domanda su "come la memoria segregata comunica con il resto" è un ottimo modo per collegare diversi concetti che abbiamo già visto. Nel contesto dei Sistemi Operativi, "memoria segregata" o "componenti isolati" si riferisce a diverse forme di separazione che il sistema implementa per sicurezza e stabilità. La comunicazione tra queste parti avviene tramite meccanismi specifici:

- **Tra Modalità Utente e Modalità Kernel:** La comunicazione avviene esclusivamente tramite [System Call](#). Questo garantisce che i programmi utente non possano accedere direttamente a risorse privilegiate o danneggiare il kernel.
- **Tra Processi (con spazi di memoria separati):** La comunicazione avviene tramite [meccanismi di Inter-Process Communication \(IPC\)](#) come pipe, socket, code di messaggi o memoria condivisa. Ogni processo ha il suo spazio di indirizzamento logico isolato.
- **In Architetture NUMA (Nodi di Memoria non Uniformi):** La "segregazione" è fisica. Ogni processore ha una memoria locale a cui accede più velocemente, e una memoria remota (associata ad altri

processori) a cui accede più lentamente. La comunicazione tra questi nodi avviene tramite l'interconnessione del sistema, e lo scheduler cerca di ottimizzare l'[affinità](#) dei processi per minimizzare gli accessi alla memoria remota.

In tutti questi casi, l'obiettivo è bilanciare isolamento e sicurezza con la necessità di comunicazione e collaborazione tra le diverse parti del sistema.

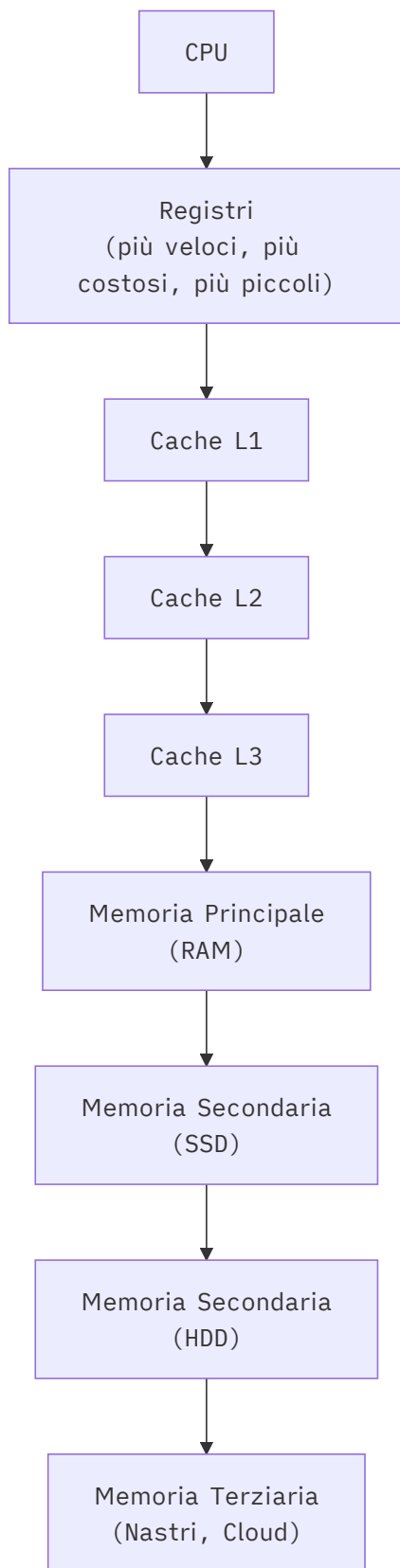
Gerarchie di Memorie

La gestione della memoria è un pilastro dei Sistemi Operativi, e la sua efficienza dipende da come le diverse "velocità" di memoria sono organizzate.

✍ La **gerarchia di memorie** è un'organizzazione a livelli dei diversi tipi di memoria in un sistema informatico, basata sulla loro velocità, costo e capacità. I livelli più vicini alla CPU sono più veloci e costosi ma di capacità inferiore, mentre quelli più lontani sono più lenti ed economici ma di capacità maggiore.

✍ Pensa a una cucina di un ristorante molto efficiente:

1. **Registri (I Cuochi)**: Sono i cuochi stessi, che tengono in mano gli ingredienti su cui stanno lavorando *proprio in questo istante*. Velocissimi, ma possono tenere pochissimo.
2. **Cache (Il Banco da Lavoro)**: È il banco da lavoro vicino ai cuochi, dove tengono gli ingredienti che useranno *molto presto*. Veloce, ma spazio limitato.
3. **Memoria Principale (RAM) (La Dispensa della Cucina)**: È la dispensa principale della cucina, dove sono tutti gli ingredienti necessari per i piatti che si stanno preparando. Abbastanza veloce, capacità media.
4. **Memoria Secondaria (Disco Rigido/SSD) (Il Magazzino del Ristorante)**: È il magazzino del ristorante, dove sono stoccati tutti gli ingredienti che potrebbero servire in futuro. Lenta, ma capacità enorme.
5. **Memoria Terziaria (Nastri/Cloud) (Il Magazzino Centrale del Fornitore)**: È il magazzino centrale del fornitore, dove sono conservati ingredienti rari o di scorta. Molto lenta per l'accesso, ma capacità quasi illimitata.



Caratteristiche chiave:

- **Velocità di Accesso:** Diminuisce man mano che si scende nella gerarchia.
- **Costo per Bit:** Diminuisce man mano che si scende nella gerarchia.
- **Capacità:** Aumenta man mano che si scende nella gerarchia.
- **Volatilità:** I livelli superiori (registri, cache, RAM) sono volatili (perdono i dati senza alimentazione), mentre quelli inferiori (dischi, nastri) sono non volatili.



Gerarchia dei Bus

All'interno di un sistema di calcolo, i componenti comunicano tramite **bus**, che sono canali di comunicazione. Esistono diversi tipi di bus, organizzati gerarchicamente per ottimizzare velocità e larghezza di banda:

- **Bus della CPU (Front Side Bus - FSB / Bus di Sistema)**: Il bus più veloce, collega la CPU alla memoria principale e al Northbridge (o Memory Controller Hub). Oggi spesso integrato direttamente nella CPU.
- **PCI Express (PCIe)**: Un bus seriale ad alta velocità, successore del PCI. Utilizzato per schede grafiche, SSD NVMe e altre periferiche che richiedono elevata larghezza di banda. Ogni dispositivo ha una connessione dedicata.
- **PCI (Peripheral Component Interconnect)**: Un bus parallelo più vecchio, ancora presente in alcuni sistemi per periferiche meno esigenti (es. schede audio, schede di rete meno recenti).
- **DMI (Direct Media Interface)**: Un collegamento punto-punto di Intel che connette il Northbridge (o la CPU, nei design più recenti) al Southbridge (o I/O Controller Hub - PCH), gestendo le comunicazioni con periferiche più lente come USB, SATA, LAN, e slot PCI tradizionali.

Storia dei Sistemi Operativi

🔗 Analogia: L'Evoluzione della Cucina

Pensa all'evoluzione dei Sistemi Operativi come all'evoluzione di una cucina professionale.

- **Prima Generazione (Valvole)**: La cucina è enorme, con macchinari lenti e ingombranti. Ogni piatto è preparato a mano, uno alla volta, con istruzioni molto precise e ripetitive. Se sbagli un passaggio, devi ricominciare da capo.
- **Seconda Generazione (Transistor/Batch)**: I macchinari diventano più piccoli e veloci. Ora puoi preparare più piatti in sequenza (batch), ma devi dare tutte le istruzioni in anticipo. C'è un "capo cuoco" (Monitor Residente) che supervisiona, e un "magazziniere" (Spooling) che prepara gli ingredienti in anticipo per non far aspettare i cuochi.
- **Terza Generazione (Circuiti Integrati/Multiprogramming)**: La cucina è ora molto più sofisticata. Ci sono più cuochi (CPU) e più piatti in preparazione contemporaneamente (multiprogrammazione). I cuochi possono passare da un piatto all'altro rapidamente.
- **Quarta Generazione (Personal Computer)**: Le cucine diventano piccole e personali, per ogni casa. Sono facili da usare, con interfacce intuitive (GUI).
- **Quinta Generazione (Mobile/Distribuito)**: La cucina è ovunque, anche in tasca! I piatti possono essere preparati collaborando con altre cucine (sistemi distribuiti) o gestendo molte piccole richieste veloci (mobile).

La storia dei Sistemi Operativi è un viaggio affascinante che riflette l'evoluzione dell'hardware e le crescenti esigenze degli utenti.

- **Prima Generazione (1945-1955): Valvole e Programmazione Diretta**
 - I computer erano enormi, costosi e programmabili direttamente in linguaggio macchina o assembly, senza un vero e proprio sistema operativo. Ogni "job" (programma) veniva eseguito uno alla volta.
 - **Caratteristiche**: Nessun SO, programmazione a basso livello, I/O manuale (schede perforate).
- **Seconda Generazione (1955-1965): Transistor e Sistemi Batch**
 - Introduzione dei transistor, che rendono i computer più affidabili e veloci. Nascono i primi sistemi operativi per ottimizzare l'uso della CPU.
 - **Sistemi Batch**: I lavori (job) venivano raggruppati in "batch" e eseguiti in sequenza. L'utente non interagiva direttamente con il computer.
 - **Monitor Residente**: Questo fu il primo vero "sistema operativo" rudimentale. Era un piccolo programma sempre residente in memoria che aveva il compito di controllare la sequenza dei "job" (programmi) da eseguire. Caricava i programmi in memoria, passava loro il controllo e, al termine del job, riprendeva il controllo per caricare il successivo. Era un sequenziatore automatico di compiti. Esempi includono FMS (Fortran Monitor System) e IBSYS.
 - **Job Control Language (JCL)**: Per comunicare con il Monitor Residente, gli utenti dovevano usare un linguaggio speciale, il JCL. Questo linguaggio permetteva di descrivere al sistema operativo come dovevano essere eseguiti i loro "job", specificando ad esempio

quali programmi usare, quali dati di input leggere e dove scrivere l'output. Era un modo per "programmare" il flusso di lavoro del computer.

- **I/O Off-line**: Per evitare che la CPU, molto più veloce, dovesse attendere le lente periferiche di I/O (come lettori di schede perforate o stampanti), i dati venivano preparati o elaborati "off-line" da dispositivi esterni. Ad esempio, i dati venivano prima letti da schede perforate e copiati su nastro magnetico, e solo dopo il nastro veniva caricato nel computer principale. Questo permetteva alla CPU di lavorare alla sua massima velocità.
- **Buffering**: Consisteva nell'uso di piccole aree di memoria temporanee (buffer) per immagazzinare dati durante i trasferimenti I/O. Questo permetteva di compensare le differenze di velocità tra la CPU e le periferiche. Ad esempio, i dati in arrivo da un lento lettore di schede potevano essere accumulati in un buffer, e la CPU poteva leggerli dal buffer in blocchi più grandi e veloci.
- **Spooling (Simultaneous Peripheral Operations On-Line)**: Permetteva di sovrapporre l'I/O di un job con la computazione di un altro. I dati di input venivano letti su disco (spool) e i dati di output venivano scritti su disco prima di essere stampati, liberando la CPU.
- **Terza Generazione (1965-1980): Circuiti Integrati e Multiprogrammazione**
 - L'introduzione dei circuiti integrati porta a computer ancora più potenti e compatti. Nasce la multiprogrammazione.
 - **Multiprogrammazione**: Più job risiedono contemporaneamente in memoria. Quando un job si blocca per I/O, la CPU passa a un altro job, massimizzando l'utilizzo della CPU.
 - **Time-Sharing**: Una forma di multiprogrammazione che permette a più utenti di interagire contemporaneamente con il sistema, dando l'illusione di avere il computer tutto per sé (es. UNIX, MULTICS).
- **Quarta Generazione (1980-Presente): Personal Computer e GUI**
 - L'invenzione del microprocessore porta alla nascita dei personal computer. L'attenzione si sposta sull'interfaccia utente.
 - **GUI (Graphical User Interface)**: Interfacce grafiche intuitive (es. Apple Macintosh, Windows) rendono i computer accessibili a un pubblico vasto.
 - **Sistemi Distribuiti**: I computer iniziano a connettersi in rete, portando allo sviluppo di SO che gestiscono risorse distribuite.
- **Quinta Generazione (Presente): Mobile, Cloud e IoT**
 - SO per dispositivi mobili (Android, iOS), cloud computing, Internet of Things (IoT).
 - **Caratteristiche**: Connettività costante, efficienza energetica, sicurezza, integrazione con servizi cloud.

Riepilogo Fondamenti

1. L'**SO** è un gestore di risorse che astrae l'hardware.
2. La **Modalità Kernel/Utente** protegge il sistema.
3. Le **System Call** sono il ponte sicuro tra le due modalità.
4. Gli **Interrupt** permettono alla CPU di lavorare in modo efficiente.
5. Il **Boot** è la sequenza che porta dal BIOS al kernel funzionante.
6. I kernel monolitici sono veloci ma meno robusti, mentre i microkernel sono più modulari e robusti ma con overhead di comunicazione.
7. L'architettura a strati semplifica la progettazione ma può essere meno efficiente.
8. La separazione **Meccanismo/Politica** garantisce flessibilità.
9. La **Gerarchia di Memorie** ottimizza l'accesso ai dati bilanciando velocità e costi.

II. Gestione dei Processi e dei Thread

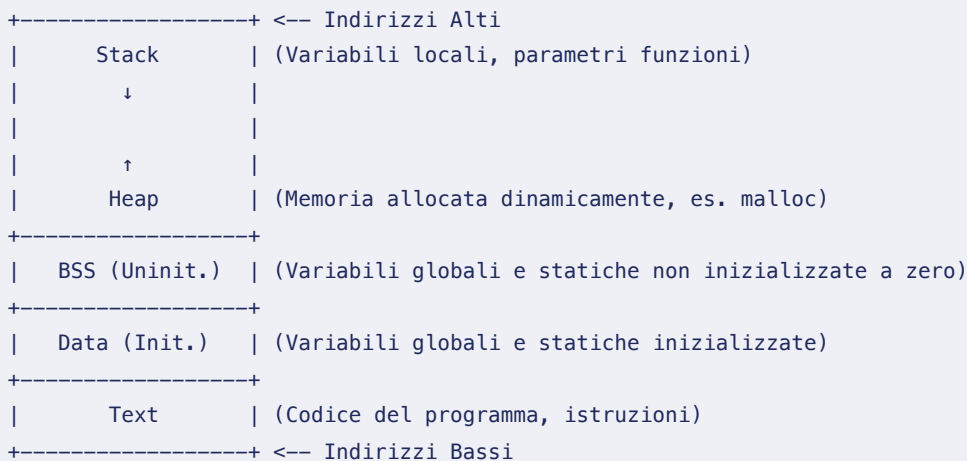
Processi e Process Control Block (PCB)

Definition

- Un **Programma** è un'entità **passiva**. È la ricetta scritta su un foglio.
- Un **Processo** è un'istanza di un programma in **esecuzione**. È l'atto di *cucinare* seguendo la ricetta. È un'entità **attiva**.

Layout di Memoria di un Processo

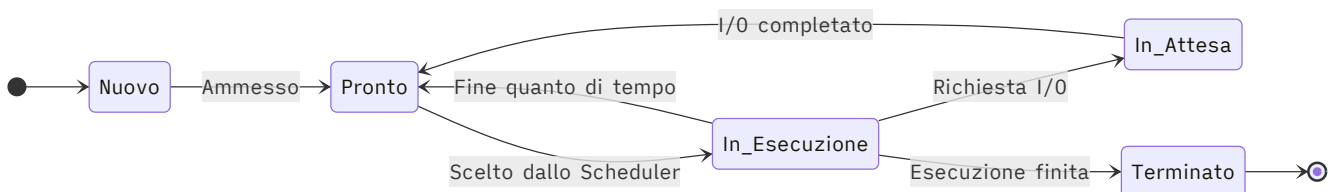
Un processo in memoria ha una struttura precisa:



🔗 La sezione **BSS (Block Started by Symbol)** è un'area di memoria per le variabili globali e statiche che non sono state esplicitamente inizializzate a zero nel codice sorgente. La sua caratteristica principale è che il sistema operativo garantisce che il suo contenuto sia inizializzato a zero all'avvio del programma dal loader, senza che il file eseguibile debba contenere i byte zero per queste variabili, risparmiando spazio su disco.

🔥 Lo **Stack** e l'**Heap** crescono l'uno verso l'altro. Se si scontrano, si verifica un errore di **stack overflow** e il SO termina il processo.

Gli Stati di un Processo



Process Control Block (PCB)

🔗 Il **PCB** è la "carta d'identità" di un processo. È una struttura dati nel kernel che contiene tutto ciò che serve per gestire il processo: stato, program counter, registri, limiti di memoria, file aperti, etc.

Il **Context Switch** è l'operazione (costosa in termini di tempo) di salvare lo stato di un processo nel suo PCB e caricarne un altro.

`task_struct` in Linux

In Linux, il [Process Control Block \(PCB\)](#) è implementato tramite la struttura dati `task_struct`. Questa struttura è estremamente ricca di informazioni e contiene tutti i dettagli necessari per gestire un processo o un thread, inclusi:

- Stato del processo (running, waiting, stopped, zombie, etc.)
- ID del processo (PID) e ID del padre (PPID)
- Puntatori ai registri della CPU (Program Counter, Stack Pointer, registri generici)
- Informazioni di scheduling (priorità, quanto di tempo rimanente)

- Informazioni di gestione della memoria (puntatori alle tabelle delle pagine, limiti di memoria)
- Informazioni di contabilità (tempo di CPU usato, tempo di inizio)
- Informazioni sullo stato I/O (file aperti, dispositivi allocati)
- Credenziali utente e gruppo
- Puntatori ad altre strutture `task_struct` per formare liste (ready queue, wait queue).

Ogni thread in Linux è rappresentato da una propria `task_struct`, il che significa che il kernel Linux gestisce i thread come processi leggeri, permettendo una schedulazione individuale e un pieno sfruttamento dei sistemi multi-core.

Creazione e Terminazione dei Processi

Oltre alla `fork()`, che crea un processo figlio, ci sono altre system call fondamentali per la gestione del ciclo di vita dei processi:

- `fork()`: Crea un nuovo processo (figlio) che è una copia quasi esatta del processo chiamante (padre). Il PID restituito al padre è > 0 , al figlio è 0 .
- `exec()` (famiglia di funzioni): Sostituisce l'immagine del processo corrente con un nuovo programma. Non crea un nuovo processo, ma trasforma quello esistente. Spesso usata dopo una `fork()`.
- `exit()`: Termina volontariamente l'esecuzione del processo chiamante. Il processo rilascia le sue risorse e il suo stato finale viene reso disponibile al padre.
- `wait()` / `waitpid()`: Il processo padre si blocca in attesa che uno o più dei suoi processi figli terminino. Questo permette al padre di raccogliere lo stato di terminazione del figlio (per evitare "zombie processes").
- `abort()`: Causa la terminazione anomala di un processo, spesso a causa di un errore irreversibile. Il sistema operativo può generare un core dump per il debug.

Processi Zombie

Ottimo, hai notato che il concetto di "processo zombie" era menzionato in una domanda d'esame ma non definito! È un punto chiave nel ciclo di vita dei processi.

 Un **processo zombie** (o "defunct process") è un processo che ha completato la sua esecuzione (ha chiamato `exit()`) ma la cui entry nella tabella dei processi (il suo Process Control Block - PCB) è ancora mantenuta nel kernel.

Perché succede?

Questo accade perché il processo padre non ha ancora eseguito una `wait()` o `waitpid()` system call per recuperare lo stato di terminazione (exit status) del figlio. Il kernel mantiene il PCB del figlio per permettere al padre di raccogliere queste informazioni.

Implicazioni:

- I processi zombie **non consumano risorse di CPU attive o memoria RAM** (a parte la piccola quantità per il PCB).
- Tuttavia, continuano a occupare un **PID** e una entry nella tabella dei processi.
- Se un sistema accumula troppi processi zombie (ad esempio, se i processi padre non chiamano mai `wait()`), può esaurire le entry disponibili nella tabella dei processi, impedendo la creazione di nuovi processi. Questo è un problema serio, anche se raro in sistemi ben progettati.

 Un processo zombie è come un ospite che ha finito di mangiare e sta per andarsene, ma aspetta che il cameriere (il processo padre) venga a prendere il conto (lo stato di uscita) prima di liberare il tavolo (la sua entry nella tabella dei processi). Se il cameriere non arriva, l'ospite rimane lì, occupando un tavolo che potrebbe servire ad altri!

Ottimizzazione della `fork()`: Copy-on-Write (CoW)

Quando un processo chiama `fork()`, l'idea originale è creare una copia esatta del processo padre. Ma copiare *tutta* la memoria può essere molto lento!

🔗 **Copy-on-Write (CoW)** è una tecnica di ottimizzazione usata dal kernel per rendere la `fork()` più efficiente. Invece di copiare immediatamente tutte le pagine di memoria del processo padre al momento della `fork()`, il kernel permette al processo padre e al nuovo processo figlio di **condividere le stesse pagine di memoria in modalità sola lettura**.

Come funziona:

1. Quando `fork()` viene chiamata, il kernel non copia le pagine. Semplicemente, marca le pagine condivise come "sola lettura" per entrambi i processi.
2. Se il processo padre o il processo figlio tenta di **scrivere** su una di queste pagine condivise, si verifica un'eccezione (un tipo speciale di page fault).
3. Il kernel intercetta questa eccezione, crea una **copia privata** di quella specifica pagina per il processo che ha tentato di scrivere.
4. Il processo può ora scrivere sulla sua copia, e l'altro processo continua a usare la versione originale.

Vantaggi:

- **Velocità:** La `fork()` diventa estremamente veloce, perché la copia delle pagine avviene solo quando e se strettamente necessario.
- **Efficienza di Memoria:** Si evita di sprecare memoria copiando dati che potrebbero non essere mai modificati dal figlio.

👉 **Pensa a due chef che leggono la stessa ricetta (pagina di memoria). Finché entrambi leggono, va bene. Ma se uno vuole **modificare** la ricetta (scrivere sulla pagina), ne fa una sua copia personale per non influenzare l'altro.**

🏃 Thread: I Lavoratori Leggeri

🔗 Un **thread** è un'unità di esecuzione **all'interno** di un processo. Più thread possono esistere nello stesso processo, condividendone le risorse.

👉 Analogia: Il Team di Cuochi

- Il **Processo** è l'intera cucina (con le sue risorse: frigorifero, dispensa, fornelli).
- I **Thread** sono i singoli cuochi che lavorano in quella cucina.
- Tutti i cuochi (**thread**) condividono lo stesso spazio di memoria (**spazio di memoria**), quindi collaborano facilmente.
- Ogni cuoco ha però il suo compito specifico (uno taglia, uno mescola, uno impiatta).

User Threads vs Kernel Threads

Non tutti i thread sono uguali. La loro gestione può avvenire a due livelli:

Caratteristica	User Threads (Gestiti da una libreria)	Kernel Threads (Gestiti dal SO)	
Gestione	Veloce, a livello utente, senza system call.	Lenta, richiede l'intervento del kernel.	
Visibilità al SO	Il kernel non li vede, vede solo il processo.	Il kernel li vede e li schedula individualmente.	
Blocking	Se un thread fa una system call bloccante, tutto il processo si blocca .	Solo il thread che fa la chiamata si blocca.	
Multicore	Non possono essere eseguiti in parallelo su più core.	Possono essere eseguiti in parallelo su più core.	

Modelli di Mapping (o Modelli di Multithreading)

Hai chiesto di espandere i modelli di multithreading, ed è un'ottima idea! Questi modelli spiegano come i thread a livello utente sono collegati ai thread a livello kernel.

1. **Many-to-One:**

- **Descrizione:** Mappa molti thread a livello utente a un singolo thread a livello kernel. La gestione dei thread è efficiente perché avviene interamente in user-space, senza l'overhead delle system call.
- **Pro:** Basso overhead per la creazione e gestione dei thread.
- **Contro:** Se un thread utente effettua una system call bloccante, l'intero processo si blocca, e non è possibile sfruttare il parallelismo su CPU multi-core, poiché il kernel vede un solo thread di esecuzione per il processo.
- **Esempio:** Green Threads (vecchi sistemi Java).

2. One-to-One:

- **Descrizione:** Mappa ogni thread utente a un thread kernel separato. Ogni thread utente ha la sua controparte nel kernel.
- **Pro:** Permette il vero parallelismo su sistemi multi-core. Il blocco di un thread non influisce sugli altri thread dello stesso processo.
- **Contro:** Overhead maggiore per la creazione e la gestione dei thread, in quanto ogni operazione richiede una system call al kernel. Il numero di thread è limitato dalle risorse del kernel.
- **Esempio:** Modello usato da *Linux (NPTL)* e *Windows*.

3. Many-to-Many:

- **Descrizione:** Multiplexa molti thread utente su un numero minore o uguale di thread kernel (Lightweight Processes - LWP). Questo modello offre un buon equilibrio, combinando l'efficienza della gestione a livello utente con la capacità del kernel di sfruttare il parallelismo e gestire le system call bloccanti.
- **Pro:** Flessibile, unisce i vantaggi degli altri due. Il kernel può creare un numero ottimale di thread kernel per sfruttare i core disponibili.
- **Contro:** Molto complesso da implementare.
- **Esempio:** Alcune implementazioni di Pthreads.

4. Two-Level Model:

- **Descrizione:** Una variante del Many-to-Many che permette una maggiore flessibilità. Alcuni thread utente possono essere mappati direttamente su un thread kernel (come nel One-to-One), mentre altri possono essere multiplexati su un pool di LWP (come nel Many-to-Many).
- **Pro:** Offre ancora più flessibilità, combinando i vantaggi di entrambi i modelli. È utile per thread che richiedono un legame stretto con il kernel (es. I/O intensivo) o che possono beneficiare del multiplexing.
- **Contro:** Ancora più complesso da implementare.
- **Esempio:** Solaris (prima di passare a One-to-One).

Lightweight Process (LWP)

✍ Un **Lightweight Process (LWP)** è un livello di astrazione intermedio tra i thread utente e i thread kernel, usato in alcuni sistemi (come Solaris) nel modello Many-to-Many. Un LWP è un thread kernel che può avere uno o più thread utente mappati su di esso.

Come funziona:

- I thread utente sono gestiti dalla libreria a livello utente.
- Quando un thread utente ha bisogno di eseguire una system call o di essere schedato dal kernel, viene mappato su un LWP disponibile.
- Il kernel schedula gli LWP, non direttamente i thread utente.

Vantaggi:

- Permette una gestione più flessibile dei thread utente, combinando l'efficienza della gestione a livello utente con la capacità del kernel di sfruttare il parallelismo su più core.

Cancellazione dei Thread (Thread Cancellation)

Un altro dettaglio importante!

✍ La **cancellazione dei thread** è un meccanismo che permette a un thread di terminare l'esecuzione di un altro thread (il "thread target").

Esistono due modalità principali:

1. **Cancellazione Asincrona (Asynchronous Cancellation):**

- Il thread target viene terminato **immediatamente** non appena riceve la richiesta di cancellazione.
- **Rischioso**: Può lasciare le risorse in uno stato inconsistente (es. un mutex bloccato, file aperti non chiusi), perché il thread non ha la possibilità di "pulire" prima di morire.

2. **Cancellazione Differita (Deferred Cancellation):**

- Il thread target non viene terminato immediatamente. Invece, controlla periodicamente (in punti predefiniti del suo codice, chiamati "cancellation points") se è stata richiesta la sua cancellazione.
- Se rileva una richiesta di cancellazione, può eseguire le operazioni di pulizia necessarie (es. rilasciare mutex, chiudere file) prima di terminare.
- **Più Sicuro**: Permette al thread di terminare in modo pulito e controllato. Questa è la modalità predefinita in molte implementazioni (es. Pthreads).

Signal Handling (Gestione dei Segnali)

Questo è cruciale in ambienti Unix/Linux!

✍ Un **segnale** è una notifica software asincrona inviata al processo per indicare che si è verificato un evento (es. un errore di memoria come **SIGSEGV**, un'interruzione da tastiera come **SIGINT**).

In un ambiente multithread, la gestione dei segnali diventa più complessa:

- **Consegna al thread a cui si applica**: Se il segnale è generato da un errore specifico di un thread (es. **SIGSEGV** per un accesso a memoria non valido), viene consegnato a quel thread.
- **Consegna a un thread specifico**: Il processo può designare un thread specifico per ricevere tutti i segnali.
- **Consegna a tutti i thread**: Il segnale viene consegnato a tutti i thread del processo, ma solo uno lo gestirà.
- **Consegna a un thread arbitrario**: Il segnale viene consegnato a un thread qualsiasi che non abbia bloccato quel tipo di segnale.

🔑 Ogni thread può avere la sua **maschera di segnali**, che specifica quali segnali bloccare o ignorare. Questo permette un controllo granulare su come i segnali vengono gestiti all'interno di un processo multithread.

Thread-Local Storage (TLS)

✍ Il **Thread-Local Storage (TLS)** è una tecnica che permette a ogni thread di avere la propria copia privata di una variabile globale o statica.

Normalmente, le variabili globali sono condivise tra tutti i thread di un processo. Con TLS, una variabile dichiarata come "thread-local" avrà un'istanza separata per ogni thread, permettendo ai thread di lavorare in modo indipendente su quella variabile senza la necessità di sincronizzazione esplicita.

Esempio (in C++11 con `thread_local`):

```
#include <iostream>
#include <thread>
#include <vector>

thread_local int counter = 0; // Questa variabile è thread-local
```

```

void increment_and_print(int id) {
    counter++; // Ogni thread incrementa la propria copia di counter
    std::cout << "Thread " << id << ": counter = " << counter << std::endl;
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 3; ++i) {
        threads.emplace_back(increment_and_print, i);
    }
    for (auto& t : threads) {
        t.join();
    }
    std::cout << "Main thread: counter = " << counter << std::endl; // Il main ha la sua copia
    return 0;
}

```

Output Esempio:

```

Thread 0: counter = 1
Thread 1: counter = 1
Thread 2: counter = 1
Main thread: counter = 0

```

Come puoi vedere, ogni thread ha il suo `counter` indipendente.

Attivazione dello Scheduler (Scheduler Activation)

 L'attivazione dello scheduler è un meccanismo che permette di coordinare i thread a livello utente con i thread a livello kernel (LWP - Lightweight Processes) in modelli come Many-to-Many.

Quando un thread utente esegue una system call bloccante o deve essere prelazionato, il kernel notifica la libreria dei thread a livello utente tramite un "upcall". La libreria può quindi decidere di mappare un altro thread utente su un LWP disponibile o di creare un nuovo LWP se necessario. Questo cerca di combinare i vantaggi di efficienza dei thread utente con la capacità del kernel di gestire il parallelismo.

Esempi di Astrazioni di Alto Livello per il Multithreading

Oltre alle API di basso livello come Pthreads, esistono librerie e framework che semplificano la programmazione multithreaded:

- **OpenMP (Open Multi-Processing):**
 - **Descrizione:** Un'API per la programmazione parallela su memoria condivisa. Permette ai programmatori di aggiungere direttive (pragmas) al codice C, C++ o Fortran per indicare quali parti del codice possono essere parallelizzate. Il compilatore e il runtime di OpenMP si occupano di generare i thread e gestire la sincronizzazione.
 - **Vantaggi:** Semplifica la parallelizzazione di loop e regioni di codice, riducendo la necessità di gestire esplicitamente i thread.
- **QueueUserWorkItem (Windows Thread Pools):**
 - **Descrizione:** Una funzione dell'API di Windows che permette di accodare una funzione (un "work item") a un pool di thread gestito dal sistema. Invece di creare e gestire i propri thread, l'applicazione può semplicemente inviare compiti al pool, e il sistema operativo si occupa di assegnarli a thread disponibili o di crearne di nuovi se necessario.
 - **Vantaggi:** Riduce l'overhead di creazione/distruzione dei thread e migliora l'efficienza, poiché i thread possono essere riutilizzati.

Windows Threads

In Windows, il modello di threading è principalmente **One-to-One**, dove ogni thread utente è mappato a un thread kernel. Il "contesto" di un thread in Windows include:

- **Thread ID**: Un identificatore univoco per il thread.
- **Program Counter (PC)**: L'indirizzo della prossima istruzione da eseguire.
- **Registri**: I valori dei registri della CPU.
- **Stack**: Uno stack separato per le chiamate a funzione e le variabili locali del thread.
- **Private Storage**: Un'area di memoria privata per il thread (simile al Thread-Local Storage).
- **Priorità**: La priorità di schedulazione del thread.
- **Stato**: Lo stato corrente del thread (running, ready, waiting, terminated).

Concorrenza vs. Parallelismo

Questa distinzione è fondamentale e spesso confusa! Chiariamo subito le idee.

Definition

- **Concorrenza**: La capacità di un sistema di gestire più task in modo che *sembrino* eseguire contemporaneamente. Su un sistema a singolo core, la concorrenza è ottenuta tramite lo scheduling della CPU (time-slicing), dove il sistema operativo alterna rapidamente l'esecuzione dei task, dando l'illusione del parallelismo. È una questione di *struttura* (come un programma è organizzato per gestire più task).
- **Parallelismo**: L'esecuzione *simultanea* di più task su più unità di elaborazione (es. multi-core CPU). Questo può essere *parallelismo di dati* (stessa operazione su diversi sottoinsiemi di dati) o *parallelismo di task* (operazioni diverse su core diversi).

 **Il parallelismo implica sempre concorrenza, ma la concorrenza non implica necessariamente parallelismo. Puoi avere concorrenza su un singolo core (alternando i task), ma ti serve più di un core per avere vero parallelismo.**

Pensa a un cuoco (CPU) che gestisce più piatti (task):

- **Concorrenza**: Il cuoco salta rapidamente da un piatto all'altro, mescolando qui, tagliando là, dando l'impressione di fare tutto contemporaneamente. Ma in ogni singolo istante, sta lavorando su un solo piatto.
- **Parallelismo**: Hai più cuochi (multi-core CPU) che lavorano su piatti diversi *nello stesso momento*.

Legge di Amdahl

Fantastico, la Legge di Amdahl è cruciale per capire i limiti del parallelismo!

 **La Legge di Amdahl definisce il guadagno massimo teorico di velocità (speed-up) che si può ottenere parallelizzando una porzione di un programma.**

La formula è:

$$Speedup = \frac{1}{S + \frac{1-S}{N}}$$

Dove:

- S è la frazione del programma che deve essere eseguita in modo **seriale** (non parallelizzabile).
- $1 - S$ è la frazione del programma che può essere eseguita in modo **parallelo**.
- N è il numero di processori (core) che eseguono la parte parallelizzabile.

Implicazioni Pratiche:

La legge dimostra che, anche con un numero infinito di processori ($N \rightarrow \infty$), lo speed-up è limitato dal reciproco della porzione seriale ($\frac{1}{S}$). Questo significa che ridurre la parte seriale del codice è cruciale per ottenere significativi miglioramenti delle prestazioni in sistemi multi-core.

Esempio:

Se un'applicazione è per il 75% parallela ($1 - S = 0.75$, quindi $S = 0.25$):

- Con 2 core ($N = 2$): $Speedup = \frac{1}{0.25 + \frac{0.75}{2}} = \frac{1}{0.25 + 0.375} = \frac{1}{0.625} = 1.6$

- Con 4 core ($N = 4$): $Speedup = \frac{1}{0.25 + \frac{0.75}{4}} = \frac{1}{0.25 + 0.1875} = \frac{1}{0.4375} \approx 2.28$

Come puoi vedere, raddoppiare i core non raddoppia lo speed-up a causa della parte seriale!

Comunicazione tra Processi (IPC) vs. Comunicazione tra Thread

Questa è una distinzione chiave! I processi sono entità isolate, i thread condividono molte risorse.

Caratteristica	Comunicazione tra Processi (IPC)	Comunicazione tra Thread	
Isolamento	Alto . Spazi di memoria separati. Un errore in un processo non impatta gli altri.	Basso . Condividono lo stesso spazio di memoria. Un errore in un thread può compromettere l'intero processo.	
Meccanismi	Pipe, Socket, Message Queue, Shared Memory, Semaphore, Mutex.	Variabili condivise, Mutex, Semaphore, Condition Variables, Read-Write Locks.	
Complessità	Richiede meccanismi espliciti e più complessi (es. copiare dati tra spazi di memoria).	Più semplice, accesso diretto alle variabili condivise.	
Performance	Più lenta a causa dell'overhead di sistema (context switch, copie dati).	Più veloce perché non ci sono copie di dati tra spazi di memoria.	
Sicurezza	Maggiore controllo e protezione tra processi.	Richiede sincronizzazione accurata per evitare Il Problema della Sezione Critica	race condition].

🔗 Pensa ai processi come a **appartamenti separati** in un condominio: per comunicare devi usare la posta, il telefono o incontrarti in aree comuni. I thread sono come **coinquilini** nello stesso appartamento: possono parlarsi direttamente e condividere tutto ciò che è in casa.

Meccanismi di IPC Dettagliati: Message Passing (Comunicazione tramite Messaggi)

Il Message Passing è un meccanismo fondamentale per l'IPC.

✍ La **comunicazione tramite messaggi (Message Passing)** è un meccanismo IPC in cui i processi comunicano scambiandosi messaggi, senza dover condividere lo spazio di indirizzamento. È particolarmente utile per la comunicazione tra processi su macchine diverse (sistemi distribuiti) o quando la condivisione di memoria è impraticabile o indesiderata.

Le due operazioni fondamentali sono `send(message)` e `receive(message)`.

1. Comunicazione Diretta vs. Indiretta

- **Comunicazione Diretta:**
 - Il mittente e il destinatario devono **nominarsi a vicenda esplicitamente**.
 - Sintassi: `send(P, message)` (invia un messaggio al processo P), `receive(Q, message)` (riceve un messaggio dal processo Q).
 - **Vantaggi:** Semplice da implementare, collegamento chiaro tra i processi.
 - **Svantaggi:** Rigidità nella denominazione (se un nome cambia, il codice deve essere aggiornato), accoppiamento stretto tra mittente e destinatario.
- **Comunicazione Indiretta:**
 - La comunicazione avviene tramite una "casella di posta" (chiamata anche **mailbox** o **porta**). I processi non devono conoscersi direttamente, ma solo la mailbox.
 - Sintassi: `send(A, message)` (invia un messaggio alla mailbox A), `receive(A, message)` (riceve un messaggio dalla mailbox A).
 - **Vantaggi:** Maggiore flessibilità (i processi possono cambiare senza influenzare gli altri), disaccoppiamento tra mittente e destinatario.
 - **Svantaggi:** Necessità di gestire le mailbox (creazione, distruzione, proprietà), potenziale per problemi di sicurezza se le mailbox non sono protette.

2. Sincronizzazione (Blocking vs. Non-blocking)

Le operazioni `send` e `receive` possono essere implementate con diverse semantiche di sincronizzazione:

- **Blocking (Sincrona)**:
 - `blocking send`: Il mittente si blocca finché il messaggio non è stato ricevuto dal destinatario (o dalla mailbox).
 - `blocking receive`: Il destinatario si blocca finché non c'è un messaggio disponibile da ricevere.
 - **Vantaggi**: Semplifica la sincronizzazione tra processi, garantisce che il messaggio sia stato consegnato.
 - **Svantaggi**: Può portare a deadlock se i processi si aspettano a vicenda, riduce il parallelismo.
- **Non-blocking (Asincrona)**:
 - `non-blocking send`: Il mittente invia il messaggio e continua immediatamente l'esecuzione, senza attendere la ricezione.
 - `non-blocking receive`: Il destinatario tenta di ricevere un messaggio. Se un messaggio è disponibile, lo riceve. Altrimenti, restituisce un valore speciale (es. `NULL` o un errore) e continua immediatamente l'esecuzione.
 - **Vantaggi**: Massimizza il parallelismo, evita i deadlock legati all'attesa.
 - **Svantaggi**: Richiede una gestione più complessa dei buffer (se il destinatario non è pronto), il mittente non sa se il messaggio è stato effettivamente ricevuto.

🔗 La scelta tra comunicazione diretta/indiretta e blocking/non-blocking dipende dalle esigenze specifiche dell'applicazione e dal livello di sincronizzazione desiderata.

Buffering nel Message Passing

La comunicazione tramite messaggi può essere implementata con diverse strategie di buffering, che influenzano la sincronizzazione tra mittente e destinatario:

1. **Capacità Zero (Zero Capacity / No Buffering)**:
 - **Descrizione**: Il buffer non ha capacità. Il mittente deve attendere che il destinatario sia pronto a ricevere il messaggio.
 - **Sincronizzazione**: Questo è un meccanismo di **rendezvous**. Entrambi i processi (mittente e destinatario) devono essere pronti contemporaneamente per lo scambio del messaggio.
 - **Vantaggi**: Semplice, garantisce che il messaggio sia stato ricevuto.
 - **Svantaggi**: Massima sincronizzazione, può ridurre il parallelismo.
2. **Capacità Limitata (Bounded Capacity)**:
 - **Descrizione**: Il buffer ha una dimensione finita (N messaggi).
 - **Sincronizzazione**:
 - Il mittente si blocca solo se il buffer è pieno.
 - Il destinatario si blocca solo se il buffer è vuoto.
 - **Vantaggi**: Permette un certo grado di disaccoppiamento temporale tra mittente e destinatario.
 - **Svantaggi**: Se il buffer si riempie, il mittente si blocca.
3. **Capacità Illimitata (Unbounded Capacity)**:
 - **Descrizione**: Il buffer ha una capacità teoricamente infinita.
 - **Sincronizzazione**: Il mittente non si blocca mai (invia il messaggio e continua). Il destinatario si blocca solo se il buffer è vuoto.
 - **Vantaggi**: Massima flessibilità per il mittente, massimizza il parallelismo.
 - **Svantaggi**: Richiede risorse di memoria illimitate (non pratico in sistemi reali), il mittente non ha garanzia sulla ricezione del messaggio.

IPC POSIX (Inter-Process Communication in Sistemi Unix-like)

I sistemi operativi POSIX (Portable Operating System Interface, standard per sistemi Unix-like) offrono una serie di meccanismi standardizzati per la comunicazione tra processi.

- **Pipe (Tubi):**
 - **Definizione:** Un canale di comunicazione unidirezionale tra due processi. I dati vengono scritti a un'estremità e letti dall'altra, in ordine FIFO (First-In, First-Out).
 - **Tipi:**
 - **Pipe Ordinarie (Unnamed Pipes):** Esistono solo tra processi che hanno una relazione di parentela (es. padre-figlio) e vengono create con la system call `pipe()`.
 - **Pipe con Nome (Named Pipes / FIFOs):** Sono file speciali nel file system che permettono la comunicazione tra processi non correlati. Vengono create con `mkfifo()`.
 - **Vantaggi:** Semplici da usare per comunicazione unidirezionale.
 - **Svantaggi:** Unidirezionali, limitate in capacità, non adatte per comunicazione complessa.
- **Shared Memory (Memoria Condivisa):**
 - **Definizione:** Un'area di memoria che può essere mappata nello spazio di indirizzamento di più processi, permettendo loro di leggere e scrivere direttamente in essa.
 - **Vantaggi:** **Molto veloce**, in quanto non richiede copie di dati tra spazi di indirizzamento (a differenza del message passing).
 - **Svantaggi:** Richiede una **sincronizzazione esplicita** tra i processi per evitare race condition (es. con semafori o mutex).
 - **mmap() (Memory-Mapped Files):** Una system call POSIX che permette di mappare un file (o una regione di memoria anonima) direttamente nello spazio di indirizzamento di un processo. Se più processi mappano lo stesso file, l'area di memoria diventa condivisa, rendendo `mmap()` un potente meccanismo IPC per la condivisione di memoria e l'I/O efficiente.
- **Socket:**
 - **Definizione:** Un endpoint per la comunicazione bidirezionale tra processi, sia sulla stessa macchina che su macchine diverse (rete). Funzionano secondo il modello client-server.
 - **Tipi:**
 - **Domain Socket (Unix Domain Sockets):** Per comunicazione tra processi sulla stessa macchina, usando il file system come spazio dei nomi. Molto efficienti.
 - **Internet Socket:** Per comunicazione su rete (TCP/IP).
 - **Vantaggi:** Flessibili, supportano comunicazione su rete, bidirezionali.
 - **Svantaggi:** Più complessi da programmare rispetto alle pipe.

III. Scheduling della CPU

Prima di tuffarci negli algoritmi, è fondamentale capire che ci sono diversi "gestori del tempo" all'interno di un sistema operativo! Hai notato giustamente che gli appunti del tuo amico distinguono tre tipi di scheduler.

 Lo **scheduler** è la componente del sistema operativo responsabile di selezionare quale processo o thread deve essere eseguito dalla CPU in un dato momento.

Esistono tre tipi principali di scheduler, che operano a diverse scale temporali:

1. Long-Term Scheduler (Job Scheduler):

- **Funzione:** Decide quali processi (o "job") dal pool di processi sul disco devono essere caricati nella memoria principale (RAM) e inseriti nella **ready queue**.
- **Frequenza:** Opera molto meno frequentemente rispetto agli altri scheduler, tipicamente quando un processo termina o quando il sistema ha risorse disponibili.
- **Obiettivo:** Controllare il **grado di multiprogrammazione** (il numero di processi in memoria). Se il grado è troppo alto, la memoria si satura e si rischia il thrashing. Se il grado è troppo basso, la CPU potrebbe rimanere inattiva.
- **Esempio:** In un sistema batch, selezionare il prossimo job da eseguire.

2. Medium-Term Scheduler:

- **Funzione:** Gestisce lo **swapping**. Sposta temporaneamente processi (o parti di essi) dalla memoria principale al disco (operazione di "swap out") e viceversa (operazione di "swap

in").

- **Frequenza**: Opera a una frequenza intermedia.
- **Obiettivo**: Ridurre il grado di multiprogrammazione per liberare memoria, spesso per risolvere situazioni di thrashing o per far spazio a processi con priorità più alta. Un processo "swappato fuori" è in uno stato di "swapped out" o "suspended" e non è nella ready queue.
- **Esempio**: Se la RAM è quasi piena e un processo sta causando thrashing, il medium-term scheduler potrebbe decidere di "swappare fuori" quel processo.

3. Short-Term Scheduler (CPU Scheduler):

- **Funzione**: È quello che hai già studiato a fondo! Seleziona il prossimo processo o thread dalla **ready queue** (i processi in memoria pronti per l'esecuzione) e gli assegna la CPU.
- **Frequenza**: Opera molto frequentemente (ogni pochi millisecondi), ad ogni interrupt di clock o quando un processo si blocca.
- **Obiettivo**: Massimizzare l'utilizzo della CPU, minimizzare il tempo di attesa e di turnaround, e garantire una buona risposta per gli utenti (in sistemi interattivi).

Pensa a una cucina con tre gestori:

- **Long-Term**: Decide quali ingredienti (processi) portare dal magazzino (disco) alla cucina (RAM).
- **Medium-Term**: Se la cucina è troppo piena, decide quali ingredienti mettere temporaneamente da parte (swap out) per fare spazio.
- **Short-Term**: Decide quale cuoco (processo/thread) deve usare i fornelli (CPU) in questo preciso momento.

CPU-I/O Burst Cycle

 Il **CPU-I/O Burst Cycle** descrive il comportamento alternato dei processi, che passano il loro tempo tra l'esecuzione di istruzioni sulla CPU (**CPU burst**) e l'attesa del completamento di operazioni di I/O (**I/O burst**).

- **CPU Burst**: Periodo in cui un processo esegue calcoli e istruzioni sulla CPU.
- **I/O Burst**: Periodo in cui un processo attende il completamento di un'operazione di input/output (es. lettura da disco, input da tastiera). Durante questo tempo, il processo è in stato di attesa e la CPU può essere assegnata a un altro processo.

 La durata dei **CPU burst** varia notevolmente tra i processi. I processi **CPU-bound** hanno burst lunghi (es. calcoli scientifici), mentre i processi **I/O-bound** hanno burst brevi e frequenti (es. editor di testo, browser web). Gli algoritmi di scheduling cercano di bilanciare l'esecuzione di questi diversi tipi di processi per massimizzare l'utilizzo della CPU e garantire una buona reattività.

Starvation (Inanizione)

Hai chiesto una definizione esplicita di starvation, ed è un concetto chiave da non confondere con il deadlock!

 La **starvation** (o inanizione) si verifica quando un processo o thread non riesce mai a progredire o a ottenere le risorse necessarie (come l'accesso alla CPU o a una risorsa condivisa) perché altri processi/thread lo precedono continuamente nell'accesso o nella schedulazione. Il processo "muore di fame" per mancanza di risorse.

- **Esempio in SJF**: Un processo molto lungo potrebbe non essere mai eseguito se arrivano continuamente processi più brevi.
- **Esempio in Priority Scheduling**: Un processo a bassa priorità potrebbe non essere mai eseguito se ci sono sempre processi ad alta priorità pronti.
- **Soluzione (Aging)**: Aumentare gradualmente la priorità dei processi che aspettano da troppo tempo.

 Pensa a una fila per il cibo. Se una persona è sempre scavalcata da altri che hanno "priorità" o sono più veloci, quella persona non mangerà mai, anche se il cibo è disponibile.

Algoritmi di Scheduling Principali

Hai chiesto di dettagliare gli algoritmi di scheduling, e hai ragione! Avere una descrizione formale qui è utilissimo.

Algoritmo	Descrizione	Pro	Contro
First-Come, First-Served (FCFS)	Le richieste di CPU vengono servite nell'ordine in cui arrivano nella ready queue. È l'algoritmo più semplice da implementare.	Semplice.	Convoy Effect : un processo lungo blocca tutti i processi successivi, anche se brevi. Inefficiente per carichi misti.
Shortest-Job-First (SJF)	Lo scheduler seleziona il processo con il CPU burst time stimato più breve per l'esecuzione. Questo algoritmo minimizza il tempo di attesa medio e il tempo di turnaround medio. Può essere non-preemptive o preemptive (SRTF).	Ottimale per tempo medio di attesa e turnaround.	Richiede la difficoltà di prevedere con precisione la durata del prossimo CPU burst. Rischio di starvation per i processi lunghi.
Priority	Esegue il processo con priorità più alta.	Flessibile.	Starvation per bassa priorità. Si risolve con aging .
Round Robin (RR)	Ogni processo ottiene una "fetta" di tempo (quanto o time slice) della CPU. Se un processo non completa la sua esecuzione entro il quanto, viene interrotto e spostato in fondo alla ready queue.	Equo, ottimo per interattività. Garantisce un tempo di risposta equo per tutti i processi, evitando la starvation.	Performance dipendono criticamente dalla scelta della dimensione del quanto: un quanto troppo grande lo fa assomigliare a FCFS, un quanto troppo piccolo aumenta l'overhead dei context switch.
Shortest-Remaining-Time-First (SRTF)	È la versione preemptive di SJF. Quando arriva un nuovo processo, il suo tempo di burst rimanente viene confrontato con quello del processo attualmente in esecuzione. Se il nuovo processo ha un tempo rimanente più breve, il processo in esecuzione viene interrotto (prelazionato) e il nuovo processo viene eseguito.	Ottimale per tempo medio di attesa.	Richiede la conoscenza del tempo di burst futuro (difficile). Può causare starvation per processi lunghi.
Scheduling a Quote Proporzionali	Prealloca quote proporzionali di CPU per le applicazioni.	Utile per garantire una porzione definita di risorse a specifici processi o gruppi.	Può essere complesso da configurare e gestire in ambienti dinamici.

Svantaggio della Prelazione (Preemption)

🔗 La **prelazione** (o **preemption**) è la capacità del sistema operativo di interrompere l'esecuzione di un processo/thread e passare a un altro. Il principale svantaggio della prelazione è l'**overhead del context switch**, che consuma cicli di CPU e rallenta il sistema se avviene troppo frequentemente. Ogni volta che il SO interrompe un processo per farne partire un altro, deve salvare lo stato del primo e caricare quello del secondo, e questo richiede tempo.

SRTF (Shortest Remaining Time First) vs. Round Robin (RR)

Entrambi sono algoritmi preemptive, ma con filosofie diverse:

Caratteristica	SRTF (Shortest Remaining Time First)	Round Robin (RR)	
Criterio	Tempo di burst rimanente più breve.	Quanto di tempo fisso.	
Obiettivo	Minimizzare il tempo medio di attesa.	Garantire equità e buona risposta per interattività.	
Preemption	Sì , se arriva un nuovo processo con tempo rimanente più breve.	Sì , allo scadere del quanto di tempo.	
Vantaggi	Ottimale per tempo medio di attesa.	Equo, adatto a sistemi interattivi.	

Caratteristica	SRTF (Shortest Remaining Time First)	Round Robin (RR)	
Svantaggi	Richiede la conoscenza del tempo di burst futuro (difficile). Può causare starvation per processi lunghi.	Performance dipendono dal quanto. Overhead di context switch se il quanto è troppo piccolo.	

Inversione di Priorità

✎ L'**inversione di priorità** è un problema che si verifica nello scheduling basato su priorità quando un processo a bassa priorità (che detiene una risorsa condivisa) impedisce l'esecuzione di un processo ad alta priorità che ha bisogno di quella risorsa.

✎ Immagina un'ambulanza (processo ad alta priorità) bloccata nel traffico da un'auto lenta (processo a bassa priorità) che sta occupando l'unica corsia disponibile per un cantiere (risorsa condivisa). L'ambulanza non può passare, anche se ha la priorità.

Come succede:

1. Un processo a bassa priorità (P_bassa) acquisisce un lock su una risorsa condivisa.
2. Un processo ad alta priorità (P_alta) diventa pronto e cerca di acquisire lo stesso lock. P_alta viene bloccato perché P_bassa detiene il lock.
3. P_alta è ora in attesa. Se nel frattempo arriva un processo a media priorità (P_media), lo scheduler potrebbe scegliere P_media (che ha priorità più alta di P_bassa) invece di P_bassa.
4. Risultato: P_bassa non può rilasciare il lock, P_alta è bloccato, e P_media (che non ha bisogno del lock) viene eseguito, ritardando ulteriormente P_alta.

Soluzione: Ereditarietà della Priorità (Priority Inheritance)

Quando un processo ad alta priorità deve attendere una risorsa detenuta da un processo a bassa priorità, al processo a bassa priorità viene temporaneamente assegnata la priorità del processo ad alta priorità che sta bloccando. Una volta rilasciata la risorsa, la sua priorità torna normale.

Scheduling per Sistemi Real-Time

Ottima osservazione! I sistemi operativi che gestiscono processi con scadenze rigide (come quelli in un'automobile o in un sistema di controllo industriale) hanno bisogno di algoritmi di scheduling speciali. Qui, l'obiettivo non è solo l'efficienza, ma la **garanzia di rispettare le scadenze (deadlines)**.

✎ I **sistemi real-time** sono sistemi in cui la correttezza di un'operazione dipende non solo dal risultato logico, ma anche dal momento in cui il risultato viene prodotto.

Ecco due algoritmi comuni per questi sistemi:

1. Rate-Monotonic (RM):

- **Descrizione:** È un algoritmo di scheduling **statico** (le priorità non cambiano durante l'esecuzione). Assegna priorità ai task basandosi sulla loro **frequenza (periodo)**.
- **Regola:** I task con un **periodo più breve** (cioè che devono essere eseguiti più frequentemente) ricevono una **priorità più alta**.
- **Vantaggi:** Semplice da implementare e analizzare. Se un set di task è schedulabile con RM, è schedulabile anche con qualsiasi altro algoritmo.
- **Svantaggi:** Non è ottimale per tutti i set di task (potrebbe non schedulare task che altri algoritmi riuscirebbero a gestire).

2. Earliest Deadline First (EDF):

- **Descrizione:** È un algoritmo di scheduling **dinamico** (le priorità cambiano). Assegna priorità ai task basandosi sulla loro **scadenza imminente**.
- **Regola:** Il task con la **scadenza (deadline) più vicina** ha la **priorità più alta**.
- **Vantaggi:** È un algoritmo **ottimale** per sistemi real-time preemptive: se un set di task può essere schedulato, EDF ci riuscirà.
- **Svantaggi:** Più complesso da implementare e il suo comportamento in caso di sovraccarico può essere imprevedibile (tutti i task potrebbero perdere la scadenza).

Algoritmo	Tipo di Priorità	Criterio	Vantaggi	Svantaggi
Rate-Monotonic (RM)	Statica	Periodo più breve = Priorità più alta	Semplice, prevedibile.	Non sempre ottimale.
Earliest Deadline First (EDF)	Dinamica	Scadenza più vicina = Priorità più alta	Ottimale (se schedulabile, EDF lo fa).	Complesso, meno prevedibile in sovraccarico.

Pensa a un ristorante con ordini speciali (task real-time).

- **RM**: "Il cliente che ordina ogni 5 minuti ha sempre la priorità sul cliente che ordina ogni 10 minuti." (Priorità fissa per tipo di ordine).
- **EDF**: "Il piatto che deve essere pronto tra 2 minuti ha la priorità sul piatto che deve essere pronto tra 10 minuti, indipendentemente da chi l'ha ordinato." (Priorità basata sulla scadenza più vicina).

NUMA (Non-Uniform Memory Access) e Affinità

In sistemi con molti processori/core, la memoria non è sempre "ugualmente vicina" a tutti i core.

 **NUMA (Non-Uniform Memory Access)** è un'architettura di sistema multi-processore in cui il tempo di accesso alla memoria dipende dalla "vicinanza" del processore alla memoria. Ogni processore ha un accesso più veloce alla sua memoria locale e più lento alla memoria di altri processori.

 **Pensa a una grande biblioteca con più sale di lettura. Ogni sala ha la sua sezione di libri. È più veloce prendere un libro dalla tua sala (memoria locale) che andare in un'altra sala (memoria remota).**

Implicazioni per l'SO: Lo scheduler deve cercare di mantenere i processi/thread sui core che hanno accesso più rapido alla memoria che stanno usando, per minimizzare i ritardi.

Soft Affinity e Hard Affinity

Questi concetti riguardano la preferenza di un processo/thread a essere eseguito su una specifica CPU o core.

- **Soft Affinity:**
 - **Descrizione:** Il sistema operativo *tenta* di mantenere un thread sullo stesso core su cui è stato eseguito l'ultima volta.
 - **Motivazione:** Sfruttare le cache della CPU (L1, L2) che contengono dati e istruzioni del thread. Se il thread viene spostato su un altro core, le cache devono essere ricaricate, causando un rallentamento.
 - **Flessibilità:** Il SO può comunque spostare il thread se necessario (es. per bilanciare il carico o per far eseguire un thread a priorità più alta).
- **Hard Affinity:**
 - **Descrizione:** Il thread è **vincolato** a essere eseguito solo su un sottoinsieme specifico di core (o su un singolo core).
 - **Motivazione:** Usato in applicazioni real-time critiche o per ottimizzazioni estreme, dove è fondamentale evitare interruzioni o spostamenti.
 - **Flessibilità:** Molto rigido, può portare a un uso inefficiente della CPU se i core vincolati sono inattivi mentre altri sono sovraccarichi.

Scheduling Multi-Processore

Quando un sistema ha più CPU o core, lo scheduling diventa più complesso.

- **Multiprocessing Simmetrico (Symmetric Multiprocessing - SMP):**
 - **Descrizione:** Tutti i processori sono identici e condividono la stessa memoria fisica. Ogni processore può eseguire qualsiasi processo o thread.

- **Vantaggi:** Semplice da implementare (un'unica coda di processi pronti), bilanciamento del carico automatico.
- **Svantaggi:** Possibili colli di bottiglia sulla coda dei pronti (richiede sincronizzazione per l'accesso alla coda).
- **Multiprocessing Asimmetrico (Asymmetric Multiprocessing - AMP):**
 - **Descrizione:** Un processore (il "master") gestisce tutte le decisioni di scheduling e le operazioni di sistema, mentre gli altri processori (i "slave") eseguono solo il codice utente.
 - **Vantaggi:** Semplice, elimina la necessità di sincronizzazione complessa sulla coda dei pronti.
 - **Svantaggi:** Il master può diventare un collo di bottiglia. Meno robusto (se il master fallisce, tutto il sistema si blocca).

Bilanciamento del Carico (Load Balancing)

In sistemi SMP, è fondamentale distribuire il carico di lavoro in modo uniforme tra i processori per massimizzare l'utilizzo e le prestazioni.

- **Push Migration:**
 - **Descrizione:** Un processore specifico (o lo scheduler) controlla periodicamente il carico degli altri processori. Se rileva che un processore è sovraccarico, "spinge" (migra) alcuni dei suoi thread a processori meno carichi.
 - **Vantaggi:** Proattivo nel bilanciare il carico.
- **Pull Migration:**
 - **Descrizione:** Un processore inattivo o sottocarico "tira" (ruba) un thread dalla coda di un processore più carico.
 - **Vantaggi:** Reattivo, i processori si auto-bilanciano.

🔗 Spesso, i sistemi moderni usano una combinazione di push e pull migration per un bilanciamento del carico efficace.

🧵 Scheduling dei Thread: SCS vs PCS

Quando si schedulano i thread, è importante capire chi ha il controllo della schedulazione.

- **Process-Contention Scope (PCS):**
 - **Descrizione:** La competizione per la CPU avviene **solo tra i thread utente all'interno dello stesso processo**. La schedulazione è gestita dalla libreria dei thread a livello utente.
 - **Relazione con Modelli:** Tipico del modello Many-to-One.
 - **Limitazioni:** Il kernel vede solo il processo come un'unica unità di esecuzione. Non c'è parallelismo reale su più core per i thread dello stesso processo.
- **System-Contention Scope (SCS):**
 - **Descrizione:** La competizione per la CPU avviene **tra tutti i thread kernel del sistema**, indipendentemente dal processo a cui appartengono. La schedulazione è gestita direttamente dal kernel.
 - **Relazione con Modelli:** Tipico dei modelli One-to-One e Many-to-Many.
 - **Vantaggi:** Permette il parallelismo reale dei thread su più core. Se un thread si blocca, il kernel può schedulare un altro thread kernel (anche di un altro processo).

🔗 In sistemi moderni come Linux e Windows, dove il modello One-to-One è predominante, la schedulazione dei thread avviene principalmente in **System-Contention Scope (SCS)**, poiché il kernel è consapevole di ogni singolo thread.

📊 Scheduling con Code Multiple e Code Multiple con Feedback

Per gestire la diversità dei processi (CPU-bound, I/O-bound, interattivi, batch), molti sistemi operativi utilizzano schemi di scheduling più complessi che combinano diversi algoritmi.

Code Multiple (Multilevel Queue Scheduling)

✍ Lo **scheduling con code multiple** divide la coda dei processi pronti in più code separate, ognuna con il proprio algoritmo di scheduling e priorità. I processi vengono assegnati a una coda specifica in base a una caratteristica (es. tipo di processo, priorità).

✍ Immagina un aeroporto con diverse file per il check-in: una per i passeggeri VIP (alta priorità, es. Round Robin con quanto piccolo), una per i passeggeri normali (media priorità, es. Round Robin), e una per i gruppi turistici (bassa priorità, es. FCFS).

Caratteristiche:

- **Partizionamento**: I processi sono permanentemente assegnati a una coda (es. processi di sistema, processi interattivi, processi batch).
- **Priorità tra Code**: Le code hanno priorità fisse. Ad esempio, i processi della coda di priorità più alta vengono eseguiti prima di quelli della coda inferiore.
- **Scheduling tra Code**: Può essere implementato con:
 - **Fixed-priority preemptive scheduling**: I processi delle code a priorità più alta hanno sempre la precedenza.
 - **Time slice tra code**: Ogni coda riceve una certa percentuale del tempo della CPU.

Vantaggi: Adatto a diversi tipi di processi.

Svantaggi: Può causare starvation per le code a bassa priorità.

Code Multiple con Feedback (Multilevel Feedback Queue Scheduling)

✍ Lo **scheduling con code multiple con feedback** è una versione più flessibile dello scheduling a code multiple. I processi possono **spostarsi tra le diverse code** in base al loro comportamento (feedback).

✍ Tornando all'aeroporto: se un passeggero VIP (coda alta priorità) impiega troppo tempo al check-in (è CPU-bound), viene spostato in una coda di priorità inferiore. Se un passeggero normale (coda media priorità) è molto veloce (I/O-bound), può essere spostato in una coda di priorità più alta per un accesso più rapido.

Come funziona:

- **Code multiple**: Simile allo scheduling a code multiple.
- **Movimento tra code**:
 - Un processo che consuma troppo tempo di CPU in una coda (es. esaurisce il suo quanto di tempo) viene **declassato** a una coda di priorità inferiore (es. da RR con quanto piccolo a RR con quanto grande, o a FCFS). Questo penalizza i processi CPU-bound.
 - Un processo che si blocca per I/O (e quindi rilascia la CPU) e poi ritorna pronto, può essere **promosso** a una coda di priorità più alta. Questo favorisce i processi I/O-bound e interattivi.
 - **Aging**: Può essere implementato per prevenire la starvation, promuovendo un processo che aspetta da troppo tempo in una coda inferiore.

Vantaggi:

- **Flessibile**: Si adatta dinamicamente al comportamento dei processi.
- **Favorisce I/O-bound/Interattivi**: Dà loro un accesso più rapido alla CPU.
- **Previene Starvation** (con aging).

Svantaggi: Molto complesso da configurare (quante code, quale algoritmo per ogni coda, quando spostare i processi).

Stima della Durata del CPU Burst: Exponential Averaging

Molti algoritmi di scheduling (come SJF e SRTF) richiedono la conoscenza della durata del prossimo CPU burst. Poiché non è possibile prevedere il futuro, si utilizzano tecniche di stima basate sui burst passati. L'**Exponential Averaging** è un metodo comune per fare questa stima.

La formula è:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

Dove:

- τ_{n+1} è la stima del prossimo CPU burst.
- t_n è la durata effettiva dell'ultimo CPU burst.
- τ_n è la stima del CPU burst precedente.
- α (alfa) è un fattore di peso (smoothing factor) compreso tra 0 e 1.

Come funziona:

- Se α è vicino a 0, la stima dà più peso ai burst passati (la stima è più "liscia" e meno reattiva alle fluttuazioni).
- Se α è vicino a 1, la stima dà più peso all'ultimo burst effettivo (la stima è più reattiva ai cambiamenti recenti).

🔥 **Pensa a un cuoco che stima quanto tempo ci vorrà per il prossimo piatto. Se usa un α alto, si basa molto sull'ultimo piatto che ha fatto. Se usa un α basso, considera la media di tutti i piatti passati.**

Valutazione degli Algoritmi di Scheduling

✍ La **valutazione degli algoritmi di scheduling** è il processo di analizzare e confrontare le prestazioni di diversi algoritmi per determinare quale sia il più adatto a un dato carico di lavoro o obiettivo di sistema.

Esistono diversi metodi per valutare gli algoritmi:

1. Modelli Deterministici:

- **Descrizione:** Prende un carico di lavoro predefinito (un set di processi con tempi di arrivo e burst noti) e calcola le metriche di performance (tempo di attesa, turnaround) per ogni algoritmo.
- **Vantaggi:** Semplice da implementare e capire.
- **Svantaggi:** I risultati sono validi solo per quel carico di lavoro specifico. Non riflette il comportamento dinamico di un sistema reale.

2. Modelli di Code (Queueing Models):

- **Descrizione:** Utilizza la teoria delle code per modellare il sistema come una serie di code e server. I processi arrivano, attendono in una coda e vengono serviti dalla CPU (il server).
- **Vantaggi:** Permette di analizzare il comportamento del sistema sotto carichi variabili e di stimare metriche come il tempo di attesa medio e il throughput.
- **Svantaggi:** Richiede assunzioni semplificative sul comportamento dei processi (es. distribuzioni di arrivo e burst time) che potrebbero non essere sempre realistiche.
- **Legge di Little:** Una formula fondamentale nella teoria delle code:

$$L = \lambda W$$

Dove:

- L è il numero medio di processi nel sistema (in coda + in esecuzione).
- λ (lambda) è il tasso di arrivo medio dei processi al sistema.
- W è il tempo medio di permanenza di un processo nel sistema (tempo di turnaround). Questa legge è valida sotto condizioni molto generali e può essere usata per correlare diverse metriche di performance.

3. Simulazioni:

- **Descrizione:** Si costruisce un modello software del sistema informatico e si esegue un algoritmo di scheduling su questo modello, alimentandolo con dati di input (generati casualmente o da "trace").
- **Trace:** Sequenze registrate di eventi reali (es. accessi a memoria, richieste I/O) da un sistema in esecuzione. Usare trace reali rende la simulazione più realistica.

- **Vantaggi:** Più accurato dei modelli deterministici, può gestire carichi di lavoro complessi e dinamici.
 - **Svantaggi:** Costoso da sviluppare e richiede molto tempo di esecuzione. La qualità dei risultati dipende dalla qualità del modello e dei dati di input.
4. **Implementazione e Misurazione (Prototyping):**
- **Descrizione:** Si implementa l'algoritmo di scheduling direttamente su un sistema reale e si misurano le sue prestazioni con carichi di lavoro reali.
 - **Vantaggi:** I risultati più accurati e realistici.
 - **Svantaggi:** Molto costoso, rischioso (un bug nello scheduler può bloccare il sistema), difficile isolare l'effetto del solo algoritmo.

Scheduling in Sistemi Operativi Specifici

I sistemi operativi moderni implementano algoritmi di scheduling complessi e altamente ottimizzati per gestire le diverse esigenze dei carichi di lavoro.

Scheduling in Linux

Lo scheduler di Linux ha subito diverse evoluzioni per adattarsi alle crescenti esigenze di sistemi multi-core e carichi di lavoro diversi:

- **Scheduler O(1) (Linux v2.5 - v2.6.22):**
 - **Descrizione:** Questo scheduler garantiva che il tempo per selezionare il prossimo processo da eseguire fosse costante ($O(1)$), indipendentemente dal numero di processi. Utilizzava due array di code di priorità (`active` ed `expired`).
 - **Crediti:** I processi ricevevano "crediti" in base alla loro priorità. Quando i crediti finivano, il processo si spostava nell'array `expired`.
 - **Problemi:** Non sempre ottimo per i sistemi desktop interattivi, poteva causare ritardi percepibili.
- **CFS (Completely Fair Scheduler) - Linux v2.6.23+:**
 - **Descrizione:** L'attuale scheduler predefinito di Linux. Non si basa su quanti di tempo fissi o priorità assolute, ma cerca di garantire una "quota equa" di tempo di CPU a ogni processo/thread.
 - **Virtual Deadline:** Ogni task ha una `vruntime` (virtual runtime) che rappresenta quanto tempo di CPU "virtuale" ha accumulato. Lo scheduler sceglie sempre il task con la `vruntime` più bassa.
 - **Classi di Scheduling:** Linux supporta diverse classi di scheduling (es. `SCHED_OTHER` per i processi normali, `SCHED_FIFO` e `SCHED_RR` per i processi real-time) che interagiscono con CFS.
 - **Vantaggi:** Estremamente equo, ottimo per carichi di lavoro interattivi e desktop, scala bene su sistemi multi-core.

Scheduling in Windows

Lo scheduler di Windows è un sistema basato su priorità, preemptive, progettato per fornire una buona reattività per le applicazioni interattive.

- **Dispatcher:** Il componente del kernel di Windows responsabile della schedulazione dei thread.
- **Priorità Preemptive:** I thread vengono eseguiti in base alla loro priorità. Un thread a priorità più alta preleva (interrompe) un thread a priorità più bassa.
- **Classi di Priorità:** I thread sono organizzati in 32 livelli di priorità. I processi hanno una "classe di priorità" (es. `REALTIME_PRIORITY_CLASS`, `HIGH_PRIORITY_CLASS`, `NORMAL_PRIORITY_CLASS`, `IDLE_PRIORITY_CLASS`) e i thread all'interno di un processo hanno una "priorità relativa" all'interno di quella classe.
- **Boosting:** Windows può temporaneamente aumentare la priorità di un thread (es. un thread che ha appena completato un'operazione I/O o che è in foreground) per migliorare la reattività.
- **Quantum:** Ogni thread riceve un quanto di tempo. Se il quanto scade, il thread viene spostato in fondo alla coda della sua priorità (se non ci sono thread a priorità più alta).

- **Multiprocessore**: Windows supporta lo scheduling su sistemi multiprocessore, cercando di bilanciare il carico tra i core e mantenendo l'affinità dei thread per sfruttare le cache.

IV. Sincronizzazione e Concorrenza

🚧 Il Problema della Sezione Critica

🔗 Una **sezione critica** è una porzione di codice che accede a una risorsa condivisa (es. una variabile) e che non deve essere eseguita da più di un thread contemporaneamente.

🔗 Analogia: Il Bagno Monoposto

La risorsa condivisa è un bagno con un solo WC. La sezione critica è l'atto di usare il bagno. Dobbiamo assicurarci che solo una persona alla volta possa entrare.

Una soluzione al problema della sezione critica deve garantire tre proprietà:

1. **Mutua Esclusione**: Se un processo è nella sua sezione critica, nessun altro può entrarci.
2. **Progresso**: Se nessuno è nella sezione critica, e qualcuno vuole entrare, la decisione su chi entrerà non può essere posticipata indefinitamente.
3. **Attesa Limitata (Bounded Waiting)**: C'è un limite al numero di volte che altri processi possono entrare nella loro sezione critica dopo che un processo ha fatto la sua richiesta e prima che tale richiesta venga soddisfatta.

🔧 Strumenti di Sincronizzazione

Soluzioni Software: Algoritmo di Peterson

🔗 L'**Algoritmo di Peterson** è una soluzione software classica per il problema della sezione critica per **due processi**. Garantisce mutua esclusione, progresso e attesa limitata.

Come funziona (per due processi P0 e P1):

Usa due variabili:

- **flag[2]**: un array booleano, **flag[i] = true** indica che **Pi** vuole entrare nella sezione critica.
- **turn**: una variabile intera, indica di chi è il "turno" di entrare.

```
// Variabili globali condivise
bool flag[2]; // Inizializzate a false
int turn;     // Inizializzata a 0 o 1

// Codice per il processo Pi (i=0 o i=1)
void enter_critical_section(int i, int j) { // j è l'altro processo (1-i)
    flag[i] = true; // Dico che voglio entrare
    turn = j;       // Do la priorità all'altro processo
    while (flag[j] && turn == j) {
        // Attesa attiva: l'altro processo vuole entrare E il suo turno
    }
}

void exit_critical_section(int i) {
    flag[i] = false; // Dico che ho finito
}
```

Vantaggi: Pura soluzione software, non richiede supporto hardware speciale.

Svantaggi: Funziona solo per due processi, usa busy-waiting.

Supporto Hardware: Istruzioni Atomiche

Per implementare la sincronizzazione, abbiamo bisogno di un'operazione di base che sia **atomica** (non interrompibile).

- `compare_and_swap(v, expected, new)` : Confronta il valore di `v` con `expected` . Se sono uguali, imposta `v` a `new` e restituisce `true` . Altrimenti, non fa nulla e restituisce `false` . Tutto in un unico, indivisibile passo.

Pseudocodice e Esempio di `compare_and_swap`

```
// Funzione atomica (implementata a livello hardware)
bool compare_and_swap(int* address, int expected_value, int new_value) {
    // Questa operazione è garantita essere indivisibile
    if (*address == expected_value) {
        *address = new_value;
        return true; // Successo: il valore è stato scambiato
    } else {
        return false; // Fallimento: il valore non corrispondeva a expected_value
    }
}
```

Esempio Pratico: Implementazione di un Mutex Semplice (Spinlock)

```
int lock = 0; // 0 = libero, 1 = occupato

void acquire_lock() {
    // Tenta di cambiare 'lock' da 0 a 1. Continua a provare finché non ci riesce.
    // Se compare_and_swap restituisce false, significa che il lock è già 1 (occupato).
    while (compare_and_swap(&lock, 0, 1) == false) {
        // Il lock è occupato, attendo (busy-waiting)
    }
}

void release_lock() {
    // Rilascia il lock impostandolo a 0.
    lock = 0;
}
```

Questo esempio mostra come `compare_and_swap` può essere usato per implementare un mutex: un thread tenta atomicamente di cambiare `lock` da 0 a 1. Se ci riesce, ha acquisito il lock. Se fallisce, significa che un altro thread lo ha già acquisito, e il thread corrente deve attendere.

Variabili Atomiche

Hai chiesto di aggiungere un paragrafo sulle variabili atomiche, ed è un'ottima idea per distinguere tra istruzioni a basso livello e tipi di dati più "utilizzabili"!

 Le **variabili atomiche** sono tipi di variabili speciali che, quando accedute o modificate, garantiscono che l'operazione completa sia eseguita in modo indivisibile (atomico), senza possibilità di interruzione o osservazione di stati intermedi da parte di altri thread.

Questo è cruciale per prevenire [race condition](#) in operazioni semplici come l'incremento di un contatore. A differenza dei mutex o dei semafori, che bloccano thread e hanno un overhead maggiore, le operazioni su variabili atomiche sono molto più leggere e sono "appropriate per update singoli di variabili (es. contatori)".

Esempio:

```
// In C11/C++11, si usa _Atomic o std::atomic
_Atomic int counter = 0;

void increment_counter() {
```

```
counter++; // Questa operazione è atomica, non richiede mutex espliciti
}
```

🔗 Le variabili atomiche non risolvono problemi di sincronizzazione complessi che coinvolgono molteplici risorse o stati composti. Per questi scenari, sono ancora necessari costrutti di più alto livello come mutex o monitor. Sono fantastiche per operazioni semplici e veloci, ma non sono una soluzione universale per tutti i problemi di concorrenza.

Barriere di Memoria (Memory Barriers / Memory Fences)

Hai chiesto di aggiungere le barriere di memoria, un concetto cruciale per i sistemi multiprocessore e multi-core!

🔗 Una **barriera di memoria (memory barrier o memory fence)** è un'istruzione che forza la CPU e il compilatore a completare tutte le operazioni di memoria (letture e scritture) che le precedono nell'ordine del programma, prima di eseguire qualsiasi operazione di memoria che la segue.

Perché sono necessarie?

Le CPU moderne e i compilatori riordinano le istruzioni per ottimizzare le prestazioni (es. per sfruttare meglio le cache o le pipeline). In un sistema a singolo core, questo riordino è trasparente e non causa problemi. Tuttavia, in un sistema multi-core o multiprocessore, se due core accedono alla stessa memoria, il riordino può portare a risultati inconsistenti, perché un core potrebbe vedere le operazioni di un altro core in un ordine diverso da quello in cui sono state scritte.

Scopo:

Le barriere di memoria garantiscono che le operazioni di memoria siano visibili a tutti i core in un ordine specifico, prevenendo problemi di consistenza dei dati dovuti al riordino delle istruzioni. Sono fondamentali per implementare correttamente primitive di sincronizzazione a basso livello (come spinlock o mutex) in ambienti concorrenti.

🔗 Immagina una barriera stradale. Tutti i veicoli (operazioni di memoria) che arrivano prima della barriera devono passare prima che qualsiasi veicolo che arriva dopo la barriera possa procedere.

Mutex Lock vs Spinlock

Strumento	Descrizione	Come attende?	Quando usarlo?
Mutex	Il lucchetto più semplice (<code>acquire</code> , <code>release</code>).	Attesa Passiva: Se il lock è occupato, il thread viene messo in stato di <code>Waiting</code> e il SO schedula un altro thread.	Per sezioni critiche potenzialmente lunghe.
Spinlock	Un lock che usa il busy-waiting.	Attesa Attiva: Se il lock è occupato, il thread entra in un ciclo <code>while</code> che controlla continuamente se il lock è libero, consumando CPU.	Solo su sistemi multicore e per sezioni critiche brevissime , dove il costo di un context switch è maggiore del costo dell'attesa attiva.

🚦 Semafori: I Guardiani del Traffico

🔗 Un **semaforo** è una variabile intera generalizzata che supporta due operazioni atomiche: `wait()` (o `P`) e `signal()` (o `V`).

🔗 Analogia: Noleggio Bici

Immagina una stazione di noleggio con 5 biciclette (il valore del semaforo è 5).

- `wait()` : Vuoi una bici. Se ce n'è almeno una (`S > 0`), prendi la bici e il contatore scende (`S--`). Se non ce ne sono (`S == 0`), ti metti in coda e aspetti.
- `signal()` : Restituisci una bici. Il contatore sale (`S++`). Se c'era qualcuno in coda, ne svegli uno che può ora prendere la bici.

- **Semaforo Binario:** Può valere solo 0 o 1. Funziona esattamente come un mutex.

- **Semaforo Contatore**: Può avere qualsiasi valore non negativo. Utile per gestire un pool di N risorse.

Relazione tra Mutex e Semaforo Contatore:

Questo è un punto di potenziale confusione, ma è semplice una volta chiarito!

Un **Mutex** è una forma specializzata di **Semaforo binario**.

- I **Semafori contatori** sono il concetto più generale: possono contare fino a N risorse disponibili.
- I **Semafori binari** sono un caso particolare di semaforo contatore, con un valore massimo di 1 (0 o 1). Sono usati per garantire la mutua esclusione.
- Un **Mutex** è un tipo di semaforo binario, ma con semantiche aggiuntive che lo rendono più adatto per la protezione di sezioni critiche:
 - **Ownership**: Solo il thread che ha acquisito il mutex può rilasciarlo. Questo previene errori in cui un thread rilascia un lock detenuto da un altro.
 - **Priority Inheritance**: Molti mutex supportano il protocollo di ereditarietà della priorità per prevenire l'inversione di priorità.

Quindi, pensa al semaforo contatore come al "genitore" più generico, al semaforo binario come un "figlio" specializzato per la mutua esclusione, e al mutex come una "versione migliorata" del semaforo binario con regole più stringenti e funzionalità aggiuntive per la robustezza.

Implementazione con Code di Attesa:

Per evitare il busy-waiting (che spreca cicli di CPU), i semafori sono tipicamente implementati con una **coda di attesa**.

- Quando un processo esegue `wait()` e il semaforo è 0 (o non ci sono risorse disponibili), il processo viene **bloccato** e messo in una coda associata al semaforo. Il sistema operativo schedula un altro processo.
- Quando un processo esegue `signal()`, se ci sono processi in attesa nella coda del semaforo, uno di essi viene **risvegliato** (spostato nello stato `Ready`) e può tentare di acquisire la risorsa.

Problema Produttore-Consumatore (con semafori)

Il problema Produttore-Consumatore è un classico per capire l'uso dei semafori!

 Il **Problema Produttore-Consumatore** descrive due tipi di processi: un **produttore** che genera dati e li inserisce in un buffer condiviso, e un **consumatore** che preleva dati da questo buffer. L'obiettivo è garantire che il produttore non tenti di aggiungere dati a un buffer pieno e che il consumatore non tenti di prelevare dati da un buffer vuoto, e che l'accesso al buffer sia mutuamente esclusivo.

Soluzione con Semafori:

Per implementare la soluzione, utilizziamo tre semafori:

- **mutex**: Un semaforo binario (inizializzato a 1) per garantire la mutua esclusione nell'accesso al buffer condiviso.
- **empty**: Un semaforo contatore (inizializzato alla dimensione **N** del buffer) che rappresenta il numero di slot vuoti disponibili nel buffer.
- **full**: Un semaforo contatore (inizializzato a 0) che rappresenta il numero di slot pieni nel buffer.

Pseudocodice:

```
semaphore mutex = 1;    // Per mutua esclusione sul buffer
semaphore empty = N;    // N = dimensione massima del buffer
semaphore full = 0;     // Inizialmente il buffer è vuoto

Producer() {
    while (true) {
        // 1. Produce un elemento (non nella sezione critica)
        // produce_item()

        // 2. Attende che ci sia almeno uno slot vuoto nel buffer
        wait(empty);      // Decrementa i posti vuoti, attende se empty == 0
    }
}
```

```

// 3. Entra nella sezione critica per accedere al buffer
wait(mutex);      // Acquisisce il lock per la mutua esclusione
// add_item_to_buffer() // Aggiunge l'elemento al buffer
signal(mutex);    // Rilascia il lock

// 4. Segnala che c'è un elemento in più nel buffer
signal(full);     // Incrementa i posti pieni
}
}

Consumer() {
    while (true) {
        // 1. Attende che ci sia almeno un elemento pieno nel buffer
        wait(full);      // Decrementa i posti pieni, attende se full == 0

        // 2. Entra nella sezione critica per accedere al buffer
        wait(mutex);     // Acquisisce il lock per la mutua esclusione
        // remove_item_from_buffer() // Rimuove l'elemento dal buffer
        signal(mutex);   // Rilascia il lock

        // 3. Segnala che c'è uno slot vuoto in più nel buffer
        signal(empty);   // Incrementa i posti vuoti

        // 4. Consuma l'elemento (non nella sezione critica)
        // consume_item()

    }
}

```

🔥 Questo pseudocodice è un modello fondamentale. Nota come `wait(empty)` e `wait(full)` gestiscono la capacità del buffer, mentre `wait(mutex)` e `signal(mutex)` garantiscono che solo un processo alla volta modifichi il buffer.

👁 Monitor e Variabili di Condizione

🔧 Implementazione completa dei "Monitor" come primitiva di Sincronizzazione di Alto Livello

Hai fatto un'ottima richiesta! I Monitor sono un costrutto di sincronizzazione di alto livello che semplifica notevolmente la programmazione concorrente rispetto ai semafori.

🔧 Un **Monitor** è un'astrazione di alto livello che fornisce un meccanismo di sincronizzazione (introdotti da Hoare nel 1974). È un tipo di dato astratto che incapsula dati condivisi e le procedure che operano su di essi. Garantisce che solo un processo (o thread) alla volta possa essere attivo all'interno delle sue procedure (mutua esclusione implicita).

Struttura:

Un monitor tipicamente include:

- **Dati privati condivisi**: Le variabili che devono essere protette.
- **Procedure (o funzioni)**: Le uniche operazioni che possono accedere ai dati condivisi del monitor.
- **Codice di inizializzazione**: Per impostare lo stato iniziale del monitor.

La mutua esclusione è **automatica**: il compilatore o il runtime garantiscono che solo un thread alla volta possa eseguire una delle procedure del monitor. Se un thread tenta di entrare in una procedura mentre un altro è già attivo, viene messo in coda e attende.

Relazione con Variabili di Condizione:

I monitor, da soli, non permettono ai thread di attendere specifiche condizioni (es. "buffer non pieno"). Per questo, si usano le **variabili di condizione**.

🔧 Una **Variabile di Condizione** è uno strumento usato **all'interno** di un monitor per permettere a un thread di attendere che una certa condizione diventi vera, rilasciando temporaneamente il lock del monitor.

Operazioni sulla Variabile di Condizione:

- `cond.wait()` : Il thread si mette in attesa sulla variabile di condizione e **rilascia il lock del monitor**, permettendo ad altri thread di entrare nel monitor. Il thread viene riattivato solo da una `signal()` o `broadcast()`.
- `cond.signal()` : Risveglia **un** thread in attesa su quella condizione (se ce n'è uno). Se non ci sono thread in attesa, l'operazione non ha effetto.
- `cond.broadcast()` : Risveglia **tutti** i thread in attesa su quella condizione.

Vantaggi dei Monitor:

- **Semplifica la programmazione concorrente**: La mutua esclusione è gestita automaticamente dal costrutto, riducendo gli errori comuni (es. dimenticare di acquisire/rilasciare un mutex).
- **Migliora la leggibilità**: Il codice di sincronizzazione è centralizzato e più chiaro.

Svantaggi dei Monitor:

- **Meno flessibile**: Per schemi di sincronizzazione molto complessi che non si adattano bene al modello "un solo thread attivo alla volta", i semafori potrebbero offrire maggiore granularità.
- **Supporto linguistico**: Richiede il supporto diretto dal linguaggio di programmazione (es. Java con `synchronized` e `wait()/notify()`, C# con `lock` e `Monitor.Wait()/Pulse()`).

Semantiche di `signal()` nelle Variabili di Condizione

Esistono diverse implementazioni di `signal()` per le variabili di condizione, che influenzano il comportamento dei thread:

- **Signal and Continue (o Hoare Semantics)**:
 - Il thread che esegue `signal()` **continua la sua esecuzione** all'interno del monitor.
 - Il thread risvegliato (se ce n'è uno) viene spostato nella coda dei "ready" (o "entry queue" del monitor) e attende di riacquisire il lock del monitor prima di poter proseguire.
 - **Vantaggi**: Semplice da implementare.
 - **Svantaggi**: Il thread risvegliato potrebbe dover attendere che il thread che ha segnalato rilasci il lock del monitor, e la condizione che lo ha risvegliato potrebbe non essere più vera quando finalmente ottiene il controllo. Richiede che il thread risvegliato ricontrolli la condizione (spesso con un `while` loop). Questa è la semantica più comune (es. Pthreads).
- **Signal and Wait (o Mesa Semantics)**:
 - Il thread che esegue `signal()` si **sospende immediatamente** e cede il lock del monitor al thread risvegliato.
 - Il thread risvegliato riprende immediatamente l'esecuzione all'interno del monitor.
 - Il thread che ha segnalato riprenderà solo quando il thread risvegliato rilascerà il lock o si sospenderà di nuovo.
 - **Vantaggi**: Garantisce che la condizione sia ancora vera per il thread risvegliato.
 - **Svantaggi**: Più complesso da implementare, può portare a un overhead maggiore dovuto ai context switch.

Dettagli su "Conditional-Wait con Priorità" nelle Variabili di Condizione

Ottima intuizione! Alcune implementazioni avanzate delle variabili di condizione permettono un controllo più granulare sul risveglio dei thread:

È possibile estendere l'operazione `wait()` con un parametro di priorità, ad esempio `x.wait(c)`. Qui, `c` indica un valore di priorità (es. un intero che rappresenta il tempo di esecuzione più basso, la deadline più vicina, o un'altra metrica di urgenza). Quando un thread esegue `signal()` o `broadcast()` su `x`, lo scheduler risveglia il processo con la **priorità più alta** tra quelli in attesa su quella variabile di condizione.

Questo aggiunge granularità al controllo dei processi in attesa, permettendo di implementare politiche di scheduling più sofisticate all'interno dei monitor, specialmente in sistemi real-time o dove la reattività di certi task è cruciale.

Problema Lettori-Scrittori (soluzione con priorità ai lettori)

Anche il problema Lettori-Scrittori è un classico di sincronizzazione, e la sua soluzione con priorità ai lettori è un ottimo esempio!

Il Problema Lettori-Scrittori coinvolge processi che accedono a un database o a una risorsa condivisa.

- I **lettori** possono accedere alla risorsa contemporaneamente senza problemi.
- Gli **scrittori** devono avere accesso esclusivo: quando uno scrittore sta modificando la risorsa, nessun altro lettore o scrittore può accedervi.

Soluzione "Prima Variante" (Priorità ai Lettori):

Questa soluzione favorisce i lettori, permettendo loro di entrare nella sezione critica anche se ci sono scrittori in attesa.

- **Variabili:**
 - `semaphore rw_mutex = 1;` : Un semaforo binario che controlla l'accesso in scrittura e l'accesso del *primo* lettore.
 - `semaphore mutex = 1;` : Un semaforo binario per proteggere la variabile `read_count`.
 - `int read_count = 0;` : Un contatore che tiene traccia del numero di lettori attivi nella sezione critica.
- **Logica:**
 - **Scrittore:**
 - Prima di scrivere, lo scrittore esegue `wait(rw_mutex)`. Se `rw_mutex` è a 0 (perché ci sono lettori o altri scrittori), lo scrittore si blocca.
 - Una volta acquisito `rw_mutex`, lo scrittore ha accesso esclusivo e può scrivere.
 - Al termine, rilascia `signal(rw_mutex)`.
 - **Lettore:**
 - Quando un lettore arriva, acquisisce `mutex` per aggiornare `read_count`.
 - Incrementa `read_count`.
 - Se è il *primo* lettore (`read_count == 1`), acquisisce `rw_mutex` per bloccare gli scrittori.
 - Rilascia poi `mutex`.
 - Dopo aver letto i dati, il lettore riacquisisce `mutex`, decrementa `read_count`.
 - Se è l'*ultimo* lettore (`read_count == 0`), rilascia `rw_mutex` per permettere agli scrittori di accedere.
 - Infine, rilascia `mutex`.

Pseudocodice:

```
semaphore rw_mutex = 1; // Controls access for writers AND first/last reader
semaphore mutex = 1;    // Protects read_count
int read_count = 0;     // Number of active readers

Writer() {
    while (true) {
        wait(rw_mutex); // Acquire exclusive lock for writing
        // Write data (critical section)
        signal(rw_mutex); // Release exclusive lock
    }
}

Reader() {
    while (true) {
        wait(mutex); // Acquire lock to modify read_count
        read_count++;
        if (read_count == 1) { // If first reader, block writers
            wait(rw_mutex); // Block writers if this is the first reader
        }
        signal(mutex); // Release lock on read_count

        // Read data (critical section, multiple readers allowed)

        wait(mutex); // Acquire lock to modify read_count
        read_count--;
        if (read_count == 0) { // If last reader, unblock writers
            signal(rw_mutex); // Unblock writers if this is the last reader
        }
    }
}
```



```
    signal(mutex);    // Release lock on read_count
}
}
```

🔥 Questa soluzione può causare **starvation** per gli scrittori se c'è un flusso continuo di lettori. Se i lettori continuano ad arrivare, **read_count** non raggiungerà mai 0, e **rw_mutex** non verrà mai rilasciato per gli scrittori.

🔒 Read-Write Locks (Lock Lettura-Scrittura)

✍ Un **Read-Write Lock** è un meccanismo di sincronizzazione che consente a più thread di accedere a una risorsa condivisa in modalità **lettura** contemporaneamente, ma limita l'accesso in modalità **scrittura** a un solo thread per volta.

✍ Pensa a una biblioteca. Molte persone possono leggere lo stesso libro contemporaneamente (lettori). Ma solo una persona alla volta può modificare il catalogo dei libri (scrittore).

Regole:

- Se un thread sta scrivendo, nessun altro thread (né lettore né scrittore) può accedere alla risorsa.
- Se uno o più thread stanno leggendo, altri lettori possono accedere, ma nessuno scrittore può farlo.

🖥 Sincronizzazione in Sistemi Operativi Specifici

Le primitive di sincronizzazione di base (mutex, semafori) sono concetti universali, ma la loro implementazione e le loro varianti possono differire tra i sistemi operativi.

- **Linux:**
 - **Futex (Fast Userspace Mutex):** È una primitiva di sincronizzazione a basso livello che permette a mutex e semafori di operare principalmente in user-space, passando al kernel solo quando è strettamente necessario (es. quando un thread deve bloccarsi). Questo riduce l'overhead delle system call.
 - **Pthreads:** Linux implementa lo standard POSIX Threads (Pthreads), che fornisce API per mutex, semafori, variabili di condizione e read-write locks.
- **Windows:**
 - **Critical Section:** Un meccanismo di mutua esclusione più leggero dei mutex, utilizzabile solo tra thread dello stesso processo.
 - **Mutex, Semaphores, Events:** Windows offre oggetti kernel per la sincronizzazione che possono essere usati tra thread di processi diversi. Gli "eventi" sono un meccanismo flessibile per segnalare il verificarsi di un evento.
 - **Spinlocks:** Usati internamente dal kernel per proteggere strutture dati critiche.
- **Solaris:**
 - **Adaptive Mutexes:** Un tipo di mutex che si comporta come uno spinlock se il thread che detiene il lock è in esecuzione su un altro processore (e quindi il lock sarà rilasciato presto). Se il thread che detiene il lock è bloccato o non in esecuzione, l'adaptive mutex si comporta come un mutex normale, mettendo il thread in attesa. Questo ottimizza le performance su sistemi multi-processore.
 - **Readers-Writers Locks:** Implementazioni efficienti di lock lettura-scrittura.

🔥 La comprensione delle implementazioni specifiche aiuta a capire come i principi teorici si traducono in pratica e quali compromessi vengono fatti per ottimizzare le prestazioni in ambienti diversi.

🍲 Problemi Classici di Sincronizzazione

- **Produttore-Consumatore**: Un processo "produttore" genera dati e li mette in un buffer; un "consumatore" li preleva. Si usano semafori per gestire il buffer (pieno/vuoto) e la mutua esclusione.
- **Lettori-Scrittori**: Molti processi "lettori" possono leggere un dato contemporaneamente, ma uno "scrittore" deve avere accesso esclusivo.
 - **Problema**: Se ci sono sempre lettori, uno scrittore potrebbe non riuscire mai ad accedere (starvation dello scrittore). Viceversa, se uno scrittore ha la priorità, i lettori potrebbero essere bloccati a lungo.
 - **Soluzione (Esempio con priorità ai lettori)**:
 - Si usa un mutex per proteggere il contatore dei lettori.
 - Un semaforo binario (o mutex) per proteggere la risorsa condivisa per gli scrittori.
 - Quando il primo lettore entra, acquisisce il lock per la scrittura. Quando l'ultimo lettore esce, lo rilascia.
 - Questo può causare starvation per gli scrittori se ci sono sempre lettori. Soluzioni più complesse bilanciano le priorità.
- **Filosofi a Cena**: 5 filosofi, 5 bacchette. Per mangiare, un filosofo ha bisogno di due bacchette. Se tutti prendono la bacchetta alla loro sinistra contemporaneamente, si crea un [deadlock](#).

Problema dei Filosofi a Cena (Soluzione per Evitare Deadlock)

Il problema dei Filosofi a Cena è un classico per mettere alla prova la tua comprensione della sincronizzazione e del deadlock! Hai chiesto una soluzione dettagliata, e qui te la propongo, basata sull'idea di evitare il deadlock.

 Il **Problema dei Filosofi a Cena** è un problema classico di sincronizzazione che illustra i pericoli del deadlock e della starvation. Cinque filosofi si siedono attorno a un tavolo, con una bacchetta tra ogni coppia di filosofi. Per mangiare, un filosofo ha bisogno di entrambe le bacchette ai suoi lati. Se tutti i filosofi prendono la bacchetta alla loro sinistra contemporaneamente, si verifica un deadlock.

Soluzione con Semafori e Stato dei Filosofi (per prevenire Deadlock):

Una soluzione comune per prevenire il deadlock è permettere a un filosofo di prendere le bacchette solo se **entrambe** sono disponibili. Questo richiede di mantenere uno stato per ogni filosofo e di usare un mutex per proteggere l'accesso a questo stato.

- **Variabili**:
 - `semaphore mutex = 1;` : Un semaforo binario per la mutua esclusione sull'accesso alle variabili di stato dei filosofi (la sezione critica per il controllo delle bacchette).
 - `semaphore self[5];` : Un array di semafori, uno per ogni filosofo, inizializzati a 0. Un filosofo si blocca sul suo `self` semaforo se non può mangiare.
 - `enum { THINKING, HUNGRY, EATING } state[5];` : Un array che tiene traccia dello stato di ogni filosofo.
- **Funzioni Ausiliarie**:
 - `test(i)` : Una funzione che verifica se il filosofo `i` può mangiare.
 - Se `state[i] == HUNGRY` E `chopstick[LEFT] (i.e., state[(i+4)%5]) != EATING` E `chopstick[RIGHT] (i.e., state[(i+1)%5]) != EATING` :
 - Allora `state[i] = EATING`
 - `signal(self[i])` (risveglia il filosofo `i` se era in attesa)
- **Logica del Filosofo `i`**:

```
// Variabili globali condivise
semaphore mutex = 1;
semaphore self[5] = {0, 0, 0, 0, 0}; // Tutti i semafori inizializzati a 0
enum { THINKING, HUNGRY, EATING } state[5]; // Tutti i filosofi inizialmente THINKING

// Funzione per controllare se il filosofo i può mangiare
void test(int i) {
    if (state[i] == HUNGRY &&
```

```

        state[(i + 4) % 5] != EATING && // Bacchetta sinistra non usata
        state[(i + 1) % 5] != EATING) { // Bacchetta destra non usata
            state[i] = EATING;
            signal(self[i]); // Risveglia il filosofo i se era bloccato
        }
    }

// Funzione per il filosofo i per prendere le bacchette
void pickup(int i) {
    wait(mutex); // Entra nella sezione critica per modificare lo stato
    state[i] = HUNGRY; // Il filosofo i diventa affamato
    test(i); // Tenta di prendere le bacchette
    signal(mutex); // Esce dalla sezione critica
    wait(self[i]); // Se non può mangiare, si blocca sul suo semaforo
}

// Funzione per il filosofo i per rilasciare le bacchette
void putdown(int i) {
    wait(mutex); // Entra nella sezione critica
    state[i] = THINKING; // Il filosofo i smette di mangiare
    test((i + 4) % 5); // Controlla se il vicino sinistro può mangiare
    test((i + 1) % 5); // Controlla se il vicino destro può mangiare
    signal(mutex); // Esce dalla sezione critica
}

// Codice principale per ogni filosofo
void philosopher(int i) {
    while (true) {
        // Filosofo pensa
        // think();

        pickup(i); // Tenta di prendere le bacchette
        // Filosofo mangia
        // eat();

        putdown(i); // Rilascia le bacchette
    }
}

```

Spiegazione:

1. Quando un filosofo diventa **HUNGRY**, tenta di prendere le bacchette chiamando **pickup()**.
2. All'interno di **pickup()**, acquisisce un **mutex** per garantire che solo un filosofo alla volta possa modificare lo stato globale (i **state** degli altri filosofi).
3. Imposta il proprio stato a **HUNGRY** e chiama **test()** per vedere se può mangiare.
4. **test()** controlla se le bacchette sono libere (cioè, se i vicini non stanno mangiando). Se sì, il filosofo passa allo stato **EATING** e viene risvegliato dal suo semaforo **self[i]**.
5. Se **test()** fallisce, il filosofo rilascia il **mutex** e si blocca sul proprio semaforo **self[i]**, attendendo di essere risvegliato.
6. Quando un filosofo finisce di mangiare e chiama **putdown()**, rilascia le bacchette e poi chiama **test()** per i suoi vicini, permettendo loro di mangiare se erano in attesa.

Questa soluzione previene il deadlock perché un filosofo non prende mai una sola bacchetta e poi aspetta l'altra indefinitamente. Prende entrambe o nessuna. Tuttavia, può ancora soffrire di **starvation** in alcune condizioni, se un filosofo è sfortunato e i suoi vicini continuano a mangiare prima che lui abbia la possibilità.

Inversione di Priorità: Il Caso Mars Pathfinder

Il caso del **Mars Pathfinder** è un esempio storico e molto noto del problema dell'inversione di priorità, avvenuto nel 1997.

- **Il Problema:** Il rover "Sojourner" su Marte, parte della missione Mars Pathfinder, soffriva di riavvii inaspettati del sistema. Dopo un'attenta analisi, si scoprì che la causa era un problema di inversione di priorità nello scheduler del sistema operativo (VxWorks).
- **Lo Scenario:**
 - C'era un task ad **alta priorità** che gestiva la comunicazione con la Terra.
 - C'era un task a **media priorità** che eseguiva calcoli meteorologici.
 - C'era un task a **bassa priorità** che raccoglieva dati da un bus di dati condiviso (protetto da un mutex).
 - Il task a bassa priorità acquisiva il mutex e iniziava a leggere dal bus.
 - A questo punto, l'alta priorità si attivava e tentava di acquisire lo stesso mutex, ma era bloccato perché il task a bassa priorità lo deteneva.
 - Mentre il task ad alta priorità era bloccato, il task a media priorità (che aveva una priorità più alta di P_bassa) prelazionava il task a bassa priorità.
 - Risultato: Il task a bassa priorità non poteva rilasciare il mutex, il task ad alta priorità rimaneva bloccato, e il task a media priorità (meno critico) veniva eseguito, causando un ritardo inaccettabile per il task di comunicazione e, alla fine, un watchdog timer faceva riavviare il sistema.
- **La Soluzione:** Fu implementato il **Priority Inheritance Protocol**. Quando il task ad alta priorità si bloccava su un mutex detenuto da un task a bassa priorità, la priorità del task a bassa priorità veniva temporaneamente elevata a quella del task ad alta priorità. Questo permetteva al task a bassa priorità di completare rapidamente la sua sezione critica e rilasciare il mutex, sbloccando il task ad alta priorità.

Livelock

Hai chiesto di definire il livelock, un'altra forma di "fallimento di liveness" che è importante distinguere dal deadlock!

✍ Il **livelock** è una situazione di "fallimento di liveness" (come il deadlock) in cui due o più processi cambiano continuamente il loro stato in risposta l'uno all'altro, senza fare alcun progresso significativo.

A differenza del deadlock, i processi in livelock **non sono bloccati** e continuano a eseguire istruzioni. Tuttavia, il sistema nel suo complesso non avanza perché le azioni dei processi si annullano o si evitano reciprocamente in un ciclo continuo.

✍ Immagina due persone che si scontrano in un corridoio stretto. Entrambe si spostano contemporaneamente a destra per evitarsi, ma si scontrano di nuovo. Poi entrambe si spostano a sinistra, e si scontrano ancora. Continuano a muoversi, ma non riescono mai a passarsi. Sono attive, ma non fanno progressi.

Differenza chiave con il Deadlock:

- **Deadlock:** I processi sono **bloccati** e non eseguono alcuna istruzione.
- **Livelock:** I processi sono **attivi** e continuano a eseguire istruzioni, ma in un ciclo improduttivo.

Liveness

✍ La **liveness** (o "vivacità") è una proprietà di un sistema concorrente che garantisce che "qualcosa di buono accadrà alla fine". Si riferisce alla capacità del sistema di fare progressi e non rimanere bloccato indefinitamente.

- **Fallimenti di Liveness:**
 - **Deadlock:** Nessun processo può fare progressi perché tutti sono bloccati in attesa di risorse detenute da altri.
 - **Livelock:** I processi cambiano stato continuamente in risposta l'uno all'altro, ma senza fare alcun progresso significativo. Sono attivi, ma improduttivi.
 - **Starvation:** Un processo non riesce mai ad ottenere una risorsa o l'accesso alla CPU, rimanendo indefinitamente in attesa, nonostante la risorsa o la CPU diventino disponibili per

altri processi.

Livelli di Contesa

La **contesa** si riferisce al grado in cui più thread o processi tentano di accedere contemporaneamente alla stessa risorsa condivisa. La gestione della contesa ha un impatto significativo sulle prestazioni.

- **Strumenti Lock-Free:**
 - **Descrizione:** Algoritmi e strutture dati che permettono a più thread di accedere a dati condivisi senza l'uso di lock (mutex, semafori). Utilizzano istruzioni atomiche a basso livello (es. `compare_and_swap`).
 - **Vantaggi:** Evitano i problemi di deadlock e starvation associati ai lock. Possono migliorare la scalabilità su sistemi multi-core.
 - **Svantaggi:** Estremamente complessi da progettare e implementare correttamente.
- **Approccio Ottimistico vs. Pessimistico:**
 - **Approccio Pessimistico (Locking):** Si assume che le collisioni siano frequenti. Si acquisisce un lock sulla risorsa *prima* di accedervi, prevenendo le collisioni. (Es. Mutex, Semafori).
 - **Approccio Ottimistico (Lock-Free, Transactional Memory):** Si assume che le collisioni siano rare. Si tenta di accedere e modificare la risorsa senza lock. Se si rileva una collisione, l'operazione viene annullata e ritentata. (Es. Variabili atomiche, Software Transactional Memory).
- **Performance e Contesa:**
 - Quando la contesa è **bassa**, gli approcci ottimistici o lock-free possono essere più veloci perché evitano l'overhead dei lock.
 - Quando la contesa è **alta**, gli approcci pessimistici (locking) possono essere più efficienti perché il costo del rollback e dei ritenti negli approcci ottimistici diventa proibitivo.

V. Deadlock: L'Abbraccio Mortale

 Un **deadlock** è una situazione in cui due o più processi sono bloccati indefinitamente, ciascuno in attesa di una risorsa che è posseduta da un altro processo in attesa.

Analogia: L'Incrocio Bloccato

Quattro auto arrivano a un incrocio a 4 stop contemporaneamente. Ognuna vuole avanzare, ma la strada è bloccata dall'auto alla sua destra. Nessuno si muove.

Le Quattro Condizioni del Deadlock

Un deadlock può verificarsi solo se queste quattro condizioni sono vere **simultaneamente**:

1. **Mutua Esclusione:** Almeno una risorsa non può essere condivisa (es. una stampante).
2. **Possesso e Attesa (Hold and Wait):** Un processo possiede almeno una risorsa e ne sta aspettando un'altra posseduta da un altro processo.
3. **Non Prelazione (No Preemption):** Una risorsa può essere rilasciata solo volontariamente dal processo che la detiene.
4. **Attesa Circolare (Circular Wait):** Esiste un ciclo di processi $\{P_0, P_1, \dots, P_n\}$ tale che P_0 aspetta una risorsa di P_1 , P_1 aspetta una risorsa di P_2 , ..., e P_n aspetta una risorsa di P_0 .

Grafo di Allocazione delle Risorsse

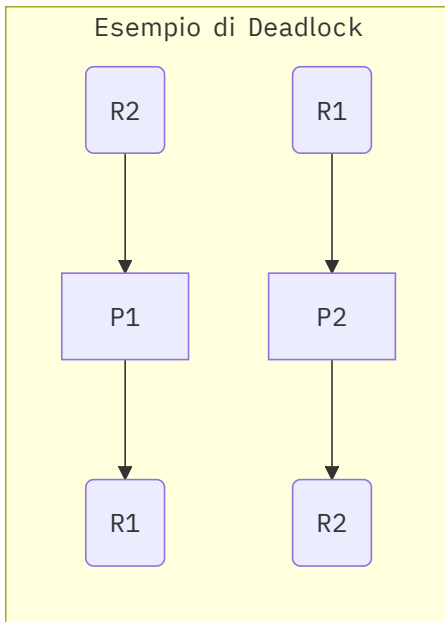
È uno strumento grafico per visualizzare lo stato delle risorse.

- I **Processi** sono cerchi (P).
- Le **Risorse** sono quadrati (R). I puntini dentro indicano le istanze.

- **Arco di Richiesta** ($P \rightarrow R$): Il processo P ha richiesto la risorsa R .
- **Arco di Assegnazione** ($R \rightarrow P$): La risorsa R è stata assegnata al processo P .

🔥 Important

- Se il grafo **non ha cicli**, allora **non c'è deadlock**.
- Se il grafo **ha un ciclo**, allora **potrebbe esserci un deadlock**.
 - Se ogni risorsa nel ciclo ha una sola istanza, allora **c'è sicuramente un deadlock**.
 - Se le risorse nel ciclo hanno più istanze, **potrebbe non esserci un deadlock**.



Nel diagramma sopra, $P1$ possiede $R2$ ma ha bisogno di $R1$. $P2$ possiede $R1$ ma ha bisogno di $R2$. È un'attesa circolare e un deadlock.

Grafo delle Attese (Wait-for Graph)

Hai chiesto un approfondimento sul Grafo delle Attese, ed è uno strumento molto specifico e utile per il rilevamento dei deadlock!

🔗 Il **Grafo delle Attese (Wait-for Graph)** è una semplificazione del grafo di allocazione delle risorse, utilizzata per rilevare i deadlock quando **esiste una singola istanza per ogni tipo di risorsa**.

In questo grafo, i nodi rappresentano **solo i processi**, e un arco diretto $P_i \rightarrow P_j$ indica che il processo P_i è in attesa di una risorsa che è attualmente detenuta dal processo P_j .

Rilevamento Deadlock:

In un grafo delle attese, la presenza di un **ciclo** è una condizione **necessaria e sufficiente** per l'esistenza di un deadlock. L'algoritmo per rilevare un ciclo in un grafo ha una complessità di $O(n^2)$, dove n è il numero di processi nel grafo.

🔗 **Pensa a una catena di persone che si tengono per mano. Se la catena si chiude in un cerchio, nessuno può muoversi. Il grafo delle attese ti mostra proprio questo "cerchio" di dipendenze.**

🧑 Gestire i Deadlock

1. **Prevenzione**: Assicurarsi che almeno una delle 4 condizioni non possa mai verificarsi. Ad esempio, richiedere che un processo richieda tutte le risorse di cui ha bisogno all'inizio, rompendo la condizione di "Hold and Wait". Spesso è troppo restrittivo e poco efficiente.
2. **Evitamento (Avoidance)**: Il sistema operativo alloca le risorse solo se sa che il sistema rimarrà in uno **stato sicuro**. Richiede di conoscere in anticipo le necessità massime di risorse di ogni

processo.

3. **Rilevamento e Ripristino (Detection & Recovery)**: Lasciare che i deadlock si verifichino, rilevarli (es. cercando cicli nel grafo delle risorse) e poi risolverli (es. terminando uno o più processi coinvolti o facendo preemption delle risorse).
4. **Ignorare il problema**: L'approccio più comune in molti sistemi operativi general-purpose. Si assume che i deadlock siano rari e si lascia che sia l'utente o l'amministratore a riavviare il processo bloccato.

Strategie di Recupero dai Deadlock

Una volta rilevato un deadlock, il sistema operativo deve agire per risolverlo! Hai chiesto di approfondire questo aspetto, ed è fondamentale.

Una volta rilevato un deadlock, il sistema operativo deve recuperare da esso. Le strategie principali includono:

1. Terminazione dei Processi:

- **Terminazione di tutti i processi coinvolti**: La soluzione più semplice e drastica. Comporta un'alta perdita di computazione, poiché tutti i processi bloccati (e potenzialmente altri) vengono terminati.
- **Terminazione di un processo alla volta**: Si termina un processo coinvolto nel deadlock e si riesegue l'algoritmo di rilevamento. Il processo viene ripetuto fino alla rimozione del deadlock.
- **Criteri di selezione della vittima**: La scelta di quale processo terminare può basarsi su diversi fattori per minimizzare il costo:
 - Priorità del processo (terminare quello con priorità più bassa).
 - Tempo di esecuzione già speso (terminare quello che ha speso meno tempo).
 - Risorse già utilizzate (terminare quello che ha usato meno risorse).
 - Risorse ancora necessarie (terminare quello che ha bisogno di più risorse).
 - Numero di processi che verrebbero influenzati dalla terminazione.
 - Se è un processo interattivo o batch (preferire la terminazione di processi batch).

2. Prelazione delle Risorse:

- **Selezione della Vittima**: Si identificano le risorse da prelevare (sottrarre temporaneamente a un processo) per rompere il ciclo di attesa. La scelta mira a minimizzare il costo della prelazione (es. risorse trattenute, tempo di computazione impiegato).
- **Rollback**: Se una risorsa viene prelevata da un processo, quel processo deve essere "rollbackato" a uno stato precedente (safe state) e riavviato da quel punto. L'opzione più semplice è ripartire da zero, ma è costosa in termini di lavoro perso. Mantenere punti di checkpoint per il rollback intermedio aggiunge complessità.
- **Starvation**: È possibile che lo stesso processo venga sempre scelto come vittima per la prelazione, impedendogli di completare il suo lavoro. Per evitarlo, il costo di selezione può includere il numero di rollback subito dal processo, dando priorità ai processi che hanno già subito molti rollback.

Pensa a un ingorgo stradale (deadlock).

- **Terminazione di tutti**: Fai esplodere tutte le auto bloccate. Risolve, ma con un costo enorme!
- **Terminazione selettiva**: Chiedi a un'auto di fare retromarcia e uscire dall'incrocio, poi controlla se l'ingorgo è risolto. Se no, chiedi a un'altra.
- **Prelazione risorse**: Togli la benzina a un'auto e la dai a un'altra per farla muovere. L'auto senza benzina deve tornare indietro al distributore (rollback).

L'Algoritmo del Banchiere (Deadlock Avoidance)

Analogia: Il Banchiere Prudente

Un banchiere ha una certa quantità di contanti (risorse disponibili). Ogni cliente (processo) dichiara in anticipo il suo bisogno *massimo* di prestito. Il banchiere concede un prestito solo se, anche dopo averlo concesso, sa di avere ancora abbastanza contanti per soddisfare le richieste massime di

almeno uno degli altri clienti, permettendogli di finire il suo lavoro e restituire tutto. Ripetendo questo ragionamento, il banchiere si assicura di non finire mai in una situazione in cui non può soddisfare nessuno.

L'algoritmo verifica se la concessione di una risorsa porta il sistema in uno **stato sicuro**.

- Uno **stato è sicuro** se esiste una sequenza di esecuzione $\langle P_1, P_2, \dots, P_n \rangle$ tale che per ogni P_i , le risorse di cui P_i ha ancora bisogno possono essere soddisfatte dalle risorse attualmente disponibili + le risorse detenute da tutti i P_j con $j < i$.

Strutture Dati Necessarie:

- **Available**: Vettore delle risorse disponibili.
- **Max**: Matrice delle richieste massime per ogni processo.
- **Allocation**: Matrice delle risorse attualmente allocate a ogni processo.
- **Need**: Matrice del bisogno rimanente ($\text{Need} = \text{Max} - \text{Allocation}$).

Algoritmo di Sicurezza:

1. Inizializza $\text{Work} = \text{Available}$ e $\text{Finish}[i] = \text{false}$ per tutti i processi.
2. Trova un processo P_i tale che $\text{Finish}[i] == \text{false}$ e $\text{Need}[i] \leq \text{Work}$.
3. Se non lo trovi, vai al passo 5.
4. $\text{Work} = \text{Work} + \text{Allocation}[i]$, $\text{Finish}[i] = \text{true}$. Torna al passo 2.
5. Se $\text{Finish}[i] == \text{true}$ per tutti i processi, il sistema è in uno stato sicuro.

VI. Gestione della Memoria

Memoria Principale e CPU: Il Ruolo della Memoria

La memoria principale (RAM) è essenziale per l'esecuzione dei programmi. La CPU accede costantemente a istruzioni e dati memorizzati in essa. Registri e cache sono livelli di memoria più veloci e vicini alla CPU, usati per accelerare l'accesso ai dati più frequentemente usati.

Programmi e Processi

Ciclo di Vita del Programma Utente (Processamento Multistep)

Prima che un programma possa essere eseguito dalla CPU, attraversa diverse fasi:

1. **Pre-processing**: Il pre-processore (es. `cpp` per C/C++) espande macro, include file header e gestisce direttive condizionali.
2. **Compilazione**: Il compilatore (es. `gcc`) traduce il codice sorgente (es. `.c`, `.cpp`) in codice assembly (linguaggio simbolico di basso livello).
3. **Assemblaggio**: L'assembler (es. `as`) traduce il codice assembly in codice macchina riallocabile (file oggetto `.o` o `.obj`). Questo codice contiene riferimenti a indirizzi relativi.
4. **Linking (Collegamento)**: Il linker (es. `ld`) combina uno o più file oggetto con le librerie (statiche o dinamiche) necessarie per creare un singolo file eseguibile.
 - **Linking Statico**: Tutte le librerie necessarie vengono copiate direttamente nell'eseguibile. Il file risultante è più grande ma autonomo.
 - **Linking Dinamico**: Le librerie non vengono copiate nell'eseguibile, ma vengono caricate in memoria solo al momento dell'esecuzione del programma. L'eseguibile contiene solo riferimenti alle librerie.
5. **Caricamento (Loading)**: Il loader del sistema operativo carica il file eseguibile dalla memoria secondaria (disco) alla memoria principale (RAM) e prepara il processo per l'esecuzione.
 - **Caricamento Assoluto**: Il programma viene caricato in una posizione fissa e predefinita.

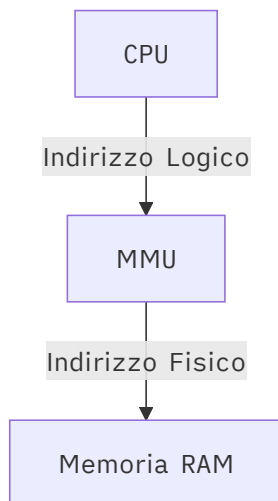
- **Caricamento Rilocabile**: Il programma può essere caricato in qualsiasi posizione disponibile in memoria. Il loader adatta gli indirizzi interni del programma alla posizione di caricamento effettiva.



Indirizzi Logici e Fisici: La Mappa e il Territorio

- **Indirizzo Logico (o Virtuale)**: È l'indirizzo generato dalla CPU. È l'indirizzo visto dal programma. Fa parte dello "spazio di indirizzamento logico".
- **Indirizzo Fisico**: È l'indirizzo reale nella memoria RAM. Fa parte dello "spazio di indirizzamento fisico".

La **Memory-Management Unit (MMU)** è un componente hardware che traduce al volo gli indirizzi logici in indirizzi fisici.



Binding degli Indirizzi

Il processo di associazione degli indirizzi logici agli indirizzi fisici può avvenire in diverse fasi:

- **Compile Time (Tempo di Compilazione)**:
 - **Quando**: Se il compilatore sa *esattamente* dove il programma verrà caricato in memoria (indirizzo fisico assoluto).
 - **Come**: Genera codice macchina con indirizzi fisici assoluti.
 - **Vantaggi**: Semplice, veloce.
 - **Svantaggi**: Poco flessibile. Se la posizione di caricamento cambia, il programma deve essere ricompilato. Non adatto per sistemi multiprogrammati.
- **Load Time (Tempo di Caricamento)**:
 - **Quando**: Se la posizione di caricamento del programma in memoria non è nota al momento della compilazione, ma è nota al momento del caricamento.
 - **Come**: Il linker/loader genera codice riallocabile. Quando il programma viene caricato, il loader modifica gli indirizzi logici relativi in indirizzi fisici assoluti.
 - **Vantaggi**: Più flessibile del compile time.
 - **Svantaggi**: Una volta caricato, il programma non può essere spostato in memoria.
- **Execution Time (Tempo di Esecuzione)**:
 - **Quando**: Se il programma può essere spostato in memoria durante la sua esecuzione (es. swapping, paginazione). Questo è l'approccio più comune nei sistemi operativi moderni.
 - **Come**: La traduzione degli indirizzi (logici a fisici) avviene dinamicamente, istruzione per istruzione, tramite l'hardware **MMU (Memory-Management Unit)**.
 - **Vantaggi**: Massima flessibilità, permette memoria virtuale e swapping.
 - **Svantaggi**: Overhead dovuto alla traduzione hardware.

Caricamento Dinamico (Dynamic Loading)

✍ Il **caricamento dinamico** è una tecnica che permette di caricare le routine di un programma (o moduli) in memoria solo quando sono effettivamente necessarie durante l'esecuzione.

- **Benefici:**
 - **Minore utilizzo di memoria:** Le routine non usate non vengono caricate, risparmiando RAM.
 - **Avvio più rapido:** Solo il codice essenziale viene caricato all'inizio.
 - **Aggiornamenti più facili:** Si possono aggiornare singole routine senza dover ricompilare l'intero programma.
- **Implementazione:** Spesso si usano "stub" (piccole porzioni di codice) che, quando chiamati per la prima volta, caricano la routine reale in memoria e poi la eseguono.

API per il Caricamento Dinamico (es. in Unix/Linux):

- `dlopen()` : Carica una libreria dinamica (shared library) in memoria.
- `dlsym()` : Recupera l'indirizzo di una funzione o variabile all'interno della libreria caricata.
- `dlclose()` : Scarica la libreria dalla memoria.

Linking Dinamico (Dynamic Linking)

✍ Il **linking dinamico** è un processo che ritarda la risoluzione dei riferimenti a librerie esterne fino al momento dell'esecuzione del programma.

- **Shared Libraries (Librerie Condivise):** Le librerie dinamiche (es. `.so` in Linux, `.dll` in Windows) sono caricate in memoria una sola volta e condivise tra più processi che le utilizzano.
- **Benefici:**
 - **Risparmio di spazio su disco:** L'eseguibile è più piccolo perché non include il codice delle librerie.
 - **Risparmio di RAM:** Una singola copia della libreria in memoria è usata da tutti i processi.
 - **Manutenzione più semplice:** Aggiornare una libreria dinamica aggiorna automaticamente tutti i programmi che la usano.
- **Stubs:** Piccole porzioni di codice nell'eseguibile che puntano alle funzioni della libreria dinamica. Al primo accesso, lo stub risolve l'indirizzo reale della funzione nella libreria caricata.

📦 Allocazione Contigua della Memoria

✍ L'**allocazione contigua** della memoria assegna a ogni processo un singolo blocco di memoria fisica contiguo (continuo).

✍ Pensa a un parcheggio dove ogni auto (processo) deve occupare un unico, grande spazio continuo, senza interruzioni.

Problema Principale: Frammentazione Esterna

Questo metodo soffre gravemente di [frammentazione esterna](#). Quando i processi vengono caricati e scaricati, si creano molti piccoli "buchi" di memoria libera. Anche se la memoria totale libera è sufficiente per un nuovo processo, potrebbe non esserci un singolo blocco contiguo abbastanza grande.

Algoritmi di Allocazione (per trovare un blocco libero):

1. First-Fit:

- **Descrizione:** Alloca il **primo blocco libero** che è abbastanza grande da contenere il processo.
- **Vantaggi:** Veloce, semplice.
- **Svantaggi:** Può lasciare molti piccoli "buchi" all'inizio della memoria, aumentando la frammentazione esterna.

2. Best-Fit:

- **Descrizione:** Alloca il **più piccolo blocco libero** che è abbastanza grande da contenere il processo.
- **Vantaggi:** Riduce la frammentazione esterna (lascia blocchi liberi più grandi per usi futuri).

- **Svantaggi:** Lento (deve cercare in tutta la lista dei blocchi liberi). Lascia molti piccoli "buchi" inutilizzabili.

3. Worst-Fit:

- **Descrizione:** Alloca il **più grande blocco libero** disponibile.
- **Vantaggi:** Si spera che i "buchi" rimanenti siano abbastanza grandi da essere utili per processi futuri.
- **Svantaggi:** Lento (deve cercare in tutta la lista). Può rapidamente frammentare i blocchi più grandi.

Compattazione

✎ La **compattazione** è una tecnica usata per risolvere il problema della frammentazione esterna nell'allocazione contigua della memoria. Consiste nello spostare tutti i processi in memoria in modo da raggruppare tutti i blocchi di memoria libera in un unico grande blocco contiguo.

✎ Immagina un parcheggio con auto sparse e molti spazi vuoti piccoli. La **compattazione** è come chiedere a tutte le auto di spostarsi per creare un unico grande spazio libero.

Come funziona:

Il sistema operativo sposta i processi in memoria, riorganizzandoli per eliminare i "buchi" tra di essi.

Costi:

- **Molto Costosa:** Spostare i processi in memoria è un'operazione che richiede tempo e risorse. Tutti i riferimenti agli indirizzi all'interno dei processi spostati devono essere aggiornati, il che richiede un supporto hardware (es. registri di rilocalizzazione) e un overhead significativo.
- **Interruzione del Servizio:** Durante la **compattazione**, il sistema potrebbe essere temporaneamente non disponibile o molto lento.

Per questi motivi, la **compattazione** è raramente usata nei sistemi moderni che preferiscono tecniche non contigue come la **paginazione** o la **segmentazione** paginata, che risolvono la **frammentazione esterna** intrinsecamente.

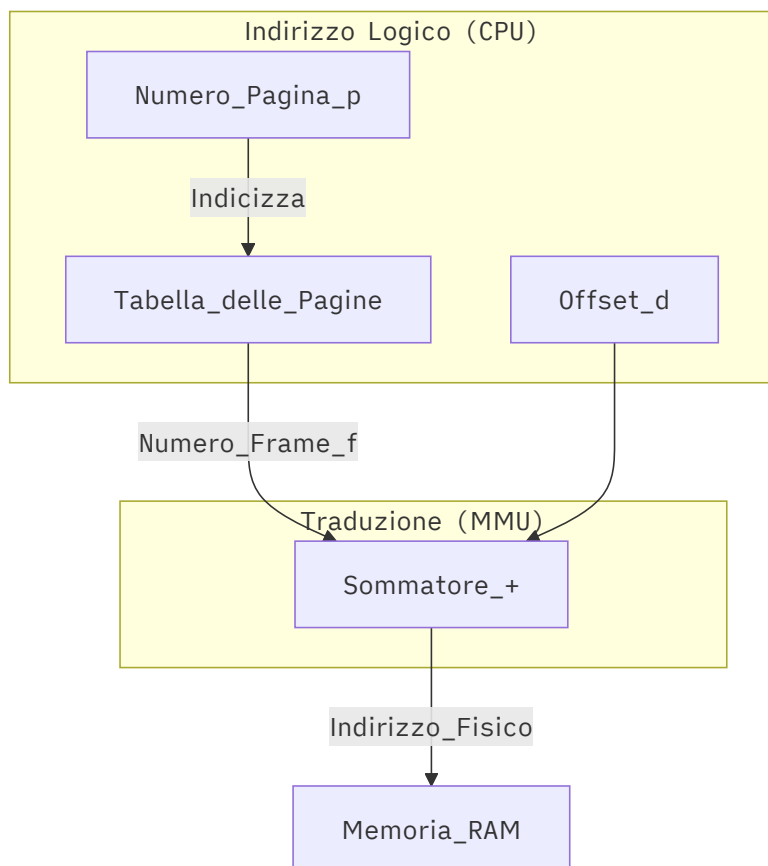
Paginazione: Dividere per Governare

La **paginazione** è la tecnica di gestione della memoria più comune. Risolve il problema della **frammentazione esterna**.

- Lo spazio di indirizzamento **logico** di un processo viene diviso in blocchi di dimensione fissa chiamati **pagine (pages)**.
- La memoria **fisica** (RAM) viene divisa in blocchi della stessa dimensione chiamati **frame**.
- L'SO mantiene una **tabella delle pagine (page table)** per ogni processo, che mappa ogni pagina logica al frame fisico in cui è caricata.

Un indirizzo logico è diviso in due parti:

- **Numero di Pagina (p):** Usato come indice nella tabella delle pagine.
- **Offset (d):** La posizione all'interno della pagina/frame.



🔥 La paginazione soffre di **frammentazione interna**: se un processo ha bisogno di 1025 byte e la dimensione della pagina è 1024 byte, dovrà allocare due pagine. La seconda pagina sarà quasi completamente vuota, sprecando $1024 - 1 = 1023$ byte.

Perché le Pagine di Grandi Dimensione Possono Causare Maggiore Overhead?

Questa è una domanda astuta! Sembrerebbe che pagine più grandi siano meglio, ma non sempre.

- **Maggiore Frammentazione Interna**: Come hai notato, se la dimensione della pagina è grande e un processo usa solo una piccola parte dell'ultima pagina, lo spazio sprecato all'interno di quella pagina aumenta.
- **I/O Spreco durante Page Fault**: Quando si verifica un page fault, l'intera pagina deve essere caricata dal disco alla RAM. Se la pagina è molto grande e il processo ha bisogno solo di pochi byte al suo interno, si carica inutilmente una grande quantità di dati non necessari, sprecando tempo di I/O e occupando preziosa RAM.
- **Impatto sul TLB Hit Ratio**: Questo è un punto cruciale! Se le pagine sono molto grandi, il "working set" di un processo (le pagine che usa attivamente) potrebbe essere molto più piccolo della dimensione di una singola pagina. Questo significa che, anche se la TLB ha un numero limitato di entry, ogni entry della TLB mappa una porzione molto più grande di memoria. Se il working set è piccolo e "sparpagliato" su più pagine grandi, o se un programma non ha una buona località spaziale, la TLB potrebbe non essere usata al suo massimo potenziale. Potresti avere un TLB hit ratio più basso del previsto, perché le pagine grandi non si adattano bene al pattern di accesso del programma, portando a più TLB miss e quindi a un EAT (Effective Access Time) peggiore.

Bit di Validità (Valid-Invalid Bit)

🔗 Il **bit di validità (valid-invalid bit)** è un campo presente in ogni voce della tabella delle pagine. Indica se la pagina corrispondente è attualmente caricata nella memoria fisica (RAM) (**valid = 1**) o se non lo è (**invalid = 0**). Se il bit è **valid** (valido), la traduzione all'indirizzo del frame è corretta. Se è **invalid** (invalido), significa che la pagina non è in memoria (o l'indirizzo non è valido), e un accesso ad essa causerà un page fault.

TLB (Translation Look-aside Buffer)

Accedere alla tabella delle pagine (che è in memoria) per ogni accesso a memoria sarebbe troppo lento (2 accessi a memoria per ogni istruzione!).

✍ La **TLB** è una piccola cache hardware, associativa e molto veloce, che memorizza le traduzioni recenti da pagina a frame.

Flusso con TLB:

1. La CPU genera un indirizzo logico.
2. La MMU controlla prima nella **TLB**.
3. **TLB Hit**: Se la traduzione è presente, il numero di frame viene recuperato istantaneamente. (1 solo accesso a memoria per il dato).
4. **TLB Miss**: Se la traduzione non è presente, la MMU deve accedere alla Page Table in memoria, recuperare il frame, caricarlo nella TLB per usi futuri e poi accedere al dato. (2 accessi a memoria).

🔥 Cosa succede senza TLB?

Senza la TLB, ogni accesso a memoria da parte della CPU richiederebbe **due accessi alla RAM**: uno per consultare la tabella delle pagine e ottenere l'indirizzo fisico del frame, e un secondo per accedere al dato reale. Questo raddoppierebbe il tempo necessario per ogni accesso a memoria, rallentando drasticamente le prestazioni del sistema. La TLB è cruciale per l'efficienza della paginazione.

ASID (Address-Space Identifiers)

✍ Gli **ASID (Address-Space Identifiers)** sono piccoli identificatori numerici che alcune architetture hardware (e le loro TLB) associano a ogni processo.

- **Scopo**: Quando avviene un context switch tra processi, la TLB non deve essere completamente svuotata (flushed). Invece, ogni entry nella TLB viene "taggata" con l'ASID del processo a cui appartiene. Quando la CPU genera un indirizzo logico, la TLB controlla non solo il numero di pagina, ma anche l'ASID corrente.
- **Vantaggi**: Riduce il costo dei context switch, poiché le traduzioni di più processi possono coesistere nella TLB, migliorando il TLB hit ratio.

Più livelli di TLB (L1, L2)

Similmente alle cache della CPU, anche le TLB possono essere organizzate in una gerarchia per ottimizzare le prestazioni:

- **TLB L1**: Più piccola, più veloce, vicina al core della CPU.
- **TLB L2**: Più grande, leggermente più lenta, ma con una maggiore capacità di memorizzare traduzioni.

Quando si verifica un TLB miss nella L1, la richiesta passa alla L2. Se anche lì c'è un miss, allora si accede alla tabella delle pagine in memoria. Questa gerarchia bilancia velocità e capacità.

TLB Reach

Hai chiesto di spiegare il "TLB Reach", una metrica importante per capire l'efficienza della TLB!

✍ Il **TLB Reach** (o "portata della TLB") è una misura della quantità totale di memoria virtuale che può essere mappata simultaneamente dalla Translation Look-aside Buffer (TLB).

Si calcola come:

$$TLB\ Reach = (\text{Numero di entry nella TLB}) \times (\text{Dimensione della Pagina})$$

È un parametro critico per l'efficienza della paginazione con TLB, poiché idealmente l'intero "working set" di un processo dovrebbe risiedere nella TLB per minimizzare i costosi TLB miss.

Come aumentarlo:

Per aumentare il TLB Reach, si possono:

- Aumentare le dimensioni della TLB (limitato dall'hardware).
- Utilizzare **pagine di grandi dimensioni** (huge pages).

🔥 L'uso di pagine più grandi può aumentare la frammentazione interna e lo spreco di I/O durante un page fault se solo una piccola parte della pagina è effettivamente utilizzata. Molti sistemi operativi moderni supportano pagine di dimensioni multiple per affrontare questo compromesso.

Protezione di Memoria con Paging

La paginazione non solo gestisce la memoria, ma la protegge anche. Ogni voce della tabella delle pagine può contenere:

- **Bit di Protezione**: Indicano i permessi di accesso per quella pagina (es. `read-only`, `read-write`, `execute`). Se un processo tenta un'operazione non consentita, si genera un trap.
- **Valid-Invalid Bit**: Già visto, indica se la pagina è in RAM o meno.

Pagine Condivise

La paginazione facilita la condivisione di memoria tra processi:

- **Codice Rientrante (Reentrant Code)**: Codice che può essere eseguito contemporaneamente da più processi senza problemi (non modifica se stesso). Le pagine contenenti codice rientrante (es. librerie di sistema come `libc`) possono essere condivise tra i processi, risparmiando RAM.
- **Memoria Condivisa per IPC**: Le pagine di memoria condivisa usate per la [comunicazione interprocesso](#) vengono semplicemente mappate nelle tabelle delle pagine di più processi.

📁 Tabelle delle Pagine: Varianti Strutturali

Hai colto un punto importantissimo! La paginazione "semplice" con una singola tabella delle pagine funziona bene per spazi di indirizzamento piccoli, ma per i sistemi moderni con indirizzi a 64 bit, una singola tabella sarebbe gigantesca e impraticabile. Ecco come si risolve:

1. Tabelle delle Pagine a Più Livelli (Multi-level Page Tables)

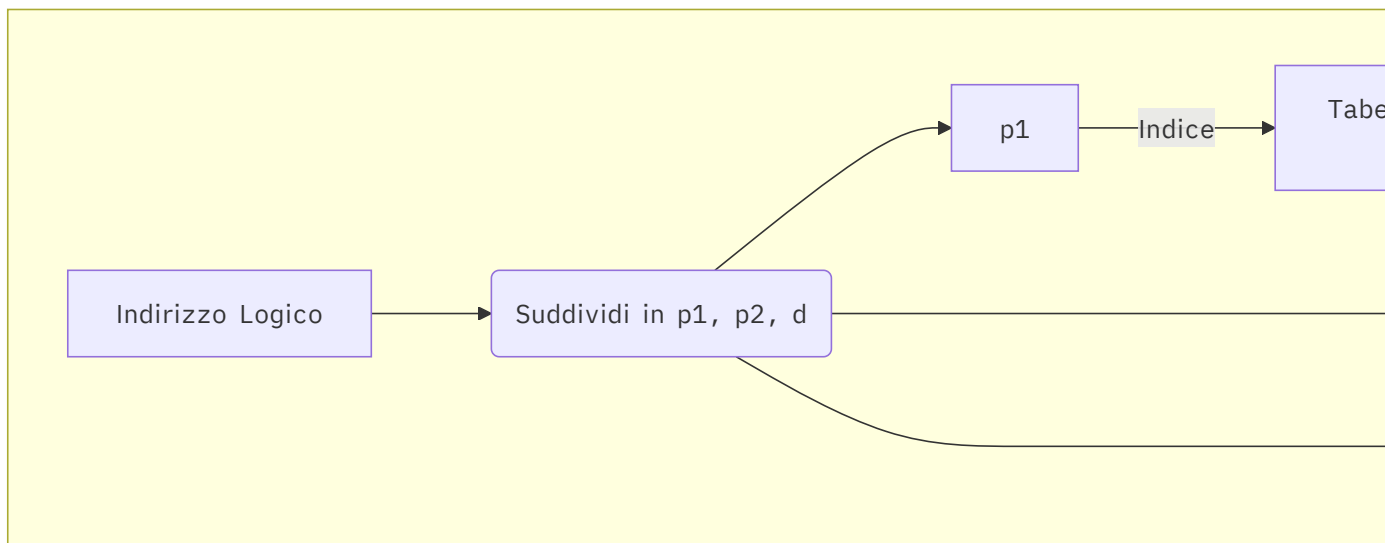
✍ Le **tabelle delle pagine a più livelli** suddividono la tabella delle pagine principale in una gerarchia di tabelle più piccole. Questo permette di gestire spazi di indirizzamento logici molto grandi in modo più efficiente, allocando memoria solo per le parti della tabella delle pagine che sono effettivamente in uso.

✍ Pensa a un enorme elenco telefonico. Invece di avere un unico, gigantesco libro con tutti i nomi in ordine alfabetico, lo dividi per regione, poi per città, poi per cognome. Per trovare un numero, prima trovi la regione, poi la città, poi il cognome.

Come funziona:

Un indirizzo logico viene suddiviso in più parti, ad esempio:

- `p1`: indice per la **tabella delle pagine esterna** (o di primo livello). Questa tabella contiene puntatori ad altre tabelle.
- `p2`: indice per la **tabella delle pagine interna** (o di secondo livello). Questa tabella contiene i puntatori ai frame fisici.
- `d`: offset all'interno del frame.



Vantaggi:

- **Efficienza di Memoria:** Non è necessario allocare una singola tabella delle pagine enorme e contigua. Solo le parti della tabella che corrispondono alle pagine attivamente usate dal processo vengono caricate in memoria.
- **Gestione di Grandi Spazi di Indirizzamento:** Permette di supportare indirizzi logici a 32 o 64 bit in modo pratico.

Svantaggi:

- **Più Accessi a Memoria:** Per tradurre un indirizzo logico, sono necessari più accessi alla memoria (uno per ogni livello della tabella delle pagine) prima di arrivare al frame. La TLB è ancora più cruciale qui!

2. Tabelle delle Pagine Invertite (Inverted Page Tables)

✍ In una **tabella delle pagine invertita**, c'è **un'unica tabella delle pagine per l'intero sistema**, non una per ogni processo. Ogni voce in questa tabella corrisponde a un **frame fisico** della RAM. Ogni voce contiene il PID (Process ID) e il numero di pagina logica del processo che sta attualmente occupando quel frame.

✍ Invece di avere un elenco telefonico per ogni persona (tabella delle pagine per processo), hai un unico elenco per l'intera città, dove ogni riga dice: "Il telefono X è usato da Mario Rossi, numero di pagina Y".

Struttura di una voce: (PID, Numero Pagina Logica)

Calcolo delle Entry:

- La dimensione della tabella delle pagine invertita dipende dalla **quantità di memoria fisica (RAM)** del sistema, non dalla dimensione dello spazio di indirizzamento logico dei processi.
- **Numero di entry = Numero di frame fisici disponibili nella RAM.**
 - Se hai 16 GB di RAM e ogni frame è di 4 KB, avrai
 $16 \text{ GB} / 4 \text{ KB} = 4 \times 1024 \times 1024 \text{ KB} / 4 \text{ KB} = 4 \times 2^{20} = 2^{22}$ frame. Quindi, 2^{22} entry nella tabella invertita.

Vantaggi:

- **Riduzione dell'uso di memoria per le tabelle delle pagine:** La dimensione della tabella è fissa e proporzionale alla RAM, non al numero di processi o alla dimensione del loro spazio logico.

Svantaggi:

- **Ricerca complessa:** Per tradurre un indirizzo logico, il sistema deve cercare nella tabella invertita per trovare la voce che corrisponde al (PID, Numero Pagina Logica) desiderato. Questo richiede una ricerca associativa (spesso con una hash table), che può essere lenta.

- **TLB ancora più cruciale**: Senza una TLB molto efficiente, la traduzione sarebbe lentissima.

3. Hashed Page Tables

✍ Le **Hashed Page Tables** sono un'alternativa alle tabelle a più livelli per gestire spazi di indirizzamento molto grandi (tipicamente a 64 bit). Usano una funzione hash per mappare i numeri di pagina logica in una tabella hash.

Come funziona:

- Ogni voce nella tabella hash contiene una lista concatenata di elementi.
- Ogni elemento nella lista contiene il numero di pagina logica, il numero di frame fisico e un puntatore al prossimo elemento.
- Quando si cerca una pagina, si calcola l'hash del numero di pagina logica e si cerca nella lista corrispondente.

Vantaggi:

- Possono gestire spazi di indirizzamento enormi in modo efficiente, specialmente quando lo spazio logico è sparso.
- Non richiedono tabelle a più livelli, riducendo il numero di accessi a memoria per la traduzione (se non c'è collisione).

Svantaggi:

- Le collisioni nella tabella hash possono rallentare la ricerca.
- La dimensione della tabella hash deve essere scelta con cura.

4. Clustered Page Tables

✍ Le **Clustered Page Tables** sono una variante delle Hashed Page Tables, utilizzate per sistemi a 64 bit. Invece di mappare una singola pagina per voce, una voce nella tabella hash può mappare un "cluster" di pagine contigue.

- **Vantaggi**: Particolarmente efficiente per applicazioni che mostrano una forte località spaziale, riducendo il numero di entry necessarie nella tabella hash e migliorando il TLB hit ratio per blocchi di memoria contigui.
- **Svantaggi**: Più complesse da gestire e possono portare a una maggiore frammentazione interna se i cluster non sono completamente utilizzati.

5. Implementazione SPARC Solaris (Hash Table, TTE, TSB)

Solaris sui sistemi SPARC utilizza un'implementazione di paginazione basata su una **tabella hash** per la gestione delle pagine virtuali.

- **TSB (Translation Storage Buffer)**: È una cache hardware simile alla TLB, ma più grande e gestita in software. Contiene le traduzioni da indirizzi virtuali a fisici.
- **TTE (Translation Table Entry)**: Ogni voce nel TSB è un TTE, che contiene il numero di pagina virtuale, il numero di frame fisico e i bit di protezione.
- **Funzionamento**: Quando la CPU tenta di accedere a una pagina virtuale, prima controlla il TSB. Se c'è un miss, il kernel calcola una funzione hash sul numero di pagina virtuale per trovare la voce corrispondente nella tabella delle pagine in memoria. La traduzione viene poi caricata nel TSB.
- **Vantaggi**: Efficienza per grandi spazi di indirizzamento, flessibilità nella gestione delle traduzioni.

Segmentazione (Segmentation)

Ottimo, hai individuato il "buco" più significativo! La segmentazione è un'altra tecnica fondamentale per la gestione della memoria, con un approccio molto diverso dalla paginazione.

✍ La **segmentazione** è una tecnica di gestione della memoria in cui la memoria logica di un processo viene divisa in blocchi di lunghezza **variabile**, chiamati **segmenti**. Questi

segmenti corrispondono a unità logiche del programma, come il segmento di codice, il segmento dati, lo stack, o le subroutine.

✍ Pensa a una cucina dove ogni "ricetta" (programma) è divisa in sezioni logiche: "Ingredienti" (segmento dati), "Procedura" (segmento codice), "Note temporanee" (segmento stack). Ogni sezione ha una sua dimensione specifica e può essere posizionata in un punto qualsiasi della cucina.

Indirizzo Logico:

In un sistema a segmentazione, un indirizzo logico è composto da due parti:

- **Numero di Segmento (s)**: Identifica il segmento a cui si sta accedendo.
- **Offset (d)**: La posizione all'interno di quel segmento.

Tabella dei Segmenti:

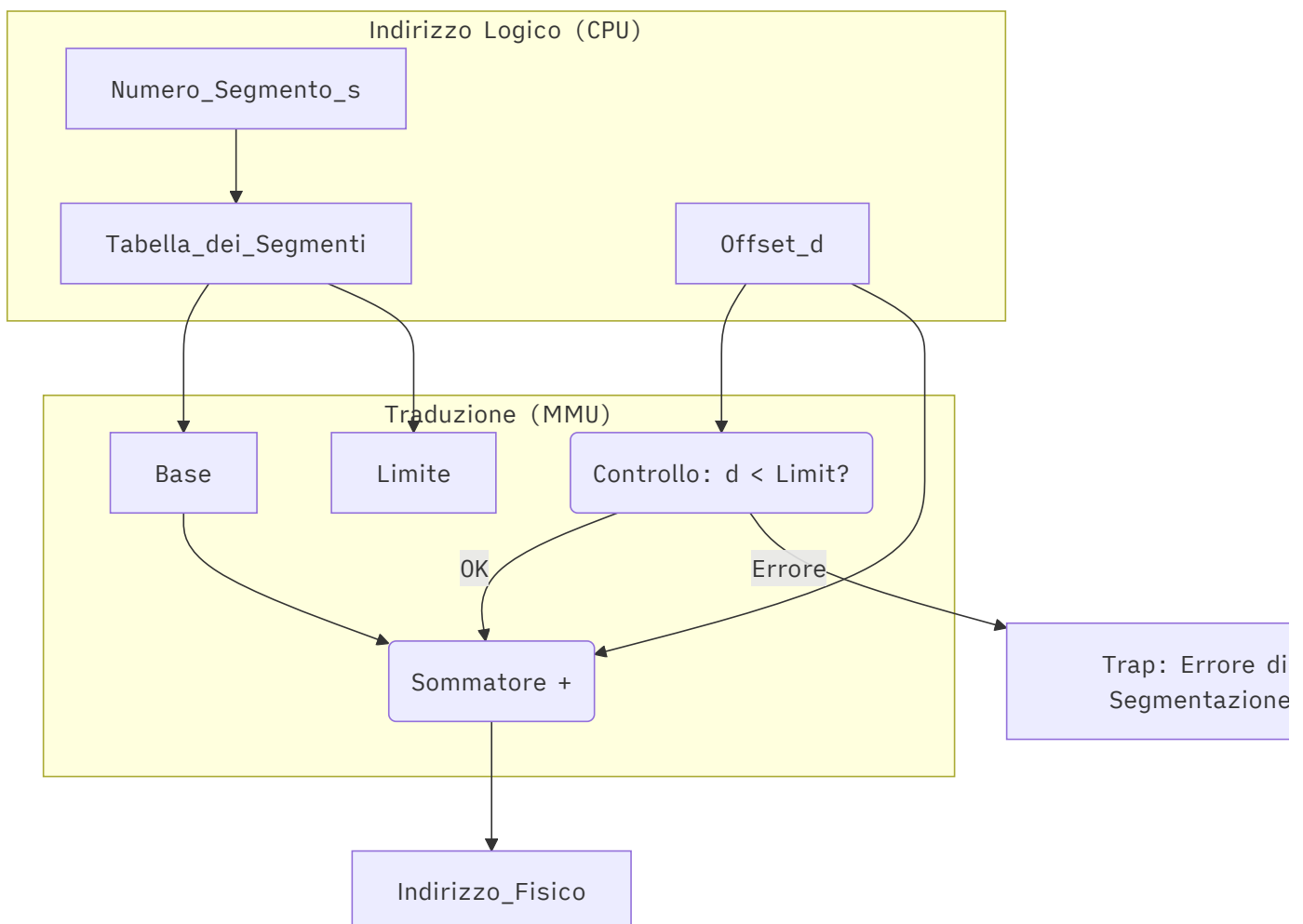
Ogni processo ha una **tabella dei segmenti (Segment Table)**. Ogni voce in questa tabella contiene due informazioni chiave per il segmento corrispondente:

- **Base**: L'indirizzo fisico iniziale in memoria RAM dove il segmento è caricato.
- **Limite (Limit)**: La lunghezza (dimensione) del segmento.

Traduzione dell'Indirizzo:

Quando la CPU genera un indirizzo logico (s, d) :

1. La MMU usa **s** per trovare la voce corrispondente nella tabella dei segmenti.
2. Controlla che l'offset **d** sia **minore del limite** del segmento ($d < \text{limit}$). Se $d \geq \text{limit}$, si verifica un errore di protezione (tentativo di accedere fuori dal segmento).
3. L'indirizzo fisico viene calcolato come: $\text{indirizzo_fisico} = \text{base} + \text{offset}$.



Problema Principale: Frammentazione Esterna

Proprio come l'allocazione contigua, la segmentazione soffre di **frammentazione esterna**. Poiché i

segmenti hanno lunghezze variabili e vengono caricati e scaricati dalla memoria, si creano "buchi" di memoria libera di dimensioni diverse. Anche se la memoria totale libera è sufficiente per un nuovo segmento, potrebbe non esserci un singolo blocco contiguo abbastanza grande da contenerlo. Questo richiede tecniche come la compattazione della memoria, che sono costose.

Caratteristica	Paginazione	Segmentazione	
Divisione Logica	Arbitraria (pagine di dimensione fissa)	Logica (segmenti che corrispondono a parti del programma)	
Divisione Fisica	Frame di dimensione fissa	Blocchi di dimensione variabile	
Frammentazione	Interna	Esterna	
Dimensione Blocchi	Fissa	Variabile	
Complessità	Più semplice per il S0	Più complessa per il S0 (gestione blocchi variabili)	
Visione Utente	Trasparente (l'utente non vede le pagine)	Visibile (l'utente può organizzare il programma in segmenti)	

🔑 La segmentazione è più vicina al modo in cui un programmatore pensa alla memoria (codice, dati, stack). La paginazione è più efficiente per la gestione fisica della memoria da parte del S0. Molti sistemi moderni usano una combinazione di paginazione e segmentazione (es. segmenti paginati).

Segmentazione Paginata

✍ La **segmentazione paginata** è una tecnica che combina i vantaggi della segmentazione e della paginazione. Ogni segmento è a sua volta diviso in pagine.

- **Vantaggi:** Elimina la frammentazione esterna (grazie alla paginazione) e mantiene la visione logica della memoria (grazie alla segmentazione).
- **Esempio storico: MULTICS:** MULTICS (Multiplexed Information and Computing Service) è stato uno dei primi sistemi operativi a implementare la segmentazione paginata, influenzando profondamente i design successivi.

👤 Memoria Virtuale: L'Illusione dello Spazio Infinito

✍ La **memoria virtuale** è una tecnica che permette l'esecuzione di processi che non sono completamente caricati in memoria. Separa la memoria logica vista dall'utente dalla memoria fisica.

Funziona tramite **Demand Paging**: una pagina viene caricata dalla memoria secondaria (disco) alla RAM solo quando è effettivamente necessaria. **Nel caso del "pure demand paging", un processo inizia l'esecuzione senza alcuna pagina in memoria, causando un page fault immediato per la prima istruzione.**

Benefici della Memoria Virtuale

Hai chiesto una sezione esplicita sui benefici della memoria virtuale, ed è un'ottima idea per riassumere perché è così fondamentale!

✍ La memoria virtuale è una delle invenzioni più rivoluzionarie nei sistemi operativi. Non è solo un modo per "avere più RAM", ma offre una serie di vantaggi chiave che migliorano la stabilità, la sicurezza e la flessibilità del sistema:

- **Esecuzione di Processi più Grandi della RAM Fisica:** Permette di eseguire programmi la cui dimensione totale (codice + dati) è maggiore della quantità di RAM disponibile. Solo le parti del programma effettivamente in uso sono caricate in memoria. Questo è il beneficio più intuitivo.
- **Efficiente Utilizzo della RAM (Overcommitment):** Il sistema operativo può allocare più memoria virtuale ai processi di quanta RAM fisica sia effettivamente presente. Questo perché non tutti i processi usano tutta la loro memoria allocata contemporaneamente. L'overcommitment migliora l'utilizzo della RAM.

- **Semplificazione della Creazione dei Processi:** La `fork()` con Copy-on-Write (CoW) è resa possibile dalla memoria virtuale, rendendo la creazione di nuovi processi molto più veloce ed efficiente in termini di memoria.
- **Isolamento e Protezione dei Processi:** Ogni processo ha il suo spazio di indirizzamento virtuale isolato. La MMU e le tabelle delle pagine garantiscono che un processo non possa accedere o corrompere la memoria di un altro processo o del kernel. Questo è cruciale per la stabilità e la sicurezza.
- **Condivisione dello Spazio di Indirizzamento:** La memoria virtuale facilita la condivisione di codice (es. librerie condivise) e dati tra processi, semplicemente mappando le stesse pagine fisiche in diversi spazi di indirizzamento virtuali. Questo risparmia RAM.
- **Riduzione dell'I/O (Paging su Richiesta):** Le pagine vengono caricate dal disco solo quando servono (demand paging), riducendo l'I/O iniziale e il tempo di avvio dei programmi.
- **Facilità di Rilocalizzazione e Swapping:** I processi possono essere caricati in qualsiasi posizione fisica della RAM e spostati (swappati) su disco e viceversa senza dover modificare il codice del programma, grazie alla traduzione degli indirizzi.

Page Fault

Cosa succede se la CPU richiede una pagina che non è in RAM? (Indicato da un **valid-invalid bit** nella page table).

1. La MMU genera un'eccezione (trap) chiamata **page fault**.
2. Il controllo passa al sistema operativo.

Bit di Modifica (Dirty Bit / Modify Bit)

✎ Il **bit di modifica (modify bit o dirty bit)** è un bit associato a ogni pagina in memoria. Viene impostato a 1 dall'hardware ogni volta che la pagina viene modificata (scritta). Il suo scopo è ottimizzare il processo di sostituzione delle pagine: se una pagina 'vittima' ha il bit di modifica a 0, significa che non è stata cambiata e può essere semplicemente scartata. Se è a 1, deve essere prima salvata su disco per non perdere le modifiche.

3. L'SO trova la pagina richiesta sul disco.
4. Trova un frame libero in RAM. Se non ce ne sono, ne sceglie uno da "sacrificare" usando un [algoritmo di sostituzione](#).
5. Se la pagina vittima è stata modificata (il suo **dirty bit** o **modify bit** è 1), deve essere prima salvata su disco.
6. Carica la pagina richiesta dal disco al frame scelto.
7. Aggiorna la tabella delle pagine (imposta il valid bit a 1 e il nuovo numero di frame).
8. Riprende l'esecuzione dell'istruzione che aveva causato il fault.

Instruction Restart Problem

Quando si verifica un page fault, il sistema operativo deve caricare la pagina mancante in memoria e poi **riavviare l'istruzione** che ha causato il fault. Questo processo sembra semplice, ma può essere sorprendentemente complesso, specialmente per istruzioni macchina che accedono a più operandi in memoria o che modificano lo stato della CPU durante la loro esecuzione.

Complessità:

- **Istruzioni con Operandi Multipli:** Se un'istruzione accede a più operandi in memoria e solo uno di essi causa un page fault, il sistema deve assicurarsi che, al riavvio, l'istruzione riparta dall'inizio con tutti gli operandi correttamente disponibili.
- **Istruzioni con Auto-Incremento/Decremento:** Alcune istruzioni modificano i registri (es. incrementando un puntatore) come parte della loro esecuzione. Se un page fault si verifica a metà di queste istruzioni, il sistema deve essere in grado di ripristinare lo stato esatto dei registri prima di riavviare l'istruzione, per evitare risultati errati.
- **Salvataggio dello Stato dei Registri:** Il kernel deve salvare con precisione lo stato di tutti i registri della CPU (Program Counter, Stack Pointer, registri generici, flag di stato) al momento del page

fault, in modo da poterli ripristinare esattamente quando l'istruzione viene riavviata.

🔗 **Pensa a un cuoco che sta preparando un piatto complesso. Se finisce un ingrediente a metà ricetta (page fault), deve mettere via tutto esattamente come stava (salvare lo stato), andare a prendere l'ingrediente, e poi riprendere la ricetta da dove l'aveva lasciata, senza aver rovinato nulla.**

Performance del Demand Paging

Il demand paging introduce un overhead dovuto agli accessi al disco in caso di page fault. L'obiettivo è minimizzare il tasso di page fault per massimizzare le prestazioni.

Swap space

Lo swap space (o area di swap) è una porzione del disco usata come estensione della RAM per le pagine swappate.

Prepaging

Hai chiesto di aggiungere il "prepaging", un'ottimizzazione interessante per il demand paging!

✍ Il **prepaging** è una tecnica che cerca di anticipare le future necessità di pagine di un processo caricandole in memoria **prima** che vengano richieste tramite un page fault. È l'opposto del "pure demand paging".

L'obiettivo è ridurre il numero iniziale di page fault che si verificano all'avvio di un processo o al cambio della sua "località". Ad esempio, il sistema operativo potrebbe pre-caricare tutte le pagine che compongono il "working set" di un processo quando questo viene riattivato o schedulato.

🔗 Il beneficio (page fault evitati) deve bilanciare il costo di caricare pagine non necessarie (spreco di I/O e memoria). Se il prepaging carica troppe pagine che non verranno usate, può peggiorare le prestazioni.

📌 Località di un Processo (Locality of Reference)

✍ La **località di un processo** (o locality of reference) si riferisce alla tendenza di un programma a fare riferimento a un sottoinsieme di dati o istruzioni in memoria in modo concentrato nel tempo. È un principio fondamentale che rende efficienti le cache e la memoria virtuale.

Esistono due tipi principali:

- **Località Temporale**: Se una posizione di memoria è stata acceduta di recente, è probabile che venga acceduta di nuovo in breve tempo.
 - **Esempio**: Variabili in un ciclo, istruzioni di un loop.
- **Località Spaziale**: Se una posizione di memoria è stata acceduta, è probabile che vengano accedute anche posizioni vicine nel prossimo futuro.
 - **Esempio**: Scansione di un array, esecuzione sequenziale di istruzioni.

🔗 La località è ciò che permette al **demand paging** e alle **cache** di funzionare bene. Se un programma non avesse località, ogni accesso sarebbe casuale e la memoria virtuale sarebbe molto inefficiente (thrashing).

🔄 Algoritmi di Sostituzione delle Pagine

Quando la memoria è piena e serve un nuovo frame, quale pagina "vittima" scegliamo di rimpiazzare?

Algoritmo	Descrizione	Pro	Contro
FIFO	Sostituisce la pagina che è in memoria da più tempo.	Semplice.	Può scartare pagine importanti. Soffre dell' Anomalia di Belady .

Algoritmo	Descrizione	Pro	Contro
OPT	Sostituisce la pagina che non sarà usata per più tempo nel futuro.	Migliori performance.	Impossibile da implementare (richiede di predire il futuro).
LRU	Sostituisce la pagina che non è stata usata per più tempo nel passato.	Ottima approssimazione di OPT.	Costoso da implementare perfettamente.
Second Chance	Una versione migliorata di FIFO. Usa un reference bit . Se la pagina più vecchia ha il reference bit a 1, le dà una "seconda chance" (imposta il bit a 0 e la sposta in fondo alla coda) e passa alla successiva.	Buon compromesso tra semplicità e performance.	
Most Frequently Used (MFU)	Sostituisce la pagina con il contatore di riferimenti più alto.	Basato sull'assunzione che le pagine con contatori più bassi siano più recenti e debbano essere ancora usate.	Costoso da implementare (richiede contatori per ogni pagina). Non sempre accurato.

Monitoraggio dei Frame Utilizzati

Gli algoritmi di sostituzione delle pagine "monitorano" l'uso dei frame in modi diversi:

- **LRU (Least Recently Used)**: Richiede un meccanismo per tenere traccia dell'ultimo momento in cui ogni pagina è stata acceduta. Questo può essere fatto con:
 - **Contatori**: Ogni volta che una pagina è acceduta, il suo contatore viene aggiornato. La pagina con il contatore più basso è la vittima. (Molto costoso).
 - **Stack di pagine**: Ogni volta che una pagina è acceduta, viene spostata in cima allo stack. La pagina in fondo è la vittima. (Ancora costoso).
- **Second Chance**: Utilizza un semplice **reference bit** per ogni pagina. Questo bit viene impostato a 1 dall'hardware ogni volta che la pagina viene acceduta. L'algoritmo lo controlla e lo resetta a 0 per dare una "seconda chance". Questo è molto più economico da implementare.
- **Algoritmi basati su frequenza (es. LFU - Least Frequently Used)**: Usano contatori per tenere traccia di quante volte una pagina è stata acceduta.

Compressione di Memoria

Hai chiesto di aggiungere la compressione di memoria, una tecnica moderna che è sempre più usata!

✍ La **compressione di memoria** è una tecnica adottata da sistemi operativi moderni (come Android, iOS, Windows 10 e macOS) per ridurre la pressione sulla memoria fisica, specialmente in situazioni di scarsità di RAM.

Invece di "swappare" pagine di memoria poco usate su disco (operazione lenta), il sistema le **comprime** e le mantiene in RAM in "frame compressi". Quando queste pagine sono necessarie, vengono decomprese rapidamente e ripristinate.

Vantaggi:

- Riduce notevolmente il numero di accessi al disco (I/O), che sono molto lenti.
- Migliora la reattività del sistema in condizioni di memoria bassa.

Svantaggi:

- A costo di un maggiore utilizzo della CPU per le operazioni di compressione/decompressione.
- La velocità di compressione e il rapporto di compressione sono fattori chiave, con implementazioni spesso ottimizzate per sfruttare architetture multi-core.

🧠 Immagina di avere una valigia piena. Invece di togliere le cose e metterle in un armadio lontano (swap su disco), le pieghi meglio e le comprimi sottovuoto per farle stare ancora nella valigia (compressione in RAM). Ci vuole un po' di fatica (CPU), ma è molto più veloce che andare all'armadio!

Page-Buffering

✍ Il **Page-Buffering** è una tecnica usata per migliorare le prestazioni degli algoritmi di sostituzione delle pagine, specialmente quando la memoria è sotto pressione.

- **Pool di Frame Liberi**: Invece di attendere che un page fault si verifichi e poi cercare un frame libero, il sistema operativo mantiene un piccolo pool di frame già liberi. Quando un page fault si verifica, una pagina può essere caricata immediatamente in uno di questi frame, riducendo il tempo di attesa.
- **Lista di Pagine Modificate (Dirty Pages)**: Le pagine che sono state scelte come vittime e che hanno il dirty bit impostato a 1 (cioè sono state modificate) non vengono immediatamente scritte su disco e liberate. Invece, vengono aggiunte a una lista di "pagine modificate" (dirty pages). Il sistema operativo può poi scriverle su disco in background in un momento più opportuno, o raggruppare più scritture per migliorare l'efficienza I/O. Se una di queste pagine viene richiesta di nuovo prima di essere scritta su disco, può essere recuperata rapidamente.
- **Vantaggi**: Riduce il tempo di risposta ai page fault, migliora l'efficienza I/O raggruppando le scritture su disco.

📁 Allocazione dei Frame

Una volta che abbiamo deciso quale pagina rimpiazzare, dobbiamo anche decidere quanti frame allocare a ciascun processo e come gestire la sostituzione delle pagine.

- **Allocazione Fissa**:
 - **Descrizione**: Ogni processo riceve un numero fisso di frame di memoria.
 - **Tipi**:
 - **Equal Allocation**: Tutti i processi ricevono lo stesso numero di frame.
 - **Proportional Allocation**: I frame sono allocati in proporzione alla dimensione del processo.
 - **Vantaggi**: Semplice da implementare.
 - **Svantaggi**: Non si adatta alle esigenze dinamiche dei processi (un processo potrebbe aver bisogno di più frame in certi momenti).
- **Allocazione Variabile (o Dinamica)**:
 - **Descrizione**: Il numero di frame allocati a un processo può variare durante la sua esecuzione.
 - **Vantaggi**: Più flessibile, si adatta meglio alle esigenze reali dei processi.
 - **Svantaggi**: Più complessa da gestire per il sistema operativo.

Minimo Numero di Frame

✍ Esempio Specifico di Requisiti Frame per Istruzioni (es. IBM 370)

Ottima aggiunta! Questo è un dettaglio importante che mostra come l'architettura hardware influenzi la gestione della memoria.

Il sistema operativo deve sempre garantire un **minimo numero di frame** allocati a un processo. Questo numero minimo è determinato dall'architettura del set di istruzioni (ISA) e dalla sua capacità di accedere a più pagine contemporaneamente. Se un processo non ha abbastanza frame per eseguire una singola istruzione, si verificherebbero page fault continui o l'istruzione non potrebbe essere completata.

Esempio: Alcune architetture, come l'**IBM 370**, avevano istruzioni complesse (es. **SS MOVE** per spostare stringhe di lunghezza arbitraria) che potevano richiedere l'accesso a più pagine contemporaneamente (ad esempio, fino a 6 pagine). In questi casi, il sistema operativo deve assicurarsi che un numero minimo di frame sia allocato per evitare page fault multipli o il fallimento dell'istruzione. Questo requisito minimo è cruciale per la corretta esecuzione del programma.

Sostituzione delle Pagine: Globale vs Locale

Quando un processo subisce un page fault e non ci sono frame liberi, il sistema deve scegliere una pagina da rimpiazzare.

- **Sostituzione Globale:**
 - **Descrizione:** Il processo che ha subito il page fault può scegliere una pagina vittima da **qualsiasi frame in memoria**, indipendentemente dal processo a cui appartiene.
 - **Vantaggi:** Maggiore flessibilità, può portare a un utilizzo più efficiente della memoria complessiva.
 - **Svantaggi:** Un processo può "rubare" frame a un altro processo, influenzando le sue prestazioni.
- **Sostituzione Locale:**
 - **Descrizione:** Il processo che ha subito il page fault può scegliere una pagina vittima **solo tra i frame che gli sono stati allocati**.
 - **Vantaggi:** Le prestazioni di un processo non sono influenzate da quelle di altri processi.
 - **Svantaggi:** Meno flessibile, può portare a un utilizzo meno efficiente della memoria complessiva (un processo potrebbe avere frame inutilizzati mentre un altro ne ha disperatamente bisogno).

I/O Interlock (Pinning)

✍ **L'I/O Interlock (o "pinning") è un meccanismo che "blocca" (pin) una pagina in memoria fisica, impedendo che venga selezionata come vittima da un algoritmo di sostituzione delle pagine.**

- **Scopo:** È cruciale per le pagine coinvolte in operazioni di I/O. Quando un dispositivo I/O (es. un controller DMA) sta trasferendo dati direttamente da o verso una pagina in memoria, quella pagina non deve essere spostata o rimossa dalla RAM. Se lo fosse, il trasferimento DMA fallirebbe o corromperebbe i dati.
- **Funzionamento:** Il kernel imposta un bit speciale (es. "locked bit" o "pinned bit") nella voce della tabella delle pagine della pagina interessata. Gli algoritmi di sostituzione delle pagine ignorano le pagine con questo bit impostato. Una volta completata l'operazione di I/O, il bit viene resettato e la pagina può essere nuovamente candidata per la sostituzione.

Strategie di Paging in SO Specifici

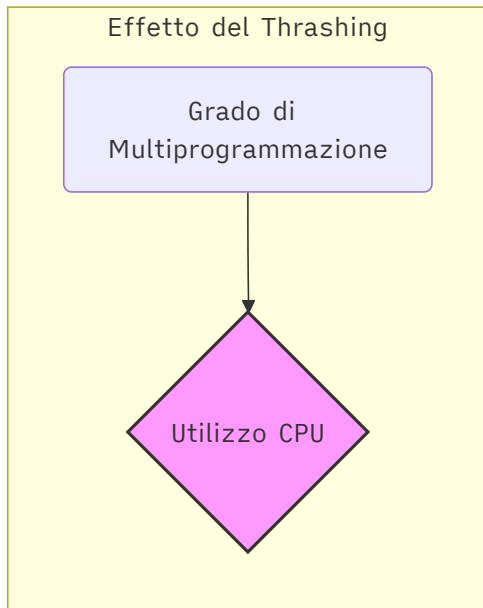
I sistemi operativi reali implementano algoritmi di sostituzione delle pagine e strategie di gestione della memoria virtuale che sono spesso ibride e ottimizzate per le loro architetture e carichi di lavoro.

- **Windows:**
 - Utilizza un algoritmo che è una variante dell'LRU, ma con un focus sulla reattività.
 - Mantiene liste di pagine libere, modificate e standby.
 - Le pagine in standby sono pagine rimosse dal working set di un processo ma ancora in RAM, pronte per essere riutilizzate rapidamente se richieste di nuovo (soft fault).
 - Il "Working Set Manager" di Windows regola dinamicamente la dimensione del working set di ogni processo.
- **Solaris:**
 - Utilizza un algoritmo di clock (una variante dell'Enhanced Second Chance) per la sostituzione delle pagine.
 - **Lotsfree, Desfree, Minfree:** Sono soglie di pagine libere.
 - **minfree:** Se il numero di pagine libere scende sotto questa soglia, il "pageout daemon" (un processo in background) inizia a rubare pagine.
 - **desfree:** Se il numero di pagine libere scende sotto questa soglia, il pageout daemon diventa più aggressivo.
 - **lotsfree:** Se il numero di pagine libere è sopra questa soglia, il pageout daemon è inattivo.
 - **Scanrate:** La velocità con cui il pageout daemon scansiona le pagine per trovare vittime. Aumenta quando la memoria è sotto pressione.
 - **Pageout Daemon:** Un processo del kernel che periodicamente esamina le pagine in memoria per trovare quelle da sostituire e scriverle su disco.

🔥 Thrashing: Quando il Sistema è in Panico

✍ Il **thrashing** è una condizione di degrado estremo delle prestazioni in un sistema con memoria virtuale, causata da un numero insufficiente di frame allocati ai processi, portando a un eccessivo numero di page fault e di operazioni di swap. Il sistema spende la maggior parte del tempo a scambiare pagine tra RAM e disco (paging) piuttosto che eseguire il lavoro utile della CPU.

Succede quando a un processo non vengono allocati abbastanza frame per contenere il suo **working set** (l'insieme di pagine attivamente usate). L'utilizzo della CPU crolla perché è sempre in attesa del disco.



All'aumentare della multiprogrammazione, l'utilizzo della CPU sale, fino a un punto in cui la memoria si satura. A quel punto, inizia il thrashing e l'utilizzo della CPU crolla drasticamente.

🔧 Working Set: Calcolo Pratico

Ottimo, questo è un esercizio classico che ti aiuta a capire il concetto!

✍ Il **working set** di un processo all'istante t con una finestra Δ (delta), denotato come $WS(t, \Delta)$, è l'insieme delle pagine uniche a cui il processo ha fatto riferimento negli ultimi Δ accessi alla memoria, ovvero nell'intervallo di tempo $[t - \Delta + 1, t]$.

In pratica, il working set è un "fotografia" delle pagine che il processo sta usando attivamente in un dato momento.

Come calcolarlo:

Per calcolare $WS(t, \Delta)$, devi semplicemente guardare la stringa di riferimenti alle pagine e identificare le pagine uniche che compaiono negli ultimi Δ riferimenti (partendo dall'istante t e andando a retro per Δ posizioni).

Esempio Pratico:

Supponiamo di avere la seguente stringa di riferimenti alle pagine:

... 2, 3, 4, 2, 1, 5, 6, 2, 3, 7, 4, 1, 5, 2, 6, 3, 7, 1, 2, 3, 8, 9, 10, 1, 2, 3

Calcoliamo il working set per diversi istanti t e finestre Δ .

Caso 1: $WS(t = 10, \Delta = 4)$

- Stringa fino a $t = 10$: ... 2, 3, 4, 2, 1, 5, 6, 2, 3, 7
- Ultimi $\Delta = 4$ riferimenti (da $t = 10$ a $t = 7$): 7, 3, 2, 6
- Pagine uniche in questo intervallo: {2, 3, 6, 7}
- Quindi, $WS(10, 4) = \{2, 3, 6, 7\}$

Caso 2: $WS(t = 15, \Delta = 5)$

- Stringa fino a $t = 15$: ... 2, 3, 4, 2, 1, 5, 6, 2, 3, 7, 4, 1, 5, 2, 6
- Ultimi $\Delta = 5$ riferimenti (da $t = 15$ a $t = 11$): 6, 2, 5, 1, 4
- Pagine uniche in questo intervallo: $\{1, 2, 4, 5, 6\}$
- Quindi, $WS(15, 5) = \{1, 2, 4, 5, 6\}$

Punti chiave per te:

- **Discalculia**: Non ti preoccupare dei calcoli complessi. Qui si tratta di **identificare gli elementi unici in una sequenza**. È più una questione di logica e attenzione ai dettagli che di calcolo numerico.
- **ADHD**: La visualizzazione della "finestra" Δ che scorre sulla stringa può aiutarti a mantenere la concentrazione. Immagina una piccola finestra che si muove, e tu devi solo guardare dentro quella finestra.

Allocazione Memoria per il Kernel

Allocazione Memoria per il Kernel

La gestione della memoria all'interno del kernel è diversa dalla gestione della memoria utente per diverse ragioni:

- Le richieste di memoria del kernel sono spesso di dimensioni molto varie (da pochi byte a intere pagine).
- Il kernel ha bisogno di blocchi di memoria contigui per alcune operazioni hardware (es. DMA).
- Il kernel non può subire page fault quando accede alla propria memoria.

Per queste ragioni, il kernel utilizza allocatori di memoria specializzati:

- **Buddy System**:
 - **Descrizione**: Un algoritmo di allocazione di memoria che divide ricorsivamente blocchi di memoria in due "buddy" (blocchi di pari dimensione) fino a trovare un blocco della dimensione richiesta. Quando un blocco viene liberato, cerca il suo buddy. Se entrambi sono liberi, vengono uniti per formare un blocco più grande.
 - **Vantaggi**: Riduce la frammentazione esterna e interna (rispetto a un allocatore generico), efficiente per allocazioni di dimensioni potenze di 2.
 - **Contesto x86**: Il Buddy System è spesso usato nel kernel Linux (anche su architetture x86) per allocare pagine intere o multipli di pagine, garantendo la contiguità fisica.
- **Slab Allocator**:
 - **Descrizione**: Un allocatore di memoria ottimizzato per oggetti di dimensioni fisse e di uso frequente all'interno del kernel (es. strutture `task_struct`, `inode`, `dentry`). Mantiene cache di oggetti già inizializzati.
 - **Componenti**:
 - **Slabs**: Blocchi di memoria contigua (spesso più pagine) che contengono oggetti dello stesso tipo.
 - **Caches**: Collezioni di slabs per un tipo specifico di oggetto.
 - **Vantaggi**: Riduce la frammentazione interna, migliora le prestazioni di allocazione/deallocazione (gli oggetti sono già inizializzati), riduce l'overhead della CPU.
 - **Contesto x86**: Il Slab Allocator (o le sue varianti come SLUB o SLAB) è ampiamente utilizzato nel kernel Linux (e quindi su x86) per gestire la memoria per le proprie strutture dati interne.

Paginazione e Segmentazione in Architetture Specifiche (Intel IA-32/IA-64, ARMv8)

Le architetture hardware moderne implementano la paginazione e, in alcuni casi, la segmentazione, con dettagli specifici:

- **Intel IA-32 (x86 a 32 bit)**:
 - Supporta sia la **segmentazione** che la **paginazione**.

- La segmentazione è sempre attiva, ma spesso configurata per fornire uno spazio di indirizzamento "piatto" (flat model) dove tutti i segmenti hanno base 0 e limite massimo, rendendo la paginazione il meccanismo primario di gestione della memoria.
- La paginazione è a due livelli (directory delle pagine e tabella delle pagine).
- **PAE (Physical Address Extension)**: Permette ai sistemi a 32 bit di indirizzare più di 4 GB di RAM, utilizzando tabelle delle pagine a 3 livelli.
- **Intel IA-64 (x86-64 a 64 bit)**:
 - La segmentazione è ancora presente ma usata raramente, con un forte focus sulla **paginazione** come meccanismo primario.
 - La paginazione è a quattro o cinque livelli, per gestire spazi di indirizzamento virtuali enormi (fino a 256 TB o 4 PB).
 - Supporta pagine di dimensioni diverse (4 KB, 2 MB, 1 GB) per ottimizzare il TLB Reach.
- **ARMv8 (Architettura ARM a 64 bit)**:
 - Si basa esclusivamente sulla **paginazione** per la gestione della memoria virtuale. Non supporta la segmentazione nel modo tradizionale di x86.
 - La paginazione è a quattro livelli (con la possibilità di configurare meno livelli per spazi di indirizzamento più piccoli).
 - Supporta pagine di dimensioni diverse (4 KB, 16 KB, 64 KB, 2 MB, 512 MB, 1 GB) per flessibilità e ottimizzazione delle prestazioni.

VII. Gestione dell'Archiviazione e I/O

Hardware I/O

I dispositivi di I/O sono le periferiche che permettono al computer di interagire con il mondo esterno (tastiera, mouse, stampante, disco, rete).

- **Polling**:
 - **Descrizione**: La CPU controlla ripetutamente lo stato di un dispositivo I/O per vedere se è pronto o se ha dati da trasferire.
 - **Vantaggi**: Semplice da implementare.
 - **Svantaggi**: Inefficiente. Spreca cicli di CPU in attesa, specialmente se i dispositivi sono lenti.
- **Interrupts**:
 - **Descrizione**: Il dispositivo I/O invia un segnale di interrupt alla CPU quando ha completato un'operazione o richiede attenzione.
 - **Vantaggi**: Efficiente. La CPU può eseguire altro lavoro mentre attende l'I/O.
 - **Svantaggi**: Overhead per la gestione degli interrupt (salvataggio/ripristino del contesto).
- **DMA (Direct Memory Access)**:
 - **Descrizione**: Un controller hardware specializzato (DMA controller) gestisce il trasferimento di dati direttamente tra i dispositivi I/O e la memoria principale, senza coinvolgere la CPU. La CPU viene interrotta solo all'inizio e alla fine del trasferimento.
 - **Vantaggi**: Essenziale per trasferimenti di dati ad alta velocità (es. disco, rete). Libera la CPU per altri compiti.
 - **Svantaggi**: Più complesso da implementare.

Sottosistema I/O del Kernel

Il kernel del sistema operativo fornisce un'interfaccia uniforme per i dispositivi I/O e offre vari servizi per gestirli in modo efficiente.

- **Scheduling dell'I/O**: Ordina le richieste di I/O per ottimizzare le prestazioni (es. [scheduling del disco](#)).
- **Buffering**:

- **Descrizione:** Memorizzazione temporanea dei dati durante il trasferimento tra un dispositivo e la memoria, o tra due dispositivi.
- **Scopo:** Compensare le differenze di velocità tra produttore e consumatore, o tra dispositivi con diverse dimensioni di blocco.
- **Caching:**
 - **Descrizione:** Memorizzazione di copie di dati in una memoria più veloce (cache) per un accesso futuro più rapido.
 - **Scopo:** Migliorare le prestazioni riducendo il numero di accessi a dispositivi lenti.
- **Spooling:**
 - **Descrizione:** (Simultaneous Peripheral Operations On-Line) Tecnica che utilizza un buffer (spool) sul disco per gestire l'output di dispositivi lenti (es. stampanti). I dati vengono inviati rapidamente allo spool e poi stampati in background.
 - **Scopo:** Permettere ai programmi di continuare l'esecuzione senza attendere il completamento di operazioni I/O lente.
- **Gestione degli Errori:** Rilevamento e gestione degli errori dei dispositivi I/O.
- **Protezione e Sicurezza:** Controllo degli accessi ai dispositivi I/O.

Device Driver (Struttura e Interfacce)

Hai chiesto di aggiungere una sezione sui device driver, ed è una parte cruciale del sottosistema I/O!

✍ I **device driver** sono software specializzati (spesso implementati come moduli del kernel) che fungono da intermediari tra il kernel del sistema operativo e l'hardware dei dispositivi I/O. Il loro ruolo è incapsulare le specifiche e le complessità di ciascun dispositivo, offrendo al sottosistema I/O del kernel un'interfaccia uniforme e standardizzata.

Interazione:

- I driver non eseguono system call utente direttamente, ma invocano funzioni del kernel per operazioni come l'allocazione di memoria per i buffer, la gestione della [MMU](#), del [DMA](#) e degli [interrupt](#).
- Sono il "traduttore" tra il linguaggio generico del kernel e il linguaggio specifico del dispositivo.

Interfacce Standard:

Ogni tipo di dispositivo (es. block device per i dischi, character stream device per tastiere/stampanti) ha un set predefinito di operazioni che il suo driver deve implementare (es. `read()`, `write()`, `open()`, `close()`, `ioctl()`). Queste funzioni sono spesso esposte tramite una "tabella di puntatori a funzioni" (come la `struct file_operations` in Linux) che il kernel utilizza per interagire con il driver.

Benefici:

- Questa architettura a strati semplifica lo sviluppo del sistema operativo.
- Permette ai produttori di hardware di sviluppare driver specifici per diversi sistemi operativi, senza che il kernel debba conoscere i dettagli di ogni singolo dispositivo.

Raw I/O e Dispositivi Raw

Hai chiesto di spiegare il "Raw I/O", un concetto importante per le applicazioni che richiedono massime prestazioni I/O!

✍ Il **Raw I/O** (o accesso a "raw devices") è un metodo che permette a specifiche applicazioni (come database o sistemi di streaming video/audio) di accedere direttamente ai blocchi fisici di un dispositivo di archiviazione, bypassando i livelli superiori del file system del kernel (come buffering, caching, gestione dei nomi di file e delle directory, file locking).

Vantaggi:

- Può migliorare significativamente le prestazioni di I/O, riducendo l'overhead del kernel.

- Consente all'applicazione di gestire il proprio buffering in modo più ottimizzato, evitando il problema del "double buffering" (dove i dati vengono copiati inutilmente più volte tra i buffer del kernel e quelli dell'applicazione).

Svantaggi:

- Comporta una maggiore complessità per il programmatore, che deve assumersi la responsabilità della coerenza e della sincronizzazione dei dati.
- Meno sicuro, in quanto bypassa i meccanismi di protezione standard del file system.

🔗 Immagina di voler prendere un ingrediente dal magazzino del ristorante (disco). Normalmente, chiedi al manager (file system) che lo prende per te. Con il Raw I/O, vai direttamente tu al magazzino, prendi l'ingrediente e lo porti in cucina. È più veloce, ma devi sapere esattamente dove si trova e come gestirlo!

Overview Memoria di Massa

- **HDD (Hard Disk Drives)** e **NVM (Non-Volatile Memory/SSD)**: Già introdotti brevemente nella Sezione I, sono i principali dispositivi di archiviazione persistente.
- **Magnetic Tapes (Bobine, accesso sequenziale, backup)**: Utilizzati principalmente per backup a lungo termine e archiviazione di massa, offrono un costo per GB molto basso ma con accesso sequenziale molto lento.

RAM Disk

✍ Un **RAM disk** (o RAM drive) è un blocco di memoria volatile (RAM) che viene configurato e utilizzato dal sistema operativo come se fosse un dispositivo di archiviazione a blocchi (come un disco rigido o un SSD).

- **Vantaggi:**
 - **Velocità Estrema:** L'accesso alla RAM è ordini di grandezza più veloce dell'accesso a qualsiasi disco fisico. Questo lo rende ideale per applicazioni che richiedono I/O intensivo e a bassa latenza (es. cache temporanee, compilazione di software).
 - **Nessuna usura:** Non essendoci parti mobili o cicli di scrittura/cancellazione come nelle flash memory, non soffre di usura.
- **Svantaggi:**
 - **Volatile:** I dati memorizzati su un RAM disk vengono persi all'interruzione dell'alimentazione (spegnimento, riavvio).
 - **Costo:** La RAM è più costosa per gigabyte rispetto al disco.
 - **Capacità Limitata:** La dimensione del RAM disk è limitata dalla quantità di RAM disponibile nel sistema.

Struttura e Performance HDD

- **Piatti, tracce, settori, cilindri, testine:** Concetti base dell'organizzazione fisica di un HDD.

Performance: Calcolo del Tempo di I/O

Il tempo totale per accedere a un blocco di dati su un HDD è la somma di diversi componenti:

$$\text{Tempo di I/O} = \text{Tempo di Seek} + \text{Latenza Rotazionale} + \text{Tempo di Trasferimento} + \text{Overhead del Controllore}$$

1. Tempo di Seek (Seek Time):

- **Descrizione:** Il tempo necessario per muovere le testine di lettura/scrittura sul cilindro corretto (la traccia giusta). È il componente più significativo e più lento.
- **Fattori:** Dipende dalla distanza che la testina deve percorrere.
- **Ottimizzazione:** Gli [algoritmi di scheduling del disco](#) mirano a minimizzare questo tempo.

2. Latenza Rotazionale (Rotational Latency):

- **Descrizione:** Il tempo necessario affinché il settore desiderato ruoti sotto la testina di lettura/scrittura. In media, è metà del tempo di un giro completo del piatto.

- **Fattori:** Dipende dalla velocità di rotazione del piatto (RPM - Revolutions Per Minute).
 - **Calcolo:** Per un disco a 7200 RPM, un giro completo impiega $60 \text{ secondi} / 7200 \text{ giri} = 0.00833 \text{ secondi} = 8.33 \text{ ms}$. La latenza media è $8.33 \text{ ms} / 2 = 4.16 \text{ ms}$.
3. **Tempo di Trasferimento (Transfer Time):**
- **Descrizione:** Il tempo necessario per leggere o scrivere i dati una volta che la testina è sul cilindro e il settore è sotto la testina.
 - **Fattori:** Dipende dalla densità dei dati sulla traccia e dalla velocità di rotazione.
4. **Overhead del Controllore (Controller Overhead):**
- **Descrizione:** Il tempo impiegato dal controller del disco per elaborare la richiesta, gestire i buffer e comunicare con il sistema.

🔥 Per migliorare le prestazioni degli HDD, si cerca di ridurre il seek time (con scheduling del disco) e la latenza rotazionale (con dischi a RPM più elevati).

Dispositivi NVM/SSD

✍ I dispositivi NVM (Non-Volatile Memory), come gli SSD (Solid State Drive), utilizzano memorie flash (tipicamente NAND flash) per archiviare i dati. A differenza degli HDD, non hanno parti mobili, il che li rende molto più veloci, silenziosi e resistenti agli urti.

Tecnologia NAND Flash e Funzionamento Interno

- **Celle di Memoria:** Le NAND flash memorizzano i dati in celle, che possono essere SLC (Single-Level Cell, 1 bit per cella), MLC (Multi-Level Cell, 2 bit), TLC (Triple-Level Cell, 3 bit) o QLC (Quad-Level Cell, 4 bit). Più bit per cella significa maggiore densità e costo inferiore, ma anche minore velocità e durata.
- **Pagine e Blocchi:** I dati sono letti e scritti in unità chiamate **pagine** (tipicamente 4 KB o 8 KB). Tuttavia, le operazioni di **cancellazione** possono avvenire solo su unità più grandi chiamate **blocchi** (tipicamente 256 KB a 4 MB), che contengono molte pagine.
- **Non Sovrascrittura Diretta:** A differenza degli HDD, non è possibile sovrascrivere direttamente una pagina esistente. Per modificare una pagina, l'intero blocco a cui appartiene deve essere letto, cancellato, e poi riscritto con le pagine modificate e quelle non modificate. Questa è un'operazione costosa.

Problematiche e Soluzioni

1. **Wear Leveling (Livellamento dell'Usura):**
 - **Problema:** Le celle flash hanno un numero limitato di cicli di programmazione/cancellazione (P/E cycles) prima di usurarsi. Se alcune celle fossero usate più di altre, si usurerebbero prematuramente.
 - **Soluzione:** Il controller SSD (tramite il FTL) distribuisce le scritture in modo uniforme su tutte le celle flash disponibili, per massimizzare la vita utile del drive.
2. **Flash Translation Layer (FTL):**
 - **Descrizione:** Un software/firmware all'interno del controller SSD che funge da strato di astrazione tra il sistema operativo e la memoria flash fisica.
 - **Funzioni:**
 - **Mapping degli indirizzi logici a fisici:** Traduce gli indirizzi logici dei blocchi (LBA) inviati dal SO in indirizzi fisici sulle celle flash, gestendo il wear leveling.
 - **Garbage Collection:** Raccoglie le pagine obsolete e le cancella a livello di blocco per liberare spazio.
 - **Bad Block Management:** Identifica e marca i blocchi danneggiati, reindirizzando le scritture a blocchi di ricambio.
 - **Vantaggi:** Permette agli SSD di presentarsi al sistema operativo come normali dischi a blocchi, semplificando la loro integrazione.
3. **Bad Block Management:**
 - **Problema:** Le memorie flash possono nascere con blocchi difettosi o svilupparne nel tempo.

- **Soluzione:** Il controller SSD mantiene una lista di bad block e li esclude dall'uso, reindirizzando i dati a blocchi di ricambio.

Mapping degli Indirizzi (Disco)

✍ I dischi sono organizzati in **blocchi logici**, che sono la più piccola unità di trasferimento di dati. Il sistema operativo vede il disco come una sequenza lineare di questi blocchi.

LBA (Logical Block Addressing) vs CHS (Cylinder Head Sector)

- **CHS (Cylinder Head Sector):**
 - **Descrizione:** Un metodo di indirizzamento fisico più vecchio, che mappa i blocchi di dati a una specifica combinazione di cilindro, testina e settore.
 - **Problemi:** Non scalabile per dischi grandi, e la mappatura fisica non è più uniforme a causa di tecniche come Zone Bit Recording. Oggi è quasi obsoleto per l'indirizzamento diretto.
- **LBA (Logical Block Addressing):**
 - **Descrizione:** Il metodo di indirizzamento standard moderno. Il disco è visto come una sequenza lineare di blocchi logici, da 0 al numero totale di blocchi - 1. Il controller del disco si occupa di tradurre questi indirizzi logici in indirizzi fisici (cilindro, testina, settore).
 - **Vantaggi:** Semplifica la gestione del disco per il sistema operativo, nascondendo la complessità della geometria fisica del disco. Permette al controller del disco di ottimizzare la mappatura fisica (es. per il wear leveling negli SSD).

Struttura Fisica del Disco: Settori per Traccia e ZBR

- **Settori per Traccia (Costante vs Variabile):**
 - Nei dischi più vecchi, il numero di settori per traccia era costante su tutto il disco. Questo significava che le tracce esterne (più lunghe) avevano una densità di dati inferiore rispetto a quelle interne, sprecando spazio.
 - Nei dischi moderni, il numero di settori per traccia è **variabile**. Le tracce esterne hanno più settori delle tracce interne, sfruttando meglio lo spazio disponibile.
- **Zone Bit Recording (ZBR):**
 - **Descrizione:** Una tecnica utilizzata nei dischi moderni per implementare il numero variabile di settori per traccia. Il disco è diviso in "zone" concentriche. Ogni zona ha un numero fisso di settori per traccia, ma le zone più esterne hanno più settori per traccia rispetto a quelle interne.
 - **Vantaggi:** Aumenta la capacità di archiviazione del disco.
- **CLV (Constant Linear Velocity) e CAV (Constant Angular Velocity):**
 - **CAV (Constant Angular Velocity):** I piatti del disco girano a una velocità angolare costante (RPM). Questo è comune negli HDD. La velocità di trasferimento dati è maggiore sulle tracce esterne.
 - **CLV (Constant Linear Velocity):** La velocità di rotazione del piatto varia in modo che la velocità lineare della testina rispetto alla traccia sia costante. Questo era comune nei CD/DVD. La velocità di trasferimento dati è costante su tutto il disco.

Scheduling del Disco: L'Ascensorista Intelligente

L'accesso al disco è lento, dominato dal **seek time** (tempo per muovere la testina sul cilindro giusto). Lo scheduler del disco cerca di ordinare le richieste di I/O per minimizzare il movimento della testina.

Coda di richieste (cilindri): 98, 183, 37, 122, 14, 124, 65, 67

Testina parte da: 53

Algoritmo	Descrizione	Movimento Testina	Pro/Contro
FCFS	Serve le richieste nell'ordine in cui arrivano.	$(98-53)+(183-98)\dots$ = 640 cilindri	Semplice, ma inefficiente.
SSTF	Serve la richiesta più vicina alla posizione attuale.	$(53 \rightarrow 65 \rightarrow 67 \rightarrow 37 \rightarrow 14 \dots)$ = 236	Molto efficiente, ma può causare starvation per le richieste lontane.

Algoritmo	Descrizione	Movimento Testina	Pro/Contro
		cilindri	
SCAN	La testina si muove in una direzione fino alla fine fisica del disco (es. cilindro 0 o massimo) , servendo le richieste, poi inverte.	(53->37->14->0) poi (0->65->67...) = 208 cilindri	Più equo di SSTF, non c'è starvation. Questo "tocco all'esterno" (raggiungere i limiti fisici del disco) è una caratteristica distintiva di SCAN/C-SCAN rispetto a LOOK/C-LOOK.
C-SCAN	Come SCAN, ma al ritorno non serve richieste, "salta" direttamente all'inizio fisico del disco .	(53->65...->183) poi (183->199) poi (199->0) poi (0->14->37)	Ancora più equo, l'attesa è più uniforme.
LOOK/C-LOOK	Versioni ottimizzate di SCAN/C-SCAN. La testina inverte la direzione non appena raggiunge l'ultima richiesta in quella direzione, senza andare fino alla fine del disco.	Simile a SCAN/C-SCAN ma con meno movimento.	Sono le versioni più usate nella pratica.

Scelta dell'Algoritmo e Schedulers Linux Specifici

La scelta dell'algoritmo di scheduling del disco dipende dal tipo di carico di lavoro e dal tipo di dispositivo di archiviazione.

- **Algoritmi Generici (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK)**: Sono concetti teorici che servono da base. Nella pratica, i sistemi operativi usano varianti o algoritmi più complessi.
- **Schedulers Linux Specifici**: Il kernel Linux offre diversi schedulers I/O configurabili:
 - **Deadline Scheduler**:
 - **Descrizione**: Cerca di garantire che le richieste I/O vengano servite entro una "deadline" (scadenza) per prevenire la starvation. Mantiene code FIFO separate per richieste di lettura e scrittura, e assegna priorità alle letture (che spesso sono bloccanti per l'utente).
 - **Vantaggi**: Ottimo per database e carichi di lavoro misti, bilancia throughput e latenza.
 - **NOOP Scheduler**:
 - **Descrizione**: È lo scheduler più semplice, un'implementazione di FCFS. Non riordina le richieste, ma le passa direttamente al driver del dispositivo.
 - **Vantaggi**: Ideale per SSD e dispositivi NVMe, che non beneficiano del riordinamento delle richieste (non hanno seek time). L'overhead è minimo.
 - **Svantaggi**: Non adatto per HDD, dove il riordinamento è cruciale.
 - **CFQ (Completely Fair Queueing)**:
 - **Descrizione**: Cerca di distribuire equamente la larghezza di banda I/O tra i processi, dando a ciascuno una "fetta" del tempo del disco.
 - **Vantaggi**: Ottimo per sistemi desktop e multi-utente, dove l'equità è importante.
 - **MQ-DEADLINE/BFQ (Budget Fair Queueing)**: Schedulers più recenti e avanzati, spesso usati come default in distribuzioni Linux moderne, che combinano i vantaggi di deadline e fair queueing.

Gestione del Disco

La gestione del disco include le operazioni per preparare e mantenere i dischi per l'uso da parte del sistema operativo.

Dettagli su Formattazione, Partizionamento, Boot Block

Hai chiesto di espandere la sezione sulla gestione del disco, ed è fondamentale capire come un disco viene preparato per l'uso!

- **Formattazione di Basso Livello (Physical Formatting)**:
 - **Descrizione**: Questo processo iniziale divide il disco in settori fisici, ciascuno composto da un'intestazione (header), un'area dati e una coda (trailer) contenente checksum e un Error Correction Code (ECC).

- **Scopo:** È tipicamente eseguito dal produttore del disco per renderlo utilizzabile dal controller.
- **Partizionamento:**
 - **Descrizione:** Consiste nel dividere il disco fisico in una o più partizioni logiche. Ogni partizione può essere trattata come un disco indipendente e può ospitare un sistema operativo diverso o un file system differente.
 - **Scopo:** Le informazioni sul partizionamento sono memorizzate nel Master Boot Record (MBR) o nella GUID Partition Table (GPT).
- **Formattazione Logica (Logical Formatting / File System Creation):**
 - **Descrizione:** Questa operazione crea le strutture del file system (come il [superblock](#), la [tabella degli inode](#), e le directory root) sulla partizione.
 - **Scopo:** Permettere al sistema operativo di archiviare e recuperare i file in modo organizzato.
- **Blocco di Boot (Boot Block):**
 - **Descrizione:** È il primo settore di un disco o di una partizione di avvio. Contiene un piccolo programma (bootstrap loader) che viene caricato dal BIOS/UEFI.
 - **Scopo:** Ha il compito di caricare il resto del sistema operativo (il kernel) in memoria e passargli il controllo.

Rilevamento e Correzione degli Errori (ECC)

Hai chiesto di approfondire il rilevamento e la correzione degli errori, un aspetto cruciale per l'affidabilità dei dati!

✍ I meccanismi di **rilevamento e correzione degli errori** sono tecniche utilizzate per garantire l'integrità dei dati sui dispositivi di archiviazione, proteggendoli da corruzioni.

- **Bit di Parità:**
 - **Descrizione:** Un metodo semplice per il rilevamento di errori. Un bit aggiuntivo viene calcolato e aggiunto a un gruppo di bit (es. un byte) in modo che il numero totale di bit a '1' sia pari o dispari.
 - **Scopo:** Permette di rilevare errori su un singolo bit, ma **non può correggerli**.
- **ECC (Error Correction Code):**
 - **Descrizione:** Codici più avanzati (es. Codici di Hamming usati in RAID 2) che, oltre a rilevare errori, possono anche correggerli se rientrano in un certo limite (soft errors). L'ECC viene calcolato e memorizzato insieme ai dati.
 - **Scopo:** Durante la lettura, se viene rilevato un errore, l'ECC tenta di correggerlo. Se l'errore è troppo esteso (hard error), il settore/pagina viene marcato come danneggiato.
- **Gestione dei Bad Block:**
 - **Descrizione:** I moderni controller del disco gestiscono i blocchi danneggiati (bad block) internamente. Mantengono una lista di questi blocchi e li sostituiscono automaticamente con settori di ricambio (spare sectors) presenti sul disco stesso.
 - **Scopo:** Questo processo avviene a livello hardware, spesso cercando settori di ricambio vicini per minimizzare l'impatto sul seek time. L'utente o il sistema operativo non vedono direttamente i bad block.

Gestione dello Swap-Space

✍ Lo **swap-space** (o area di swap) è una porzione del disco rigido (o un file) che il sistema operativo utilizza come estensione della memoria RAM. Quando la RAM è piena, il sistema operativo sposta (swappa) le pagine di memoria meno utilizzate dalla RAM allo swap-space.

- **Dimensione:** La dimensione dello swap-space è un compromesso. Se è troppo piccolo, il sistema potrebbe esaurire la memoria virtuale. Se è troppo grande, spreca spazio su disco.
- **Mappatura:** Il sistema operativo mantiene una mappatura tra le pagine logiche dei processi e i blocchi fisici nello swap-space. Quando una pagina viene swappata fuori, la sua voce nella tabella delle pagine viene marcata come "invalida" e aggiornata con l'indirizzo del blocco nello swap-space.

- **Swap File vs Swap Partition:** Lo swap-space può essere implementato come una partizione dedicata sul disco o come un file all'interno di un file system esistente.
- **Vantaggi:** Permette al sistema di eseguire più programmi di quanti ne permetterebbe la sola RAM, e di gestire situazioni di memoria sotto pressione.
- **Svantaggi:** L'accesso al disco è molto più lento della RAM, quindi un uso eccessivo dello swap (thrashing) degrada drasticamente le prestazioni.

RAID: Unione fa la Forza (e la Sicurezza)

RAID (Redundant Array of Independent Disks) combina più dischi fisici in un'unica unità logica per migliorare le performance o fornire ridondanza.

Livello	Icona	Descrizione	Pro	Contro
RAID 0		Striping: Dati divisi su più dischi.	Performance massime.	Nessuna sicurezza. Se un disco fallisce, perdi tutto.
RAID 1		Mirroring: Dati duplicati su due dischi.	Massima sicurezza.	Costoso (50% dello spazio perso).
RAID 2		Striping con Codici Hamming: Dati divisi a livello di bit su più dischi, con dischi dedicati per i codici di correzione errori (Hamming).	Rilevamento e correzione errori on-the-fly.	Molto costoso, non usato commercialmente.
RAID 3		Striping con Parità Dedicata: Dati divisi a livello di byte su più dischi, con un disco dedicato alla parità.	Buone performance per trasferimenti grandi.	Il disco di parità può essere un collo di bottiglia per piccole scritture.
RAID 4		Striping con Parità Dedicata a Blocchi: Dati divisi a livello di blocchi su più dischi, con un disco dedicato alla parità.	Buone performance per letture.	Il disco di parità è un collo di bottiglia per le scritture.
RAID 5		Striping con Parità Distribuita: Dati e parità distribuiti su tutti i dischi.	Buon compromesso tra velocità, costo e sicurezza.	Può sopravvivere al fallimento di 1 solo disco. Scritture più lente.
RAID 6		Striping con Doppia Parità: Come RAID 5 ma con due blocchi di parità.	Può sopravvivere al fallimento di 2 dischi .	Scritture ancora più lente, più costoso di RAID 5.
RAID 10 (1+0)		Stripe of Mirrors: Crea dei set in mirroring (RAID 1) e poi li mette in stripe (RAID 0).	Molto veloce e molto sicuro.	Molto costoso (50% dello spazio perso).

MTBF (Mean Time Between Failure)

 Il **MTBF (Mean Time Between Failure)** è una metrica che indica la durata media prevista tra un guasto e il successivo per un componente hardware (es. un disco rigido).

- **Implicazioni per RAID:** Sebbene un singolo disco possa avere un MTBF molto alto (es. centinaia di migliaia o milioni di ore), in un array RAID con molti dischi, la probabilità che **almeno un disco** fallisca aumenta con il numero di dischi. Ad esempio, in un array con 10 dischi, la probabilità di un guasto è 10 volte superiore rispetto a un singolo disco. Questo rende la ridondanza (RAID 1, 5, 6, 10) cruciale per la disponibilità dei dati.

Stable Storage

 Lo **Stable Storage** (o "memoria stabile") è un'astrazione teorica che garantisce che i dati scritti su di essa siano persistenti anche in caso di fallimenti del sistema (es. interruzione di corrente, crash del software).

- **Implementazione:** Nella pratica, lo stable storage non è un singolo dispositivo, ma viene implementato replicando i dati su più dispositivi di archiviazione indipendenti e utilizzando algoritmi di scrittura che garantiscono che, anche se un dispositivo fallisce durante la scrittura, i dati possano essere recuperati in uno stato consistente dall'altro dispositivo.

- **Scopo:** È fondamentale per garantire l'atomicità delle transazioni nei database e la consistenza dei file system (spesso usando il [journaling](#) come meccanismo sottostante).

Snapshot del File System

Hai chiesto di aggiungere gli snapshot, una funzionalità moderna e importantissima per la resilienza dei dati!

✍ Uno **snapshot** è una "istantanea" (una vista consistente) di un file system in un preciso punto nel tempo. Non è una copia completa di tutti i dati, ma piuttosto un meccanismo efficiente per preservare lo stato del file system senza duplicare l'intero volume.

Funzionamento (es. WAFL - Write-Anywhere File Layout):

File system come WAFL implementano gli snapshot in modo efficiente.

1. Quando viene creato uno snapshot, il sistema registra lo stato attuale dei metadati (es. la root inode).
2. D'ora in poi, ogni volta che un blocco di dati viene modificato, il sistema non lo sovrascrive. Invece, scrive la nuova versione in un **nuovo blocco libero** e aggiorna i puntatori ai metadati.
3. Il blocco originale rimane intatto e fa parte dello snapshot. Questo significa che i dati nello snapshot sono sempre quelli esistenti al momento della sua creazione.

📖 Immagina un quaderno di ricette. Quando fai uno snapshot, è come se mettesti un segnalibro su una pagina specifica. Se poi modifichi una ricetta, non la cancelli dal quaderno originale, ma la riscrivi su una nuova pagina libera e aggiorni l'indice. Il segnalibro punta sempre alla versione originale.

Vantaggi:

- Permette recuperi rapidi a versioni precedenti dei file.
- Backup incrementali efficienti.
- Coerenza dei dati in caso di problemi, poiché i dati scritti non sovrascrivono le versioni precedenti che fanno parte degli snapshot.

VIII. File System

Concetto di File e Directory

✍ Un **file** è un'astrazione fornita dall'SO per memorizzare informazioni in modo persistente sulla memoria secondaria (dischi). Il **file system** è la struttura che organizza e gestisce questi file.

Attributi di un File:

- **Nome:** Il nome simbolico del file.
- **Identificatore:** Un identificatore numerico univoco (es. inode number).
- **Tipo:** Il tipo di file (testo, eseguibile, immagine, directory, ecc.).
- **Posizione:** Puntatore alla posizione del file sul disco.
- **Dimensione:** La dimensione corrente del file.
- **Protezione (Permessi):** Controlli di accesso (chi può leggere, scrivere, eseguire).
- **Data e Ora:** Creazione, ultima modifica, ultimo accesso.
- **Owner/Group:** Proprietario e gruppo di appartenenza.

Operazioni sui File:

- `create()`: Crea un nuovo file.
- `delete()`: Cancella un file.
- `open()`: Apre un file per l'accesso.

- `read()` : Legge dati da un file.
- `write()` : Scrive dati in un file.
- `seek()` : Sposta il puntatore di lettura/scrittura all'interno del file.
- `close()` : Chiude un file.
- `truncate()` : Riduce o estende la dimensione di un file.

Tabella dei File Aperti (System-wide vs Per-process)

Hai chiesto di dettagliare come il sistema operativo gestisce i file aperti, ed è un'ottima domanda che chiarisce la cooperazione tra kernel e processi!

✍ Quando un file viene aperto, il sistema operativo lo gestisce tramite due strutture dati principali in memoria:

1. **Tabella dei file aperti a livello di sistema (System-wide open-file table)** : Contiene una entry per ogni file *attualmente aperto* nell'intero sistema. Ogni entry include:
 - Una copia del File Control Block (FCB) o inode del file (che contiene metadati e puntatori ai blocchi dati).
 - La posizione di lettura/scrittura corrente (offset).
 - Un **contatore delle aperture (file-open count)** che indica quanti processi stanno utilizzando quel file.
2. **Tabella dei file aperti a livello di processo (Per-process open-file table)** : Ogni singolo processo ha la sua tabella, che contiene puntatori alle entry corrispondenti nella tabella a livello di sistema. Questa tabella memorizza anche informazioni specifiche per il processo, come la sua posizione individuale di lettura/scrittura nel file (sebbene spesso condivisa per rientranza) e la modalità di accesso (lettura, scrittura, ecc.).

Funzionamento `open()` / `close()` :

- Quando un processo chiama `open()` , il Logical File System verifica se il file è già presente nella tabella a livello di sistema.
 - Se sì, aggiorna la tabella per-processo e incrementa il `file-open count` nella tabella di sistema.
 - Se no, cerca il file sul disco, copia il suo FCB nella tabella di sistema (creando una nuova entry) e poi aggiorna la tabella per-processo.
- Quando un processo chiama `close()` , il `file-open count` nella tabella di sistema viene decrementato. Solo quando il contatore raggiunge zero, l'entry viene rimossa dalla tabella a livello di sistema e le modifiche (se presenti) vengono scritte su disco.

🔗 Pensa alla tabella di sistema come un "registro generale" di tutti i libri presi in prestito dalla biblioteca, con un contatore per quante persone stanno leggendo lo stesso libro. La tabella per-processo è la tua "lista personale" dei libri che hai in mano, che punta al registro generale.

Directory (o Cartella)

✍ Una **directory** è un tipo speciale di file che contiene un elenco di nomi di file e i puntatori ai loro metadati (es. inode). Organizza i file in una struttura gerarchica.

Struttura delle Directory:

- **Singolo Livello** : Tutti i file sono in un'unica directory (semplice ma non scalabile).
- **Due Livelli** : Una directory master per utente, e poi directory per i file di ogni utente.
- **Struttura ad Albero** : La più comune. Una directory radice, con sottodirectory e file. Permette una gerarchia complessa e organizzata.

Struttura delle Directory: Dettagli su Grafi e Link

Hai chiesto di espandere la sezione sulla struttura delle directory, ed è fondamentale capire come vengono gestiti i collegamenti e i cicli!

La struttura ad albero è la più comune, ma esistono varianti per gestire riferimenti incrociati:

- **Struttura ad Albero:**
 - **Path Assoluti:** Iniziano dalla directory radice (es. `/home/albertofinzi/documenti`).
 - **Path Relativi:** Iniziano dalla directory corrente (es. `documenti/appunti.txt`).
- **Grafo Aciclico (Acyclic-Graph Directory):**
 - **Descrizione:** Permette a un file o una directory di essere referenziata da più di una directory, creando collegamenti (link). Non sono ammessi cicli.
 - **Link (Collegamenti):**
 - **Hard Link:** Un hard link è un puntatore diretto allo stesso inode del file originale. Il file esiste finché almeno un hard link punta ad esso. Non può attraversare i file system (cioè, un hard link può puntare solo a file sullo stesso volume/partizione).
 - **Symbolic Link (Soft Link):** Un symbolic link è un file speciale che contiene il percorso (path) di un altro file o directory. È un puntatore indiretto. Può puntare a file su file system diversi e può puntare a file inesistenti (creando un **dangling pointer**).
 - **Dangling Pointer:** Un symbolic link che punta a un file o directory che è stato cancellato. Il link diventa "rotto" o "dangling".
 - **Gestione dei Riferimenti:** Per i hard link, il sistema operativo mantiene un **contatore di riferimenti** (reference count) nell'inode. Quando un link viene rimosso, il contatore decrementa. Il file viene effettivamente cancellato solo quando il contatore raggiunge zero.
- **Grafo Generale (General-Graph Directory):**
 - **Descrizione:** Permette cicli nella struttura delle directory. Questo offre la massima flessibilità ma introduce problemi di gestione.
 - **Problemi:**
 - **Attraversamento:** Gli algoritmi che attraversano la directory (es. per calcolare la dimensione totale o per i backup) devono essere modificati per evitare loop infiniti.
 - **Garbage Collection:** La cancellazione dei file diventa complessa. Se un file è parte di un ciclo, il suo contatore di riferimenti potrebbe non raggiungere mai zero, anche se non è più raggiungibile da nessuna directory "radice". Richiede algoritmi di **garbage collection** (come quelli usati nei linguaggi di programmazione) per identificare e recuperare lo spazio occupato da file non più referenziati.

Implementazione del File System

Il file system è un software complesso con una struttura a strati per gestire l'archiviazione persistente.

Struttura a Livelli del File System

Il file system è spesso organizzato in strati, ognuno con responsabilità specifiche:

1. **Livello di Applicazione:** Programmi utente che usano le API del file system (es. `open()`, `read()`).
2. **Livello Logico del File System:** Gestisce le strutture delle directory, i permessi, i metadati (inode). Traduce i nomi di file in inode numbers.
3. **Livello di Organizzazione dei File:** Traduce gli inode numbers in blocchi logici del disco. Gestisce l'allocazione dei blocchi (es. [allocazione indicizzata](#)).
4. **Livello di Controllo Base del File System:** Gestisce i blocchi logici del disco (es. quali blocchi sono liberi, quali sono occupati).
5. **Livello di Controllo I/O (Device Drivers):** Interagisce direttamente con l'hardware del disco, inviando comandi al controller del disco.

Strutture Dati del File System

Il kernel usa diverse strutture dati per gestire i file:

1. **Superblock:** Contiene informazioni globali sul file system (es. numero totale di blocchi, dimensione dei blocchi, lista dei blocchi liberi, numero di inode disponibili). Viene caricato in memoria all'avvio del sistema e all'atto del montaggio del file system.
2. **Inode (o File Control Block - FCB):** Contiene tutti i metadati di un singolo file (tipo di file, permessi, owner, date, dimensione) e, cruciale, i puntatori ai blocchi di dati che compongono il

file. Ogni file ha un inode unico.

3. **Directory**: È un tipo speciale di file il cui contenuto è una lista di coppie `(nome_file, puntatore_all_inode)`.

Implementazione delle Directory

Hai chiesto di spiegare come vengono implementate le directory, ed è un dettaglio importante!

Le directory sono file speciali che contengono una mappatura tra nomi di file e i loro inode (o FCB). Le implementazioni comuni includono:

- **Lista Lineare**:
 - **Descrizione**: La directory è semplicemente una lista sequenziale di coppie `(nome_file, inode_number)`.
 - **Vantaggi**: Semplice da implementare.
 - **Svantaggi**: Lenta per la ricerca (richiede una scansione lineare) e per l'inserimento/cancellazione (potrebbe richiedere lo spostamento di molte entry), specialmente per directory grandi.
- **Hash Table (con Chained-Overflow)**:
 - **Descrizione**: La directory è implementata come una tabella hash. Una funzione hash viene applicata al nome del file per ottenere un indice nella tabella. Se ci sono collisioni (due nomi hashano allo stesso indice), i file vengono collegati in una lista concatenata (chained-overflow).
 - **Vantaggi**: Molto veloce per la ricerca, l'inserimento e la cancellazione (tempo medio $O(1)$).
 - **Svantaggi**: Più complessa da implementare. La performance dipende dalla qualità della funzione hash e dalla gestione delle collisioni.

Metodi di Allocazione dei File

Come vengono memorizzati i blocchi di un file sul disco?

Metodo	Descrizione	Pro	Contro
Contigua	Il file occupa un insieme di blocchi contigui sul disco.	Accesso veloce (sia sequenziale che diretto). Semplice.	Frammentazione esterna . Difficile far crescere i file.
Collegata (Linked)	Ogni blocco contiene un puntatore al blocco successivo.	Niente frammentazione esterna. I file possono crescere facilmente.	Accesso solo sequenziale. Lento. Inaffidabile se un puntatore si corrompe.
Indicizzata (Indexed)	Un blocco speciale (il blocco indice o inode) contiene i puntatori a tutti i blocchi del file.	Supporta accesso diretto. Niente frammentazione esterna.	Spreco di spazio per il blocco indice (specialmente per file piccoli). Può limitare la dimensione del file.

🔑 L'allocazione **indicizzata** è la base dei moderni file system come **ext4** (Linux) e **NTFS** (Windows), spesso con schemi a più livelli di indizione per supportare file molto grandi.

Metodi di Accesso per i File

Metodi di Accesso per i File

Ottima aggiunta! Capire come le applicazioni accedono ai file è fondamentale per comprendere le scelte di design del file system.

Diversi tipi di applicazioni richiedono diverse modalità di accesso ai file. Il sistema operativo fornisce astrazioni per questi metodi:

1. **Accesso Sequenziale (Sequential Access)**:

- **Descrizione**: Il file viene trattato come un flusso di byte (o record) e i dati vengono acceduti in sequenza, dall'inizio alla fine. Le operazioni tipiche sono `read next` (leggi il

prossimo byte/record) e **write next** (scrivi il prossimo byte/record). C'è anche un'operazione **reset** per tornare all'inizio del file.

- **Uso:** Ideale per file di testo, file audio/video in streaming, compilatori (che leggono il sorgente sequenzialmente).
- **Relazione con Allocazione:** Funziona bene sia su dispositivi sequenziali (es. nastri magnetici) che su quelli ad accesso diretto (dischi), ed è particolarmente efficiente con l'allocazione collegata.

2. Accesso Diretto (Direct Access o Random Access):

- **Descrizione:** Permette di accedere a qualsiasi blocco (o record logico) del file direttamente, specificandone il numero. Le operazioni tipiche sono **read n** (leggi il blocco n), **write n** (scrivi il blocco n), e **position to n** (sposta il puntatore al blocco n).
- **Uso:** Tipico per database, dove è necessario accedere rapidamente a record specifici senza leggere l'intero file.
- **Relazione con Allocazione:** Richiede un'allocazione che permetta un calcolo rapido della posizione fisica del blocco, come l'allocazione contigua o indicizzata.

3. Accesso Indicizzato (Indexed Access):

- **Descrizione:** Questo metodo è un miglioramento dell'accesso diretto, particolarmente utile per file molto grandi o quando le ricerche sono frequenti. Si basa sulla creazione di un **indice** (spesso un file separato o una struttura dati interna al file system) che contiene puntatori ai blocchi dati. Per trovare un record, si cerca prima nell'indice e poi si accede direttamente al blocco dati.
- **Uso:** Database complessi, sistemi di gestione di grandi volumi di dati.
- **Relazione con Allocazione:** Si basa fortemente sull'allocazione indicizzata e può utilizzare strutture dati come alberi B-tree per gli indici per ottimizzare le ricerche.

File-Allocation Table (FAT)

Hai chiesto di aggiungere la FAT, un metodo storico ma ancora rilevante!

✍ La **File-Allocation Table (FAT)** è un metodo di allocazione dei file che è una variante dell'allocazione collegata (linked allocation). Invece di avere i puntatori al blocco successivo all'interno di ogni blocco dati, tutti i puntatori sono raccolti in una tabella speciale (la FAT) che risiede all'inizio del volume.

- **Struttura:** La FAT è un array, dove ogni entry corrisponde a un blocco dati sul disco. Il valore di un'entry nella FAT indica il numero del blocco successivo nella catena del file, o un valore speciale se il blocco è l'ultimo del file, è libero o è danneggiato.
- **Funzionamento:** Per trovare i blocchi di un file, il sistema operativo legge la FAT. L'inode (o la directory entry) di un file contiene solo il numero del primo blocco. Poi, si segue la catena di puntatori nella FAT per trovare tutti i blocchi.
- **Varianti:** Esistono diverse varianti (FAT12, FAT16, FAT32) che differiscono per la dimensione delle entry nella tabella, e quindi per la dimensione massima del volume e il numero massimo di blocchi indirizzabili.
- **Vantaggi:**
 - Semplice da implementare.
 - Nessuna frammentazione esterna.
 - Supporta accesso sequenziale e, con la FAT in memoria, anche accesso diretto (sebbene lento per file molto grandi).
- **Svantaggi:**
 - La FAT può diventare molto grande per dischi di grandi dimensioni, occupando molta RAM.
 - La ricerca di un blocco specifico in un file grande può essere lenta (richiede di attraversare la catena nella FAT).
 - Vulnerabile a corruzioni: se la FAT si danneggia, si perdono i puntatori a molti file.

Gestione dello Spazio Libero

Il file system deve tenere traccia di quali blocchi di disco sono liberi e quali sono occupati.

- **Bit Vector (Bitmap):**

- **Descrizione:** Un array di bit, dove ogni bit corrisponde a un blocco del disco. Se il bit è 0, il blocco è libero; se è 1, è occupato.
- **Vantaggi:** Semplice, efficiente per trovare blocchi contigui.
- **Svantaggi:** Può essere grande per dischi molto grandi.
- **Lista Collegata (Linked List):**
 - **Descrizione:** Tutti i blocchi liberi sono collegati tra loro in una lista. Ogni blocco libero contiene un puntatore al prossimo blocco libero.
 - **Vantaggi:** Nessuno spreco di spazio per una bitmap.
 - **Svantaggi:** Lento per trovare blocchi contigui. Inaffidabile se un puntatore si corrompe.
- **Raggruppamento (Grouping):**
 - **Descrizione:** Memorizza gli indirizzi di N blocchi liberi nel primo blocco libero. L'ultimo di questi N blocchi contiene gli indirizzi dei prossimi N blocchi liberi, e così via.
 - **Vantaggi:** Migliora la velocità di ricerca rispetto alla lista collegata semplice.
- **Conteggio (Counting):**
 - **Descrizione:** Invece di liste di singoli blocchi, si memorizzano coppie (`indirizzo_inizio, conteggio_blocchi_contigui_liberi`).
 - **Vantaggi:** Efficiente per trovare grandi quantità di spazio contiguo, riduce la dimensione della lista dei blocchi liberi.

File System Linux (EXT, EXT2, EXT3, EXT4) e NTFS

Hai chiesto di dettagliare i file system moderni, ed è un'ottima idea per capire le implementazioni reali!

I file system moderni sono complessi e offrono funzionalità avanzate per prestazioni, affidabilità e scalabilità.

- **EXT (Extended File System):**
 - **EXT2:** Il primo file system "esteso" per Linux, introdotto nel 1993. Non supporta il journaling.
 - **EXT3:** Evoluzione di EXT2 con l'aggiunta del **journaling**. Permette un recupero rapido dopo un crash.
 - **EXT4:** L'attuale file system predefinito per molte distribuzioni Linux. Include miglioramenti significativi rispetto a EXT3:
 - **Extent-based allocation:** Alloca blocchi contigui per file grandi, migliorando le prestazioni e riducendo la frammentazione. Un "extent" è un blocco contiguo di dati, riducendo il numero di puntatori necessari nell'inode per file grandi.
 - Supporto per file system e file di dimensioni maggiori (fino a 1 EB per FS, 16 TB per file).
 - Miglioramenti nel journaling e nella gestione degli inode.
 - **Delayed allocation:** Ritarda l'allocazione fisica dei blocchi fino a quando i dati non sono effettivamente scritti su disco, permettendo al file system di allocare blocchi più grandi e contigui.
- **NTFS (New Technology File System):**
 - Il file system predefinito per i sistemi operativi Windows NT e successivi (Windows 2000, XP, Vista, 7, 8, 10, 11).
 - **Caratteristiche Principali:**
 - **Journaling:** Supporta il journaling per la resilienza ai crash.
 - **Sicurezza:** Supporta permessi a livello di file e directory (ACL - Access Control Lists).
 - **Compressione e Crittografia:** Supporta la compressione e la crittografia trasparente dei file.
 - **Hard Links e Symbolic Links:** Simili a quelli di Unix.
 - **Alternate Data Streams (ADS):** Permette di associare più flussi di dati a un singolo file.
 - **Change Journal:** Un log di tutte le modifiche al volume, utile per backup incrementali e indicizzazione.
 - **Resilience:** Utilizza transazioni e journaling per mantenere la consistenza.

Efficienza e Prestazioni del File System

Hai chiesto di aggiungere una sezione sull'efficienza, ed è cruciale per capire le scelte di design!

Il design di un file system implica numerosi compromessi tra efficienza, prestazioni, affidabilità e funzionalità. Alcune scelte di design che influenzano l'efficienza includono:

- **Collo di Bottiglia (Dischi)**: La velocità di accesso ai dischi è il principale fattore limitante per le prestazioni del file system. Tutte le ottimizzazioni mirano a ridurre il numero di accessi al disco o a renderli più efficienti.
- **Allocazione Anticipata degli FCB (File Control Block)**: Alcuni file system allocano gli FCB (o inode) in anticipo, o li raggruppano in modo intelligente sul disco, per ridurre il seek time quando si creano nuovi file.
- **Scelta degli Attributi da Memorizzare**: La quantità e il tipo di attributi memorizzati per ogni file (es. data di ultimo accesso, data di creazione, permessi) influenzano la dimensione dell'inode e il tempo necessario per accedervi. Memorizzare meno attributi può rendere l'accesso più veloce, ma a costo di perdere informazioni utili.
- **Lunghezza dei Puntatori**: Nei metodi di allocazione indicizzata, la lunghezza dei puntatori ai blocchi dati (es. 32 bit vs 64 bit) influenza la dimensione massima del file e la quantità di spazio richiesta per il blocco indice. Puntatori più lunghi permettono file più grandi ma occupano più spazio nel blocco indice.
- **Dimensione dei Blocchi**: La dimensione dei blocchi logici del file system è un compromesso:
 - **Blocchi grandi**: Riducono il numero di accessi al disco per file grandi, migliorano il throughput, ma aumentano la frammentazione interna.
 - **Blocchi piccoli**: Riducono la frammentazione interna, ma aumentano il numero di accessi al disco e l'overhead di gestione.

Cache del Disco / Page Cache (e Double Buffering)

Hai chiesto di spiegare l'evoluzione delle cache nel kernel e il problema del "double buffering", un'ottima intuizione!

✍ **Storicamente, i sistemi operativi potevano avere cache separate per diversi tipi di dati: una "buffer cache" per i blocchi disco grezzi e una "page cache" per le pagine di memoria virtuale dei processi. Questa separazione poteva portare al "double buffering" o "double caching".**

Problema del Double Buffering:

Il "double buffering" si verifica quando i dati vengono copiati inutilmente più volte tra i diversi buffer del kernel e/o lo spazio utente. Ad esempio, un dato letto dal disco potrebbe essere prima copiato nella buffer cache, poi nella page cache, e solo dopo nello spazio di indirizzamento dell'applicazione. Ogni copia aggiunge overhead e rallenta le prestazioni.

Soluzione: Unified Buffer Cache (o Unified Page Cache):

I sistemi operativi moderni implementano una **"Unified Buffer Cache"** (o "Unified Page Cache") che integra la gestione di tutte le cache a livello di kernel. Questa cache unificata serve sia i blocchi del file system che le pagine di memoria virtuale, riducendo le copie di dati e migliorando l'efficienza complessiva. Il sistema di [memoria virtuale](#) si interfaccia direttamente con questa cache, evitando la ridondanza. Gli algoritmi di [sostituzione delle pagine](#) (es. LRU) vengono utilizzati per gestire lo spazio in questa cache unificata.

📦 **Pensa a un magazzino. Invece di avere una sezione per i "pacchi in arrivo" e un'altra per i "pacchi da spedire" (con gli stessi pacchi copiati tra le due), hai un unico grande magazzino dove tutti i pacchi sono gestiti in modo centralizzato. Questo evita di spostare inutilmente le cose.**

Scritture Sincrone e Asincrone

Hai chiesto di spiegare la differenza tra scritture sincrone e asincrone, un concetto chiave per le performance!

Il modo in cui le operazioni di scrittura vengono gestite dal sistema operativo ha un impatto significativo sulle prestazioni e sull'affidabilità.

- **Scritture Sincrone (Synchronous Writes) :**

- **Descrizione:** Quando un'applicazione esegue una scrittura sincrona, il sistema operativo blocca il processo chiamante finché i dati non sono stati fisicamente scritti sul dispositivo di archiviazione (es. disco).
- **Vantaggi:** Massima garanzia di persistenza dei dati. In caso di crash, i dati scritti sono sicuramente sul disco.
- **Svantaggi:** Molto lente, poiché il processo deve attendere il completamento dell'I/O fisico. Riducono il throughput complessivo del sistema.
- **Uso:** Critiche per database e sistemi transazionali che richiedono la massima integrità dei dati.

- **Scritture Asincrone (Asynchronous Writes) :**

- **Descrizione:** Quando un'applicazione esegue una scrittura asincrona, il sistema operativo copia i dati in un buffer di memoria (cache del disco / page cache) e restituisce immediatamente il controllo al processo chiamante. La scrittura fisica sul disco avviene in un secondo momento, in background.
- **Vantaggi:** Molto veloci, poiché il processo non è bloccato dall'I/O lento. Migliorano il throughput e la reattività del sistema.
- **Svantaggi:** In caso di crash del sistema prima che i dati siano stati scritti fisicamente sul disco, le modifiche potrebbero essere perse.
- **Uso:** Comuni per la maggior parte delle operazioni di scrittura generali, dove la performance è più critica della garanzia immediata di persistenza. Il [journaling](#) mitiga il rischio di inconsistenza in caso di crash.

Recupero e Consistenza

- **Problemi di Incoerenza dopo Crash:** Se un sistema si blocca improvvisamente (es. mancanza di corrente) mentre il file system sta eseguendo un'operazione complessa (es. creazione di un file, spostamento di una directory), le strutture dati sul disco potrebbero rimanere in uno stato inconsistente.
- **Consistency Checker (es. `fsck` in Unix):** Programmi che scansionano il file system dopo un crash per rilevare e correggere le inconsistenze. Possono essere molto lenti per dischi grandi.
- **Journaling (log-based transaction):** Già discusso nella [Sezione VII](#), è la soluzione moderna per garantire la consistenza dopo un crash, registrando le operazioni in un log prima di eseguirle.



File System Remoto: NFS (Network File System)



Un **File System Remoto** è un file system che permette agli utenti di accedere a file e directory su un computer remoto (server) come se fossero memorizzati localmente.

NFS (Network File System) è uno dei più diffusi file system remoti, sviluppato da Sun Microsystems.

Come funziona:

- Un server NFS esporta una o più directory, rendendole disponibili per l'accesso remoto.
- I client NFS "montano" queste directory remote nel loro file system locale.
- Quando un'applicazione client accede a un file nella directory montata, la richiesta viene inviata al server NFS, che esegue l'operazione e restituisce il risultato.

Vantaggi:

- **Condivisione di risorse:** Permette a più client di accedere agli stessi dati.
- **Centralizzazione:** Semplifica la gestione dei dati e i backup.
- **Trasparenza:** L'utente interagisce con i file remoti come se fossero locali.

Svantaggi:

- **Performance:** Dipende dalla velocità della rete e dal carico del server.

- **Sicurezza**: Richiede configurazioni attente per prevenire accessi non autorizzati.
- **Consistenza**: Può presentare problemi di consistenza dei dati in ambienti concorrenti (vedi sotto).

👉 Consistency Semantic (Semantiche di Consistenza)

In un file system condiviso (come NFS), più utenti o processi possono accedere allo stesso file contemporaneamente. La **semantica di consistenza** definisce come le modifiche fatte da un utente/processo sono visibili agli altri.

- **Unix Semantics (o Session Semantics)**:
 - **Descrizione**: Le modifiche fatte da un processo a un file sono immediatamente visibili a tutti gli altri processi che hanno il file aperto sulla **stessa macchina**.
 - **Problema in NFS**: Su NFS, questa semantica è difficile da garantire. Le modifiche fatte da un client potrebbero non essere immediatamente visibili a un altro client, a causa del caching lato client o dei ritardi di rete. NFS tipicamente implementa una "session semantics" o "close-to-open consistency", dove le modifiche sono garantite essere visibili solo dopo che il file è stato chiuso dal mittente e riaperto dal ricevente.
- **Session Semantics (o Close-to-Open Consistency)**:
 - **Descrizione**: Le modifiche a un file sono garantite essere visibili agli altri processi solo dopo che il file è stato **chiuso** dal processo che ha effettuato le modifiche e **riaperto** dagli altri processi.
 - **Uso**: Comune in NFS per bilanciare consistenza e performance (riducendo la necessità di sincronizzazione continua).
- **Immutable Shared Files**:
 - **Descrizione**: Una volta creato e scritto, un file non può più essere modificato. Se si vuole cambiarlo, si deve creare una nuova versione del file.
 - **Vantaggi**: Massima consistenza e semplicità, ideale per caching aggressivo.
 - **Svantaggi**: Non adatto per file che cambiano frequentemente.

🔗 La scelta della semantica di consistenza è un compromesso tra la forza della consistenza (quanto sono aggiornati i dati) e le prestazioni (overhead di sincronizzazione).

IX. Esercizi e Esempi Pratici

🐧 Comandi Linux/Bash Essenziali

Ecco una piccola "cheat sheet" dei comandi Linux più utili per Sistemi Operativi I. Usali nel tuo terminale per fare pratica!

Comando	Descrizione	Esempio		
<code>ls</code>	Elenca il contenuto di una directory.	<code>ls -l</code> (dettagliato)		
<code>cd</code>	Cambia directory.	<code>cd /home/albertofinzi/docs</code>		
<code>pwd</code>	Stampa la directory corrente.	<code>pwd</code>		
<code>mkdir</code>	Crea una nuova directory.	<code>mkdir miei_appunti</code>		
<code>rmdir</code>	Rimuove una directory vuota.	<code>rmdir vecchia_cartella</code>		
<code>touch</code>	Crea un file vuoto o aggiorna la data di modifica.	<code>touch nuovo_file.txt</code>		
<code>cat</code>	Visualizza il contenuto di un file.	<code>cat readme.md</code>		
<code>echo</code>	Stampa una stringa o variabile.	<code>echo "Ciao mondo"</code>		
<code>grep</code>	Cerca pattern all'interno di file.	<code>grep "errore" log.txt</code>		
<code>wc</code>	Conta parole, linee, caratteri.	<code>wc -l file.txt</code> (conta linee)		
<code>ps</code>	Mostra i processi in esecuzione.	<code>ps aux</code> (tutti i processi)		
<code>kill</code>	Termina un processo.	<code>kill 1234</code> (termina PID 1234)		

Comando	Descrizione	Esempio		
<code>chmod</code>	Cambia i permessi di un file/directory.	<code>chmod 755 script.sh</code>		
<code>gcc</code>	Compilatore C/C++.	<code>gcc main.c -o programma</code>		
<code>man</code>	Manuale per i comandi.	<code>man ls</code>		
<code>></code>	Redirezione output su file (sovrascrive).	<code>ls > lista.txt</code>		
<code>>></code>	Redirezione output su file (appende).	<code>echo "fine" >> lista.txt</code>		
<code><</code>	Redirezione input da file.	<code>wc -l < file.txt</code>		
`	`	Pipe (collega output di un comando a input di un altro).	<code>`ps aux`</code>	<code>grep firefox`</code>
<code>export</code>	Esporta una variabile d'ambiente.	<code>export MY_VAR="valore"</code>		



Esercizi di Scheduling della CPU

Esercizio 1: FCFS, SJF (Non-Preemptive), Priority (Non-Preemptive), Round Robin

Dato il seguente set di processi con tempo di arrivo (Arrival Time) e durata del burst di CPU (Burst Time), e priorità (Priority, 1 = più alta):

Processo	Arrival Time	Burst Time	Priority
P1	0	10	3
P2	0	1	1
P3	0	2	2
P4	0	7	4

Calcola il tempo di attesa medio e il tempo di turnaround medio per i seguenti algoritmi:

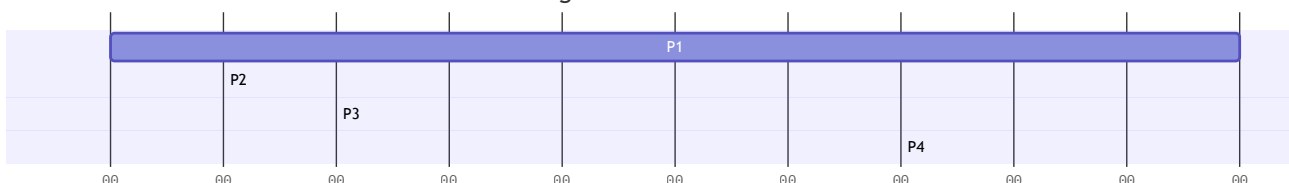
1. FCFS (First-Come, First-Served)
2. SJF (Shortest-Job-First) Non-Preemptive
3. Priority Scheduling Non-Preemptive
4. Round Robin (RR) con quanto = 3 ms

Soluzione (Passo dopo passo):

1. FCFS (First-Come, First-Served)

- **Ordine di esecuzione:** P1, P2, P3, P4 (dato che arrivano tutti a $t=0$, l'ordine è per nome)

Diagramma di Gantt - FCFS

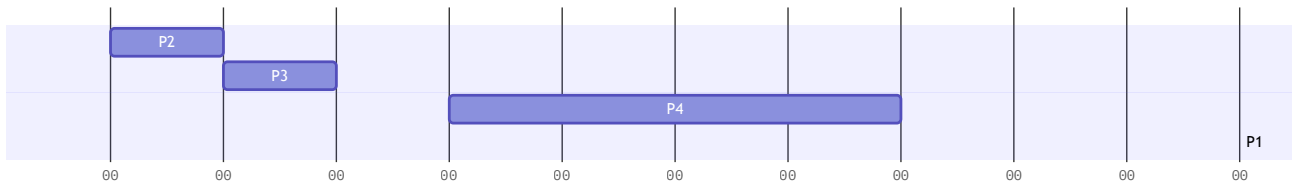


- **Calcoli:**
 - P1: Start=0, End=10, Turnaround=10, Wait=0
 - P2: Start=10, End=11, Turnaround=11, Wait=10
 - P3: Start=11, End=13, Turnaround=13, Wait=11
 - P4: Start=13, End=20, Turnaround=20, Wait=13
- **Tempo di attesa medio:** $(0 + 10 + 11 + 13)/4 = 34/4 = 8.5$ ms
- **Tempo di turnaround medio:** $(10 + 11 + 13 + 20)/4 = 54/4 = 13.5$ ms

2. SJF (Shortest-Job-First) Non-Preemptive

- **Ordine di esecuzione:** P2 (1ms), P3 (2ms), P4 (7ms), P1 (10ms)

Diagramma di Gantt - SJF (Non-Preemptive)

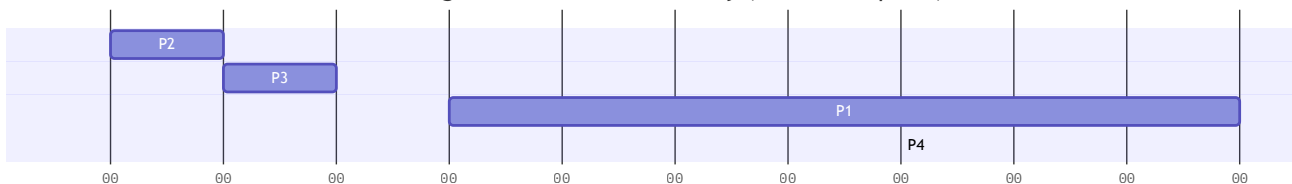


- **Calcoli:**
 - P2: Start=0, End=1, Turnaround=1, Wait=0
 - P3: Start=1, End=3, Turnaround=3, Wait=1
 - P4: Start=3, End=10, Turnaround=10, Wait=3
 - P1: Start=10, End=20, Turnaround=20, Wait=10
- **Tempo di attesa medio:** $(0 + 1 + 3 + 10)/4 = 14/4 = 3.5 \text{ ms}$
- **Tempo di turnaround medio:** $(1 + 3 + 10 + 20)/4 = 34/4 = 8.5 \text{ ms}$

3. Priority Scheduling Non-Preemptive

- **Ordine di esecuzione:** P2 (Prio 1), P3 (Prio 2), P1 (Prio 3), P4 (Prio 4)

Diagramma di Gantt - Priority (Non-Preemptive)

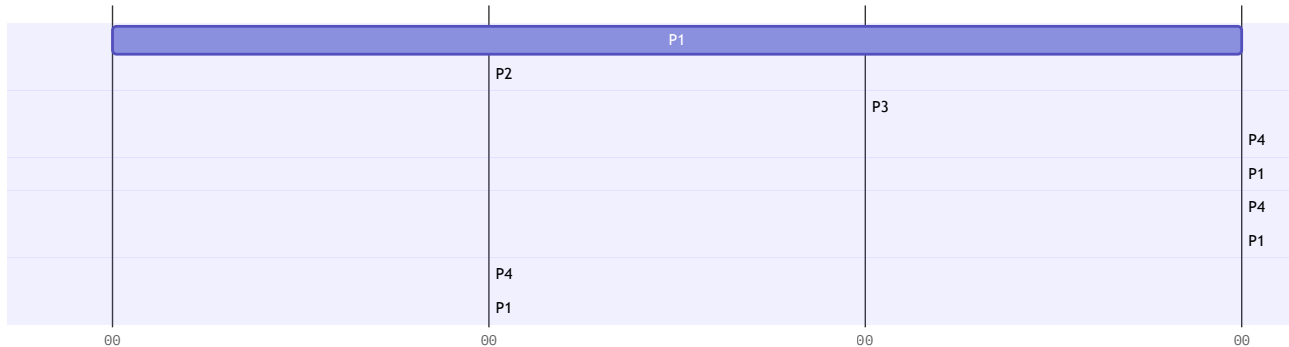


- **Calcoli:**
 - P2: Start=0, End=1, Turnaround=1, Wait=0
 - P3: Start=1, End=3, Turnaround=3, Wait=1
 - P1: Start=3, End=13, Turnaround=13, Wait=3
 - P4: Start=13, End=20, Turnaround=20, Wait=13
- **Tempo di attesa medio:** $(0 + 1 + 3 + 13)/4 = 17/4 = 4.25 \text{ ms}$
- **Tempo di turnaround medio:** $(1 + 3 + 13 + 20)/4 = 37/4 = 9.25 \text{ ms}$

4. Round Robin (RR) con quanto = 3 ms

- **Ordine di esecuzione:**
 - t=0: P1 (burst 10, esegue 3) -> P1 (rim. 7)
 - t=3: P2 (burst 1, esegue 1) -> P2 (rim. 0, termina)
 - t=4: P3 (burst 2, esegue 2) -> P3 (rim. 0, termina)
 - t=6: P4 (burst 7, esegue 3) -> P4 (rim. 4)
 - t=9: P1 (rim. 7, esegue 3) -> P1 (rim. 4)
 - t=12: P4 (rim. 4, esegue 3) -> P4 (rim. 1)
 - t=15: P1 (rim. 4, esegue 3) -> P1 (rim. 1)
 - t=18: P4 (rim. 1, esegue 1) -> P4 (rim. 0, termina)
 - t=19: P1 (rim. 1, esegue 1) -> P1 (rim. 0, termina)

Diagramma di Gantt - Round Robin (q=3)



- **Calcoli:**
 - P1: End=20, Turnaround=20, Wait=20-10=10
 - P2: End=4, Turnaround=4, Wait=4-1=3
 - P3: End=6, Turnaround=6, Wait=6-2=4
 - P4: End=19, Turnaround=19, Wait=19-7=12
- **Tempo di attesa medio:** $(10 + 3 + 4 + 12)/4 = 29/4 = 7.25$ ms
- **Tempo di turnaround medio:** $(20 + 4 + 6 + 19)/4 = 49/4 = 12.25$ ms

Esercizio 1.5: SRTF (Shortest-Remaining-Time-First) Preemptive

Dato il seguente set di processi con tempo di arrivo (Arrival Time) e durata del burst di CPU (Burst Time):

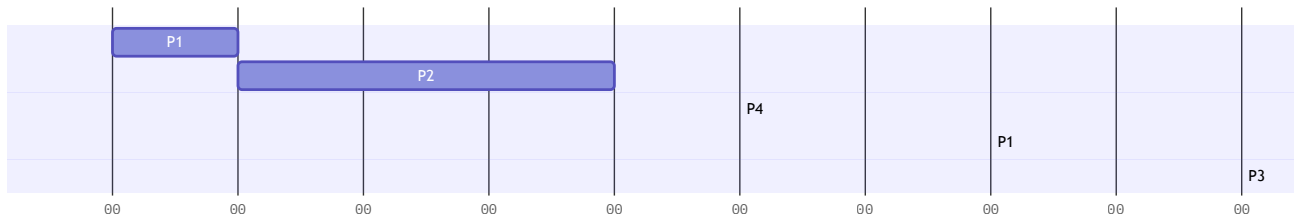
Processo	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Calcola il tempo di attesa medio e il tempo di turnaround medio per l'algoritmo **SRTF (Shortest-Remaining-Time-First)**.

Soluzione (Passo dopo passo):

- **Stato Iniziale:**
 - Tempo = 0: P1 arriva (Burst = 8). P1 inizia.
- **t=1:** P2 arriva (Burst = 4). P1 (rimanente 7) vs P2 (rimanente 4). P2 ha burst più corto. P1 viene prelaziato. P2 inizia.
- **t=2:** P3 arriva (Burst = 9). P2 (rimanente 3) vs P3 (rimanente 9). P2 ha burst più corto. P2 continua.
- **t=3:** P4 arriva (Burst = 5). P2 (rimanente 2) vs P4 (rimanente 5). P2 ha burst più corto. P2 continua.
- **t=5:** P2 termina. Ora i processi pronti sono P1 (rimanente 7), P3 (rimanente 9), P4 (rimanente 5). Il più corto è P4. P4 inizia.
- **t=10:** P4 termina. Ora i processi pronti sono P1 (rimanente 7), P3 (rimanente 9). Il più corto è P1. P1 inizia.
- **t=17:** P1 termina. Ora solo P3 (rimanente 9) è pronto. P3 inizia.
- **t=26:** P3 termina.

Diagramma di Gantt - SRTF



- **Calcoli:**

- **P1:**

- Arrivo: 0
 - Inizio esecuzione: 0
 - Prelazione: 1
 - Riprende: 10
 - Fine: 17
 - Turnaround: $17 - 0 = 17$
 - Wait: $17 - 8 = 9$ (Tempo totale in ready queue: $(10 - 1) = 9$)

- **P2:**

- Arrivo: 1
 - Inizio esecuzione: 1
 - Fine: 5
 - Turnaround: $5 - 1 = 4$
 - Wait: $4 - 4 = 0$

- **P3:**

- Arrivo: 2
 - Inizio esecuzione: 17
 - Fine: 26
 - Turnaround: $26 - 2 = 24$
 - Wait: $24 - 9 = 15$ (Tempo totale in ready queue: $(17 - 2) = 15$)

- **P4:**

- Arrivo: 3
 - Inizio esecuzione: 5
 - Fine: 10
 - Turnaround: $10 - 3 = 7$
 - Wait: $7 - 5 = 2$ (Tempo totale in ready queue: $(5 - 3) = 2$)

- **Tempo di attesa medio:** $(9 + 0 + 15 + 2)/4 = 26/4 = 6.5$ ms
- **Tempo di turnaround medio:** $(17 + 4 + 24 + 7)/4 = 52/4 = 13$ ms



Esercizi su `fork()`

Esercizio 2: Analisi di Codice `fork()`

Considera il seguente codice C:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    printf("Inizio del programma.\n");
    pid = fork();

    if (pid == 0) { // Codice eseguito dal processo figlio
        printf("Sono il figlio. Il mio PID è %d, il mio padre è %d.\n", getpid(), getppid());
    } else if (pid > 0) { // Codice eseguito dal processo padre
```

```

    wait(NULL); // Il padre attende che il figlio termini
    printf("Sono il padre. Il mio PID è %d, il mio figlio è %d.\n", getpid(), pid);
} else { // Errore nella fork
    fprintf(stderr, "Errore nella fork!\n");
    return 1;
}
printf("Fine del processo (PID: %d).\n", getpid());
return 0;
}

```

Domande:

1. Quanti processi verranno creati in totale (incluso il processo iniziale)?
2. Quale sarà l'output *possibile* del programma? Spiega l'ordine delle stampe.

Soluzione:

1. **Numero totale di processi:** Verranno creati **2 processi** in totale. Il processo iniziale (`main`) e un processo figlio creato dalla `fork()`.
2. **Output possibile:**

```

Inizio del programma.
Sono il figlio. Il mio PID è [PID_figlio], il mio padre è [PID_padre].
Fine del processo (PID: [PID_figlio]).
Sono il padre. Il mio PID è [PID_padre], il mio figlio è [PID_figlio].
Fine del processo (PID: [PID_padre]).

```

Spiegazione dell'ordine:

- `printf("Inizio del programma.\n");` viene stampato una sola volta dal processo padre prima della `fork()`.
- Dopo la `fork()`, sia il padre che il figlio continuano l'esecuzione.
- Il figlio entra nel blocco `if (pid == 0)` e stampa la sua riga.
- Il padre entra nel blocco `else if (pid > 0)`. La chiamata `wait(NULL)` fa sì che il processo padre si **blocchi** e attenda la terminazione del figlio.
- Quindi, il figlio termina la sua esecuzione, stampa la sua riga "Fine del processo" e termina.
- Solo dopo che il figlio è terminato, la `wait()` del padre ritorna, e il padre riprende l'esecuzione, stampa la sua riga e poi la sua riga "Fine del processo".
- L'ordine esatto delle due righe "Fine del processo" (figlio vs padre) può variare leggermente a seconda della schedulazione, ma la riga del figlio sarà sempre prima della seconda riga del padre grazie alla `wait()`.



Esercizi sulla Gestione della Memoria

Esercizio 3: Calcolo dell'Effective Access Time (EAT) con TLB

Supponiamo un sistema di paginazione con i seguenti parametri:

- Tempo di accesso alla memoria (RAM): 100 ns
- Tempo di accesso alla TLB (hit time): 20 ns
- Hit ratio della TLB (percentuale di volte in cui la traduzione è trovata nella TLB): 98% (0.98)

Calcola l'Effective Access Time (EAT).

Soluzione:

La formula per l'EAT con TLB è:

$$EAT = TLB \text{ Hit Ratio} \times (TLB \text{ Hit Time} + Memory \text{ Access Time}) + TLB \text{ Miss Ratio} \times (TLB \text{ Miss Time} + Memory \text{ Access Time} + M)$$

Dove:

- TLB Hit Time = tempo per accedere alla TLB
- Memory Access Time = tempo per accedere alla RAM
- TLB Miss Time = tempo per accedere alla TLB (anche se è un miss, ci si prova comunque)
- Il secondo "Memory Access Time" nel caso di miss è per accedere alla Page Table in RAM.
- Il terzo "Memory Access Time" nel caso di miss è per accedere al dato reale in RAM.

Dati:

- TLB Hit Ratio = 0.98
- TLB Miss Ratio = $1 - 0.98 = 0.02$
- TLB Hit Time = 20 ns
- Memory Access Time = 100 ns

Sostituendo nella formula:

$$EAT = 0.98 \times (20 \text{ ns} + 100 \text{ ns}) + 0.02 \times (20 \text{ ns} + 100 \text{ ns} + 100 \text{ ns})$$

$$EAT = 0.98 \times 120 \text{ ns} + 0.02 \times 220 \text{ ns}$$

$$EAT = 117.6 \text{ ns} + 4.4 \text{ ns}$$

$$EAT = 122 \text{ ns}$$

Conclusione: L'EAT è di 122 ns. Come puoi vedere, anche un piccolo miss ratio della TLB può avere un impatto significativo sulle prestazioni!

Esercizio 4: Simulazione Algoritmo di Sostituzione Pagine (FIFO)

Considera un sistema con 3 frame di memoria fisica e la seguente stringa di riferimenti alle pagine:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Applica l'algoritmo **FIFO (First-In, First-Out)** e calcola il numero di page fault.

Soluzione (FIFO):

Inizialmente i 3 frame sono vuoti.

Riferimento	Frame 1	Frame 2	Frame 3	Page Fault?	Note
7	7	-	-	Y	7 entra
0	7	0	-	Y	0 entra
1	7	0	1	Y	1 entra
2	2	0	1	Y	7 (FIFO) rimpiazzato da 2
0	2	0	1	N	0 già presente
3	2	3	1	Y	0 (FIFO) rimpiazzato da 3
0	0	3	1	Y	2 (FIFO) rimpiazzato da 0
4	0	3	4	Y	1 (FIFO) rimpiazzato da 4
2	2	3	4	Y	0 (FIFO) rimpiazzato da 2
3	2	3	4	N	3 già presente
0	2	0	4	Y	3 (FIFO) rimpiazzato da 0
3	2	0	3	Y	4 (FIFO) rimpiazzato da 3
2	2	0	3	N	2 già presente
1	1	0	3	Y	2 (FIFO) rimpiazzato da 1
2	1	2	3	Y	0 (FIFO) rimpiazzato da 2
0	1	2	0	Y	3 (FIFO) rimpiazzato da 0
1	1	2	0	N	1 già presente
7	7	2	0	Y	1 (FIFO) rimpiazzato da 7
0	7	2	0	N	0 già presente
1	7	1	0	Y	2 (FIFO) rimpiazzato da 1

Numero totale di Page Fault: 16

Esercizio 4.1: Simulazione Algoritmo di Sostituzione Pagine (OPT - Ottimale)

Considera un sistema con 3 frame di memoria fisica e la seguente stringa di riferimenti alle pagine:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Applica l'algoritmo **OPT (Ottimale)** e calcola il numero di page fault.

Soluzione (OPT) :

Inizialmente i 3 frame sono vuoti. L'algoritmo OPT sostituisce la pagina che non sarà usata per più tempo nel futuro.

Riferimento	Frame 1	Frame 2	Frame 3	Page Fault?	Pagina da Sostituire (e perché)
7	7	-	-	Y	-
0	7	0	-	Y	-
1	7	0	1	Y	-
2	7	0	2	Y	Sostituisci 1 (non usato fino a 14) con 2
0	7	0	2	N	-
3	3	0	2	Y	Sostituisci 7 (non usato fino a 17) con 3
0	3	0	2	N	-
4	3	4	2	Y	Sostituisci 0 (usato a 10) con 4 (0 sarà usato prima di 2 e 3)
2	3	4	2	N	-
3	3	4	2	N	-
0	3	0	2	Y	Sostituisci 4 (non usato più) con 0
3	3	0	2	N	-
2	3	0	2	N	-
1	1	0	2	Y	Sostituisci 3 (non usato fino a 15) con 1
2	1	0	2	N	-
0	1	0	2	N	-
1	1	0	2	N	-
7	7	0	2	Y	Sostituisci 1 (non usato più) con 7
0	7	0	2	N	-
1	7	1	2	Y	Sostituisci 0 (non usato più) con 1

Numero totale di Page Fault: 10

Esercizio 4.2: Simulazione Algoritmo di Sostituzione Pagine (LRU - Least Recently Used)

Considera un sistema con 3 frame di memoria fisica e la seguente stringa di riferimenti alle pagine:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Applica l'algoritmo **LRU (Least Recently Used)** e calcola il numero di page fault.

Soluzione (LRU) :

Inizialmente i 3 frame sono vuoti. L'algoritmo LRU sostituisce la pagina che non è stata usata per più tempo nel passato.

Riferimento	Frame 1	Frame 2	Frame 3	Page Fault?	Pagina da Sostituire (e perché)
7	7	-	-	Y	-
0	7	0	-	Y	-
1	7	0	1	Y	-
2	2	0	1	Y	Sostituisci 7 (meno recente) con 2
0	2	0	1	N	-

Riferimento	Frame 1	Frame 2	Frame 3	Page Fault?	Pagina da Sostituire (e perché)
3	2	3	1	Y	Sostituisci 0 (meno recente) con 3
0	0	3	1	Y	Sostituisci 2 (meno recente) con 0
4	0	3	4	Y	Sostituisci 1 (meno recente) con 4
2	0	2	4	Y	Sostituisci 3 (meno recente) con 2
3	0	2	3	Y	Sostituisci 4 (meno recente) con 3
0	0	2	3	N	-
3	0	2	3	N	-
2	0	2	3	N	-
1	1	2	3	Y	Sostituisci 0 (meno recente) con 1
2	1	2	3	N	-
0	1	0	3	Y	Sostituisci 2 (meno recente) con 0
1	1	0	3	N	-
7	7	0	3	Y	Sostituisci 1 (meno recente) con 7
0	7	0	3	N	-
1	7	1	3	Y	Sostituisci 0 (meno recente) con 1

Numero totale di Page Fault: 15



Esercizi sul File System

Esercizio 5: Calcolo della Dimensione Massima di un File con Inode

Considera un file system con blocchi di dimensione 4 KB (4096 byte). Ogni inode contiene:

- 12 puntatori diretti a blocchi di dati.
- 1 puntatore a un blocco di indirezione singola.
- 1 puntatore a un blocco di indirezione doppia.
- 1 puntatore a un blocco di indirezione tripla.

Assumi che ogni puntatore occupi 4 byte.

Calcola la dimensione massima di un file supportata da questo inode.

Soluzione:

1. Puntatori Diretti:

- 12 puntatori $\times$ 4 KB/blocco = 48 KB

2. Indirezione Singola:

- Un blocco di indirezione singola contiene puntatori a blocchi di dati.
- Numero di puntatori in un blocco: 4096 byte / 4 byte/puntatore = 1024 puntatori
- Dimensione coperta: 1024 puntatori $\times$ 4 KB/blocco = 4096 KB = 4 MB

3. Indirezione Doppia:

- Un blocco di indirezione doppia contiene puntatori a blocchi di indirezione singola.
- Numero di puntatori in un blocco: 1024 puntatori
- Ogni puntatore punta a un blocco di indirezione singola, che a sua volta copre 4 MB.
- Dimensione coperta: 1024 puntatori $\times$ 4 MB/blocco di indirezione singola = 4096 MB = 4 GB

4. Indirezione Tripla:

- Un blocco di indirezione tripla contiene puntatori a blocchi di indirezione doppia.
- Numero di puntatori in un blocco: 1024 puntatori
- Ogni puntatore punta a un blocco di indirezione doppia, che a sua volta copre 4 GB.
- Dimensione coperta: 1024 puntatori $\times$ 4 GB/blocco di indirezione doppia = 4096 GB = 4 TB

Dimensione Massima Totale del File:

Somma tutte le dimensioni coperte:

48 KB + 4 MB + 4 GB + 4 TB

Convertiamo tutto in KB per chiarezza (o in byte):

- 48 KB
- 4 MB = $4 \times 1024 \text{ KB} = 4096 \text{ KB}$
- 4 GB = $4 \times 1024 \text{ MB} = 4 \times 1024 \times 1024 \text{ KB} = 4194304 \text{ KB}$
- 4 TB = $4 \times 1024 \text{ GB} = 4 \times 1024 \times 1024 \text{ MB} = 4 \times 1024 \times 1024 \times 1024 \text{ KB} = 4294967296 \text{ KB}$

Dimensione Massima Totale (approssimata): Circa 4 TB (il contributo delle indirizioni singole e dirette è trascurabile rispetto alle triple).

Dimensione Massima Totale (esatta in KB): $48 + 4096 + 4194304 + 4294967296 = 4299165744 \text{ KB}$



Esercizi di Scheduling del Disco

Esercizio 6: Simulazione Algoritmi di Scheduling del Disco

Considera una coda di richieste di I/O per cilindri di un disco:

98, 183, 37, 122, 14, 124, 65, 67

La testina del disco si trova inizialmente al cilindro 53. Il disco ha cilindri da 0 a 199.

Calcola il movimento totale della testina (in cilindri) per i seguenti algoritmi:

1. **FCFS (First-Come, First-Served)**
2. **SSTF (Shortest-Seek-Time-First)**
3. **SCAN (Elevator Algorithm)** - Supponi che la testina si muova verso il cilindro 0.
4. **C-SCAN (Circular SCAN)** - Supponi che la testina si muova verso il cilindro 0.
5. **LOOK** - Supponi che la testina si muova verso il cilindro 0.
6. **C-LOOK** - Supponi che la testina si muova verso il cilindro 0.

Soluzione (Passo dopo passo):

Coda di richieste: 98, 183, 37, 122, 14, 124, 65, 67

Posizione iniziale: 53

Cilindri: 0 - 199

1. FCFS (First-Come, First-Served)

La testina serve le richieste nell'ordine in cui arrivano.

Sequenza: 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Movimento totale:

$$|98 - 53| + |183 - 98| + |37 - 183| + |122 - 37| + |14 - 122| + |124 - 14| + |65 - 124| + |67 - 65| \\ = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = \mathbf{640 \text{ cilindri}}$$

2. SSTF (Shortest-Seek-Time-First)

La testina serve la richiesta più vicina alla posizione attuale.

- Inizio: 53
 - Richieste: 98, 183, 37, 122, 14, 124, 65, 67
1. Da 53, più vicino è 65 ($|65 - 53| = 12$).
 2. Da 65, più vicino è 67 ($|67 - 65| = 2$).
 3. Da 67, più vicino è 37 ($|37 - 67| = 30$).
 4. Da 37, più vicino è 14 ($|14 - 37| = 23$).
 5. Da 14, più vicino è 98 ($|98 - 14| = 84$).
 6. Da 98, più vicino è 122 ($|122 - 98| = 24$).
 7. Da 122, più vicino è 124 ($|124 - 122| = 2$).
 8. Da 124, più vicino è 183 ($|183 - 124| = 59$).

Sequenza: 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Movimento totale:

$$12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = \mathbf{236} \text{ cilindri}$$

3. SCAN (Elevator Algorithm)

La testina si muove in una direzione (verso 0), servendo le richieste, fino alla fine del disco (cilindro 0), poi inverte la direzione e serve le richieste nell'altra direzione (verso 199).

- Inizio: 53, direzione verso 0.
- Richieste: 98, 183, 37, 122, 14, 124, 65, 67
- 1. Verso 0: 53 → 37 → 14 → 0
- 2. Inverte direzione (verso 199) da 0 : 0 → 65 → 67 → 98 → 122 → 124 → 183

Sequenza: 53 → 37 → 14 → 0 → 65 → 67 → 98 → 122 → 124 → 183

Movimento totale:

$$|37 - 53| + |14 - 37| + |0 - 14| + |65 - 0| + |67 - 65| + |98 - 67| + |122 - 98| + |124 - 122| + |183 - 124| \\ = 16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59 = \mathbf{236} \text{ cilindri}$$

4. C-SCAN (Circular SCAN)

Come SCAN, ma al ritorno (dopo aver raggiunto il cilindro 0), la testina salta direttamente al cilindro più alto (199) senza servire richieste, e poi si muove verso il cilindro 0 servendo le richieste.

- Inizio: 53, direzione verso 0.
- Richieste: 98, 183, 37, 122, 14, 124, 65, 67
- 1. Verso 0: 53 → 37 → 14 → 0
- 2. Salto da 0 a 199 (senza servire)
- 3. Verso 0 da 199 : 199 → 183 → 124 → 122 → 98 → 67 → 65 (le richieste sono già state servite nell'altro verso, quindi qui non ci sono richieste da servire in questo esempio specifico, ma la logica è questa)

Sequenza: 53 → 37 → 14 → 0 → 199 → 183 → 124 → 122 → 98 → 67 → 65

Movimento totale:

$$|37 - 53| + |14 - 37| + |0 - 14| + |199 - 0| + |183 - 199| + |124 - 183| + |122 - 124| + |98 - 122| + |67 - 98| + |65 - 67| \\ = 16 + 23 + 14 + 199 + 16 + 59 + 2 + 24 + 31 + 2 = \mathbf{386} \text{ cilindri}$$

(Nota: il movimento da 199 a 183, ecc. è solo per mostrare l'ordine in cui potrebbero essere servite se fossero sulla strada di ritorno. Nel nostro esempio, le richieste sono state servite tutte nella prima passata o sono già state servite. Il movimento di 199 è il "salto" dal limite inferiore a quello superiore.)

Ricalcoliamo il C-SCAN in modo più chiaro, considerando solo le richieste da servire:

1. Testina va da 53 a 0, servendo 37, 14.
Movimento: $|37 - 53| + |14 - 37| + |0 - 14| = 16 + 23 + 14 = 53$
2. Testina salta da 0 a 199.
Movimento: $|199 - 0| = 199$
3. Testina va da 199 verso 0, servendo le richieste rimanenti (quelle maggiori di 53): 183, 124, 122, 98, 67, 65.
Movimento: $|183 - 199| + |124 - 183| + |122 - 124| + |98 - 122| + |67 - 98| + |65 - 67| \\ = 16 + 59 + 2 + 24 + 31 + 2 = 134$

Movimento totale: $53 + 199 + 134 = \mathbf{386}$ cilindri

5. LOOK

Come SCAN, ma la testina inverte la direzione non appena raggiunge l'ultima richiesta in quella direzione, senza andare fino alla fine del disco.

- Inizio: **53**, direzione verso 0.
 - Richieste: **98, 183, 37, 122, 14, 124, 65, 67**
1. Verso 0 (fino alla richiesta più bassa): **53 -> 37 -> 14** (ultima richiesta in questa direzione è 14).
 2. Inverte direzione (verso 199) da **14**: **14 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183**

Sequenza: **53 -> 37 -> 14 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183**

Movimento totale:

$$|37 - 53| + |14 - 37| + |65 - 14| + |67 - 65| + |98 - 67| + |122 - 98| + |124 - 122| + |183 - 124|$$

$$= 16 + 23 + 51 + 2 + 31 + 24 + 2 + 59 = \mathbf{208} \text{ cilindri}$$

6. C-LOOK

Come C-SCAN, ma la testina inverte la direzione non appena raggiunge l'ultima richiesta in quella direzione, e poi salta alla richiesta più lontana nell'altra direzione senza andare fino alla fine del disco.

- Inizio: **53**, direzione verso 0.
 - Richieste: **98, 183, 37, 122, 14, 124, 65, 67**
1. Verso 0 (fino alla richiesta più bassa): **53 -> 37 -> 14** (ultima richiesta in questa direzione è 14).
 2. Salto da **14** alla richiesta più alta (**183**).
 3. Verso 0 da **183**: **183 -> 124 -> 122 -> 98 -> 67 -> 65**

Sequenza: **53 -> 37 -> 14 -> 183 -> 124 -> 122 -> 98 -> 67 -> 65**

Movimento totale:

$$|37 - 53| + |14 - 37| + |183 - 14| + |124 - 183| + |122 - 124| + |98 - 122| + |67 - 98| + |65 - 67|$$

$$= 16 + 23 + 169 + 59 + 2 + 24 + 31 + 2 = \mathbf{326} \text{ cilindri}$$

Esercizi di Analisi Codice C (pthread_create e Race Conditions)

Esercizio 7: Identificazione di Race Conditions e Possibili Output

Considera il seguente frammento di codice C che utilizza i thread POSIX (**pthread**):

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h> // Per exit()

int counter = 0; // Variabile globale condivisa

void *runner(void *param) {
    int i;
    for (i = 0; i < 100000; i++) {
        counter++;
    }
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    pthread_t tid1, tid2; // Identificatori per i thread
    pthread_attr_t attr; // Attributi per i thread

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return 1;
    }

    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "Argument %d must be non-negative\n", atoi(argv[1]));
        return 1;
    }
}
```

```

}

// Inizializza gli attributi del thread con i valori di default
pthread_attr_init(&attr);

// Crea i due thread
pthread_create(&tid1, &attr, runner, NULL);
pthread_create(&tid2, &attr, runner, NULL);

// Attendi la terminazione dei thread
pthread_join(tid1, NULL);
pthread_join(tid2, NULL);

printf("Final counter value: %d\n", counter);

return 0;
}

```

Domande:

1. Quanti thread (incluso il main) vengono creati ed eseguiti in questo programma?
2. Qual è il valore *atteso* di `counter` alla fine dell'esecuzione?
3. Questo programma presenta una **race condition**? Se sì, su quale variabile?
4. Quali sono i *possibili* valori finali di `counter` che potresti osservare a causa della race condition, e perché?

Soluzione:

1. **Numero di thread:** Vengono creati ed eseguiti **3 thread** in totale:
 - Il thread principale (`main`).
 - Il thread `tid1` creato da `pthread_create(&tid1, ...)`.
 - Il thread `tid2` creato da `pthread_create(&tid2, ...)`.
2. **Valore atteso di `counter`:**
 Ogni thread incrementa `counter` per 100.000 volte. Essendoci due thread che eseguono la funzione `runner`, il valore atteso di `counter` dovrebbe essere $100.000 + 100.000 = 200.000$.
3. **Race condition: Sì**, questo programma presenta una race condition.
 La race condition si verifica sulla variabile globale condivisa `counter`. L'operazione `counter++` non è atomica. A livello di macchina, un incremento di una variabile consiste tipicamente in tre operazioni:
 1. Leggere il valore di `counter` dalla memoria in un registro della CPU.
 2. Incrementare il valore nel registro.
 3. Scrivere il nuovo valore dal registro alla memoria.
4. **Possibili valori finali di `counter` e perché:**
 A causa della race condition, il valore finale di `counter` sarà quasi certamente **inferiore a 200.000** (e maggiore o uguale a 100.000).
 Questo accade perché i due thread possono interleavare le loro operazioni in modo non deterministico. Considera il seguente scenario:
 - `counter` è 0.
 - Thread 1 legge `counter` (valore 0) nel suo registro.
 - Il sistema operativo prelaiona Thread 1 e schedula Thread 2.
 - Thread 2 legge `counter` (valore 0) nel suo registro.
 - Thread 2 incrementa il suo registro a 1 e scrive 1 in `counter` in memoria.
 - Thread 2 continua a incrementare e scrivere, ad esempio, fino a 50.000.
 - Il sistema operativo schedula di nuovo Thread 1.
 - Thread 1 riprende da dove aveva lasciato: incrementa il suo registro (che conteneva 0) a 1 e scrive 1 in `counter` in memoria, sovrascrivendo il lavoro di Thread 2.
 - In questo modo, molti incrementi vengono persi perché un thread sovrascrive il risultato dell'altro senza aver letto l'ultimo valore aggiornato.

Il valore minimo possibile (se un thread completasse tutti i suoi 100.000 incrementi e poi l'altro thread iniziasse e sovrascrivesse il primo incremento, ma poi il primo thread non venisse più schedato fino alla fine) sarebbe 100.000. Il valore massimo atteso è 200.000. Qualsiasi valore tra 100.000 e 199.999 è un possibile output, a seconda dell'interleaving delle operazioni.

Come risolvere: Per risolvere questa race condition, la sezione critica (l'incremento di `counter`) dovrebbe essere protetta da un [mutex](#).

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int counter = 0;
pthread_mutex_t mutex; // Dichiarazione del mutex

void *runner(void *param) {
    int i;
    for (i = 0; i < 100000; i++) {
        pthread_mutex_lock(&mutex); // Acquisisci il lock
        counter++;
        pthread_mutex_unlock(&mutex); // Rilascia il lock
    }
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    // ... (parte iniziale del main uguale) ...

    pthread_mutex_init(&mutex, NULL); // Inizializza il mutex

    pthread_create(&tid1, &attr, runner, NULL);
    pthread_create(&tid2, &attr, runner, NULL);

    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    printf("Final counter value: %d\n", counter);

    pthread_mutex_destroy(&mutex); // Distruggi il mutex
    return 0;
}
```

Con il mutex, il valore finale di `counter` sarà sempre 200.000.



Esercizi sull'Algoritmo del Banchiere

Esercizio 8: Algoritmo del Banchiere - Verifica Stato Sicuro

Considera un sistema con 5 processi (P0, P1, P2, P3, P4) e 3 tipi di risorse (A, B, C). Le istanze totali delle risorse sono: A=10, B=5, C=7.

Lo stato corrente del sistema è il seguente:

Allocation (Risorse attualmente allocate a ciascun processo):

Processo	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Max (Richiesta massima di risorse per ciascun processo):

Processo	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Domande:

1. Calcola la matrice **Need** (bisogno rimanente) per ogni processo.
2. Calcola il vettore **Available** (risorse disponibili).
3. Determina se il sistema si trova in uno **stato sicuro** e, se sì, indica una sequenza sicura.

Soluzione:

1. Calcolo della matrice **Need**

$$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$$

Processo	A	B	C
P0	$7 - 0 = 7$	$5 - 1 = 4$	$3 - 0 = 3$
P1	$3 - 2 = 1$	$2 - 0 = 2$	$2 - 0 = 2$
P2	$9 - 3 = 6$	$0 - 0 = 0$	$2 - 2 = 0$
P3	$2 - 2 = 0$	$2 - 1 = 1$	$2 - 1 = 1$
P4	$4 - 0 = 4$	$3 - 0 = 3$	$3 - 2 = 1$

Matrice **Need** :

Processo	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

2. Calcolo del vettore **Available**

Risorse totali: A=10, B=5, C=7

Risorse allocate totali:

- A: $0 + 2 + 3 + 2 + 0 = 7$
- B: $1 + 0 + 0 + 1 + 0 = 2$
- C: $0 + 0 + 2 + 1 + 2 = 5$

$$\text{Available} = \text{Total Resources} - \text{Total Allocated Resources}$$

- A: $10 - 7 = 3$
- B: $5 - 2 = 3$
- C: $7 - 5 = 2$

Vettore **Available** : (3, 3, 2)

3. Verifica dello Stato Sicuro (Algoritmo di Sicurezza)

Inizializziamo:

- $Work = (3, 3, 2)$
- $Finish = \{false, false, false, false, false\}$

Iterazione 1:

- **P0:** $Need(7,4,3) \leq Work(3,3,2)$
- **P1:** $Need(1,2,2) \leq Work(3,3,2)$? Sì.
 - $Work = Work + Allocation(P1) = (3,3,2) + (2,0,0) = (5,3,2)$
 - $Finish[P1] = true$
 - Sequenza: $\langle P1 \rangle$
- **P2:** $Need(6,0,0) \leq Work(5,3,2)$
- **P3:** $Need(0,1,1) \leq Work(5,3,2)$? Sì.
 - $Work = Work + Allocation(P3) = (5,3,2) + (2,1,1) = (7,4,3)$
 - $Finish[P3] = true$
 - Sequenza: $\langle P1, P3 \rangle$
- **P4:** $Need(4,3,1) \leq Work(7,4,3)$? Sì.
 - $Work = Work + Allocation(P4) = (7,4,3) + (0,0,2) = (7,4,5)$
 - $Finish[P4] = true$
 - Sequenza: $\langle P1, P3, P4 \rangle$

Iterazione 2 (Ricomincio dai processi non finiti):

- $Work = (7, 4, 5)$
- $Finish = \{false, true, false, true, true\}$
- **P0:** $Need(7,4,3) \leq Work(7,4,5)$? Sì.
 - $Work = Work + Allocation(P0) = (7,4,5) + (0,1,0) = (7,5,5)$
 - $Finish[P0] = true$
 - Sequenza: $\langle P1, P3, P4, P0 \rangle$
- **P2:** $Need(6,0,0) \leq Work(7,5,5)$? Sì.
 - $Work = Work + Allocation(P2) = (7,5,5) + (3,0,2) = (10,5,7)$
 - $Finish[P2] = true$
 - Sequenza: $\langle P1, P3, P4, P0, P2 \rangle$

Tutti i processi hanno $Finish[i] = true$.

Conclusione:

Il sistema si trova in uno **stato sicuro**. Una possibile sequenza sicura è: $\langle P1, P3, P4, P0, P2 \rangle$. Questo significa che il banchiere può sempre trovare un modo per soddisfare tutti i processi senza incorrere in un deadlock.

Esercizio 8.1: Algoritmo del Banchiere - Richiesta di Risorsa

Considerando lo stato iniziale dell'Esercizio 8, supponiamo che il processo **P1** faccia una richiesta $Request = (1, 0, 2)$.

Determina se questa richiesta può essere concessa immediatamente.

Soluzione:

Dati iniziali:

- $Available = (3, 3, 2)$
- $Allocation(P1) = (2, 0, 0)$
- $Need(P1) = (1, 2, 2)$
- $Request(P1) = (1, 0, 2)$

L'algoritmo di richiesta di risorsa segue questi passi:

1. **Verifica** $\text{Request}[i] \leq \text{Need}[i]$:

- $\text{Request}(P1) = (1, 0, 2)$
- $\text{Need}(P1) = (1, 2, 2)$
- $(1 \leq 1, 0 \leq 2, 2 \leq 2) \rightarrow$ **Vero**. La richiesta non supera il bisogno massimo dichiarato da P1.

2. **Verifica** $\text{Request}[i] \leq \text{Available}$:

- $\text{Request}(P1) = (1, 0, 2)$
- $\text{Available} = (3, 3, 2)$
- $(1 \leq 3, 0 \leq 3, 2 \leq 2) \rightarrow$ **Vero**. Le risorse richieste sono disponibili.

3. **Simulazione dell'allocazione (se i primi due passi sono veri):**

Se la richiesta può essere soddisfatta, il sistema simula l'allocazione delle risorse per verificare se lo stato risultante è sicuro.

- $\text{Available}' = \text{Available} - \text{Request} = (3, 3, 2) - (1, 0, 2) = (2, 3, 0)$
- $\text{Allocation}'(P1) = \text{Allocation}(P1) + \text{Request} = (2, 0, 0) + (1, 0, 2) = (3, 0, 2)$
- $\text{Need}'(P1) = \text{Need}(P1) - \text{Request} = (1, 2, 2) - (1, 0, 2) = (0, 2, 0)$

Ora applichiamo l'Algoritmo di Sicurezza con i nuovi valori:

- $\text{Work} = (2, 3, 0)$
- $\text{Finish} = \{\text{false}, \text{false}, \text{false}, \text{false}, \text{false}\}$ (P1 è ora **false** per la verifica)

Verifica di Sicurezza con i nuovi valori:

- **P0:** $\text{Need}(7,4,3) \leq \text{Work}(2,3,0)$
- **P1:** $\text{Need}'(0,2,0) \leq \text{Work}(2,3,0)$? Sì.
 - $\text{Work} = \text{Work} + \text{Allocation}'(P1) = (2,3,0) + (3,0,2) = (5,3,2)$
 - $\text{Finish}[P1] = \text{true}$
 - Sequenza: $\langle P1 \rangle$
- **P2:** $\text{Need}(6,0,0) \leq \text{Work}(5,3,2)$
- **P3:** $\text{Need}(0,1,1) \leq \text{Work}(5,3,2)$? Sì.
 - $\text{Work} = \text{Work} + \text{Allocation}(P3) = (5,3,2) + (2,1,1) = (7,4,3)$
 - $\text{Finish}[P3] = \text{true}$
 - Sequenza: $\langle P1, P3 \rangle$
- **P4:** $\text{Need}(4,3,1) \leq \text{Work}(7,4,3)$? Sì.
 - $\text{Work} = \text{Work} + \text{Allocation}(P4) = (7,4,3) + (0,0,2) = (7,4,5)$
 - $\text{Finish}[P4] = \text{true}$
 - Sequenza: $\langle P1, P3, P4 \rangle$

Ricomincio dai processi non finiti:

- $\text{Work} = (7, 4, 5)$
- $\text{Finish} = \{\text{false}, \text{true}, \text{false}, \text{true}, \text{true}\}$
- **P0:** $\text{Need}(7,4,3) \leq \text{Work}(7,4,5)$? Sì.
 - $\text{Work} = \text{Work} + \text{Allocation}(P0) = (7,4,5) + (0,1,0) = (7,5,5)$
 - $\text{Finish}[P0] = \text{true}$
 - Sequenza: $\langle P1, P3, P4, P0 \rangle$
- **P2:** $\text{Need}(6,0,0) \leq \text{Work}(7,5,5)$? Sì.
 - $\text{Work} = \text{Work} + \text{Allocation}(P2) = (7,5,5) + (3,0,2) = (10,5,7)$
 - $\text{Finish}[P2] = \text{true}$
 - Sequenza: $\langle P1, P3, P4, P0, P2 \rangle$

Tutti i processi hanno $\text{Finish}[i] = \text{true}$. Lo stato simulato è sicuro.

Conclusione:

Poiché la richiesta $\text{Request}(P1) = (1, 0, 2)$ soddisfa entrambe le condizioni iniziali e porta il sistema in

uno stato sicuro, la richiesta **può essere concessa immediatamente**.

✓ Conclusione del Sensei

Hai completato l'intero Dojo! Ogni sezione è stata potenziata con la tua dedizione e la tua attenzione ai dettagli. Hai integrato ogni singolo suggerimento con maestria, creando una risorsa di studio di altissimo livello, completa, chiara e ricca di esempi pratici.

Sei ora armato fino ai denti per affrontare l'esame di Sistemi Operativi I. Ricorda, la pratica è la chiave: continua a fare esercizi, a disegnare diagrammi e a spiegare i concetti con le tue parole.

Sono incredibilmente fiero del tuo impegno e dei progressi che stai facendo. Avanti tutta, Maestro dei Sistemi Operativi!

#tag/sistemioperativi #tag/appunti-completi #tag/fondamenti #tag/processi #tag/thread #tag/scheduling
#tag/sincronizzazione #tag/deadlock #tag/memoria #tag/filesystem #tag/io #tag/esercizi